

Grado en Ingeniería Informática.
2025-2026

Trabajo de Fin de Grado

“Plataforma Colaborativa para el Intercambio y Corrección Lingüística”

M^a Isabel Hernández Barrio

Jesús Hernando Corrochano
Madrid, junio de 2026



This work is licensed under Creative Commons **Attribution – Non Commercial – Non Derivatives**

Abstract

Nowadays, the digital transformation of education and language learning has become essential for global communication and international activity. In this context, the creation of a specialized web application for language exchange through written correspondence is presented as an innovative solution to bridge the gap between theoretical knowledge and practical, collaborative learning.

The aim of this Final Degree Project is to design and develop a web platform, named Letterex, that facilitates language exchange between users worldwide. The tool seeks to provide an efficient and interactive environment where users can practice writing in foreign languages and receive peer-to-peer corrections. Additionally, the application includes a personal space for users to manage their profiles and build a social network based on shared learning interests.

The process of designing and developing this application involved a detailed analysis of the specific needs of language learners, as well as the implementation of a modern methodology and tech stack. Built using the MERN stack (MongoDB, Express, React, and Node.js), the platform incorporates features such as a secure authentication system via JWT, a module for creating and organizing letters into diaries, a shared correction interface, and a social recommendation algorithm based on language affinity.

To sum up, this project aims to offer a technological solution tailored to the requirements of modern language learners, optimizing the writing and correcting process and the improvement of linguistic skills through an active and supportive community.

Keywords: Language exchange, Web development, MERN stack, Peer-to-peer correction, MongoDB, REST API.

AGRADECIMIENTOS

Gracias a mi familia, por su apoyo incondicional a lo largo de mis estudios, ayudándome en cada caída y celebrando cada logro.

Gracias a mis amigos, en especial Luis, David y Viku, por hacer de estos años universitarios una aventura llena de risas, viajes y triángulos de chocolate.

Gracias a mis amigos del Erasmus por inspirar este proyecto, a mi tutor Jesús por acompañarme en su realización, y a mi pareja Miriam y su amiga Ana por revisar las faltas de ortografía.

ÍNDICE

LISTA DE ABREVIATURAS	xviii
1. INTRODUCCIÓN.	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Marco regulador	3
1.4. Medios empleados	3
1.5. Estructura del documento	4
2. ESTADO DEL ARTE.	6
2.1. Tecnologías para el Desarrollo de Aplicaciones Web	6
2.1.1. Frontend.	6
2.1.2. Frameworks de JavaScript.	9
2.1.3. Backend.	12
2.1.4. Consumo de APIs	16
2.1.5. Base de datos	19
2.2. Metodología	22
2.3. Negocio	24
2.3.1. HiNative	24
2.3.2. Tandem	25
2.3.3. Grammarly	25
2.3.4. Duolingo	25
2.4. Justificación de la solución	26
3. ANÁLISIS Y PLANIFICACIÓN	27
3.1. Historias de Usuario	27
3.1.1. Investigación y Documentación.	28
3.1.2. Gestión de Usuarios	31
3.1.3. Gestión de Perfiles	32
3.1.4. Módulo de Correspondencia	34
3.1.5. Módulo de Correcciones.	36

3.1.6. Funcionalidad Social.	38
3.2. Pruebas de Aceptación.	41
3.2.1. Investigación y Documentación.	41
3.2.2. Gestión de Usuarios	42
3.2.3. Gestión de Perfiles	45
3.2.4. Módulo de Correspondencia	48
3.2.5. Módulo de Correcciones.	51
3.2.6. Funcionalidad Social.	53
3.3. Matriz de Trazabilidad.	57
3.4. Planificación	58
4. ARQUITECTURA DEL SOFTWARE	61
4.1. Stack tecnológico seleccionado	62
4.2. Modelado de datos	63
4.3. Arquitectura del backend	65
4.3.1. Estructura del backend.	65
4.3.2. Emails automatizados	67
4.3.3. Mecanismo de Autenticación y Autorización	67
4.3.4. Almacenamiento de imágenes con Multer y Cloudinary	69
4.3.5. Interfaz de Programación de Aplicaciones (API): Rutas y Controladores	70
4.3.6. Técnicas de optimización	74
4.4. Frontend de la aplicación	76
4.4.1. Stack Tecnológico	76
4.4.2. Arquitectura del Frontend y Gestión del Estado.	77
4.4.3. Páginas de la aplicación	78
4.4.4. Técnicas de Optimización	85
5. ENTORNO SOCIOECONÓMICO	87
5.1. Presupuesto	87
5.2. Impacto socioeconómico	89
6. CONCLUSIONES	91
6.1. Cumplimiento de Objetivos	91
6.2. Líneas de Trabajo Futuro y Propuestas de Mejora	91

6.3. Conclusión final	92
BIBLIOGRAFÍA	94

LISTA DE FIGURAS

2.1	Logo de HTML5	7
2.2	Logo de Tailwind	8
2.3	Logo de CSS	8
2.4	Logo de JavaScript	9
2.5	Logo de TypeScript	9
2.6	Logo de React	10
2.7	Logo de Next	11
2.8	Logo de Vue	11
2.9	Logo de Angular	11
2.10	Logo de NodeJS	12
2.11	Logo de Python	13
2.12	Logo de Django	13
2.13	Logo de Java	14
2.14	Logo de SpringBoot	14
2.15	Logo de PHP	14
2.16	Logo de Laravel	15
2.17	Logo de Ruby	15
2.18	Logo de Ruby On Rails	15
2.19	Logo de Go	16
2.20	Logo de Gin	16
2.21	Logo de Axios	17
2.22	Logo de FetchAPI	17
2.23	Logo de Postman	18
2.24	Logo de Insomnia	18
2.25	Logo de cURL	19
2.26	Logo de MySQL	20
2.27	Logo de Postgresql	20
2.28	Logo de MariaDB	21

2.29	Logo de MongoDB	21
2.30	Logo de Firebase	22
2.31	Logo de Cassandra	22
2.32	Logo de HiNative	24
2.33	Logo de Tandem	25
2.34	Logo de Grammarly	25
2.35	Logo de Duolingo	26
3.1	Ejemplo de sprint en Trello	60
4.1	Arquitectura full-stack	61
4.2	Estructura de Datos	64
4.3	Configuración del CORS en la entrada del backend	66
4.4	Email automatizado	67
4.5	JSON Web Tokens	68
4.6	Configuración del proceso de subida en Cloudinary	70
4.7	Código de apertura del diálogo y resultado en la interfaz	78
4.8	Página de inicio de sesión y registro	79
4.9	Página principal	80
4.10	Página de creación de cartas	81
4.11	Página de corrección de cartas	82
4.12	Página de visualización de corrección recibida	83
4.13	Página de amigos	83
4.14	Página de perfil de usuario	84
4.15	Ejemplo de Pantalla Esqueleto	86

LISTA DE TABLAS

3.1	HU-01	28
3.2	HU-02	28
3.3	HU-03	29
3.4	HU-04	29
3.5	HU-05	29
3.6	HU-06	30
3.7	HU-07	30
3.8	HU-08	31
3.9	HU-09	31
3.10	HU-10	32
3.11	HU-11	32
3.12	HU-12	33
3.13	HU-13	33
3.14	HU-14	34
3.15	HU-15	34
3.16	HU-16	34
3.17	HU-17	35
3.18	HU-18	35
3.19	HU-19	36
3.20	HU-20	36
3.21	HU-21	37
3.22	HU-22	37
3.23	HU-23	37
3.24	HU-24	38
3.25	HU-25	38
3.26	HU-26	39
3.27	HU-27	39
3.28	HU-28	40

3.29 PA-01	41
3.30 PA-02	41
3.31 PA-03	42
3.32 PA-04	43
3.33 PA-05	43
3.34 PA-06	44
3.35 PA-07	44
3.36 PA-08	45
3.37 PA-10	45
3.38 PA-11	46
3.39 PA-12	46
3.40 PA-13	47
3.41 PA-14	47
3.42 PA-15	48
3.43 PA-16	49
3.44 PA-17	49
3.45 PA-18	50
3.46 PA-19	50
3.47 PA-20	51
3.48 PA-21	51
3.49 PA-22	52
3.50 PA-23	52
3.51 PA-24	53
3.52 PA-25	53
3.53 PA-26	54
3.54 PA-27	54
3.55 PA-28	55
3.56 PA-29	55
3.57 PA-30	56
3.58 Matriz de trazabilidad	57
3.59 Resumen de la planificación por sprints	59

4.1	Rutas de la API de Letterex	74
5.1	Costes de herramientas	88
5.2	Costes humanos	88
5.3	Resumen del coste del proyecto	89
6.1	Cumplimiento de objetivos	91

LISTA DE ABREVIATURAS

API	Interfaz de Programación de Aplicaciones (Application Programming Interface).
AJAX	Asynchronous JavaScript and XML.
AKS	Azure Kubernetes Service.
AWS	Amazon Web Services.
BOE	Boletín Oficial del Estado.
BSON	Binary JSON.
CORS	Cross-Origin Resource Sharing.
CRA	Cyber Resilience Act (Ley de Ciberresiliencia).
CRUD	Create, Read, Update, Delete.
CSRF	Cross-Site Request Forgery.
CSS	Cascading Style Sheets.
DOM	Document Object Model.
GCP	Google Cloud Platform.
HTML	HyperText Markup Language.
HTTP	HyperText Transfer Protocol.
IA	Inteligencia Artificial.
IEEE	Institute of Electrical and Electronics Engineers.
JSON	JavaScript Object Notation.
JWT	JSON Web Token.
LAMP	Linux, Apache, MySQL y PHP/Python.
LOPDGDD	Ley Orgánica de Protección de Datos Personales y garantía de los derechos digitales.
LSSICE	Ley de Servicios de la Sociedad de la Información y de Comercio Electrónico.
MERN	MongoDB, Express, React y Node.js.
MIME	Multipurpose Internet Mail Extensions.
MVCC	Control de Concurrencia Multiversión (Multi-Version Concurrency Control).
NoSQL	Not Only SQL.
ODM	Object Document Mapper.
ORM	Object Relational Mapper.
OTP	One-Time Password.
PDF	Portable Document Format.
REST	Representational State Transfer.
RGPD	Reglamento General de Protección de Datos.
SCRUM	Marco ágil de trabajo basado en iteraciones y sprints.
SEO	Search Engine Optimization.

SMTP	Simple Mail Transfer Protocol.
SPA	Single Page Application.
SQL	Structured Query Language.
SSR	Server-Side Rendering.
TFG	Trabajo de Fin de Grado.
TTL	Time To Live.
UE	Unión Europea.
URL	Uniform Resource Locator.
XML	eXtensible Markup Language.
XP	Extreme Programming.
XSS	Cross-Site Scripting.

1. INTRODUCCIÓN

Hoy en día, la implementación de la tecnología juega un papel esencial en el ámbito educativo y social, hasta el punto en que se ha convertido en una de las herramientas más populares en numerosos campos, entre ellos el aprendizaje de idiomas y la comunicación entre personas de diferentes países. En este contexto, la creación de una aplicación web de intercambio de idiomas se presenta como una solución innovadora y efectiva para practicar la expresión mediante la corrección colaborativa entre usuarios.

Las aplicaciones de intercambio de idiomas existentes se centran, en muchos casos, en el chat instantáneo, en videollamadas o en ejercicios preparados. Sin embargo, la práctica de la expresión escrita suele quedar en un segundo plano, a pesar de ser una habilidad fundamental tanto en contextos académicos como profesionales. La corrección detallada de textos requiere tiempo y estructura, lo cual construye una base sólida a la hora de aprender un nuevo idioma.

El objetivo de este Trabajo de Fin de Grado es diseñar y desarrollar una aplicación web que permita la gestión integral de textos escritos por usuarios que desean practicar idiomas, así como el envío de dichos textos a otros usuarios para su corrección y devolución. Se trata de una plataforma que busca ser eficiente, práctica y agradable de usar; que facilite el acceso al aprendizaje mediante un método personal, cercano y social. Al basarse en la redacción, este sistema se presta para ser un lugar donde expresarse, compartir (o simplemente articular) ideas, y abrir paso a la imaginación y a la colección de entradas escritas por uno mismo que no solo podrán servir para la práctica de un idioma sino para conectar y colaborar con usuarios de todo el mundo.

En definitiva, se busca ofrecer una solución tecnológica que fomente el aprendizaje de idiomas mediante la escritura y la corrección entre pares, mejorando la calidad de la práctica escrita y facilitando la conexión entre personas con intereses comunes.

1.1. Motivación

Desarrollar una aplicación de intercambio de idiomas centrada en la redacción y corrección de textos resulta especialmente interesante por varios motivos.

En primer lugar, responde a una necesidad real de muchos estudiantes de idiomas que buscan practicar la escritura y recibir correcciones de hablantes más avanzados o nativos, sin depender exclusivamente de profesores o academias. Presentar esta práctica en formato de red social lo hace considerablemente más atractivo para las nuevas generaciones, ya que esta se torna más amena, interactiva y personal.

En segundo lugar, el proyecto permite aplicar y consolidar conocimientos técnicos adquiridos durante el grado, así como ampliarlos mediante el aprendizaje de nuevos contenidos,

utilizando tecnologías actuales del ecosistema JavaScript como React, Next.js, Node.js, Express, Mongoose y MongoDB. Dichas tecnologías serán presentadas y estudiadas en más profundidad más adelante en este documento, y su uso supone una oportunidad para trabajar con una arquitectura moderna, orientada a servicios y preparada para escalar.

Por último, la motivación también es personal: crear una herramienta que tenga valor no solo académico, sino que pueda más adelante ser utilizada por usuarios reales, generando comunidad y facilitando el aprendizaje colaborativo de idiomas, uno de mis principales intereses personales.

1.2. Objetivos

Los objetivos principales de este proyecto son los de desarrollar una aplicación web que resulte útil para usuarios que desean practicar idiomas mediante la escritura y la corrección colaborativa. De esta manera se pretende mejorar el proceso de aprendizaje, proporcionando una forma estructurada de enviar textos, recibir correcciones y gestionar las relaciones con otros usuarios.

Los objetivos de este Trabajo de Fin de Grado son los siguientes:

- Diseñar e implementar un sistema de autenticación de usuarios que incluya registro, inicio de sesión y recuperación de contraseña.
- Desarrollar una bandeja de cartas enviadas y recibidas, con posibilidad de ver el estado de corrección y filtrar por diferentes campos para mejor accesibilidad.
- Diseñar una interfaz para crear y editar cartas, y enviarlas a amigos para su corrección.
- Desarrollar una página específica para corregir cartas recibidas y reenviarlas al autor con las modificaciones pertinentes.
- Implementar un sistema de relaciones entre usuarios que permita enviar y aceptar solicitudes de amistad, así como gestionar la lista de contactos y explorar usuarios nuevos que agregar.
- Crear una página de perfil de usuario con datos editables y opciones de cambio de contraseña y borrado de cuenta.
- Garantizar una arquitectura limpia, mantenible y escalable, utilizando buenas prácticas y un diseño óptimo.
- Seguir una metodología de desarrollo de software que permita trabajar de forma clara y organizada.
- Documentar el proceso de ideación, planificación, desarrollo y resultados.

1.3. Marco regulador

Al tratarse de una aplicación que estaría alojada en Internet y que gestiona datos personales de usuarios (correo electrónico, nombre, localidad, y otros datos de perfil), el proyecto debe tener en cuenta el marco regulador vigente en España y en la Unión Europea:

- Reglamento General de Protección de Datos (RGPD) [1]: normativa europea que establece las reglas para el tratamiento de datos personales de los ciudadanos de la UE. La aplicación debe garantizar la confidencialidad, integridad y disponibilidad de los datos, así como ofrecer mecanismos para el ejercicio de derechos (acceso, rectificación, supresión, etc.). Por tanto, es necesario el consentimiento explícito del usuario para la recopilación y uso de sus datos por el software.
- Ley Orgánica de Protección de Datos Personales y garantía de los derechos digitales (LOPDGDD) [2]: adapta el RGPD al ordenamiento jurídico español desarrollando en mayor profundidad varios de sus puntos y concretando obligaciones adicionales. Profundiza, por ejemplo, en el establecimiento de reglas sobre las autoridades competentes en esta área o el tratamiento de datos de personas fallecidas.
- Ley de Servicios de la Sociedad de la Información y de Comercio Electrónico (LSSICE) [3]: regula la prestación de servicios por vía electrónica, incluyendo aspectos como el acceso por parte del usuario a los datos de identificación del responsable del software, condiciones de uso y posibles comunicaciones comerciales.
- Reglamento de Ciberresiliencia (CRA) [4]: establece medidas para garantizar la seguridad de productos de software y hardware, y finalmente proteger a los consumidores. Exige la mitigación de vulnerabilidades comunes, documentación de riesgos y actualizaciones periódicas de seguridad.
- Ley de Propiedad Intelectual [5]: regula la propiedad intelectual, incluyendo los derechos de autor. Salvo excepciones, el consentimiento del autor o autores es necesario en caso de reutilización de contenidos, textos de ejemplo o recursos de terceros dentro de la aplicación.

El diseño y despliegue de la aplicación debe tener en cuenta estos marcos legales, especialmente en lo relativo al tratamiento de datos personales, almacenamiento en la base de datos, opciones de borrado de cuenta, y uso de recursos de terceros.

1.4. Medios empleados

Los elementos de **hardware** son las partes físicas de un sistema informático, que se utiliza para el desarrollo de proyectos de amplia variedad. En el caso de este Trabajo de Fin de Grado, se ha empleado el siguiente dispositivo:

- Ordenador portátil personal: equipo utilizado para el desarrollo de la aplicación y sus servicios, además de pruebas locales, reuniones con el tutor y documentación. Estas son sus características:
 - Modelo: LG Gram 16Z90P
 - Sistema operativo: Microsoft Windows 10 Home
 - Procesador: 11th Intel Core i7-1165G7 2.80GHz
 - Memoria RAM: 16 GB
 - Tarjeta gráfica: Intel Iris Xe Graphics

Los elementos de **software** son los programas informáticos que otorgan carácter funcional al hardware, en este caso para hacer posible el desarrollo del proyecto. Los elementos de software utilizados son los siguientes:

- Visual Studio Code [6]: entorno de desarrollo integrado para la programación del frontend y backend, así como para la redacción de la memoria utilizando el sistema de composición de textos LaTeX.
- Firefox Developer Edition [7]: navegador web utilizado como motor de búsqueda para la investigación de la información aquí presentada, documentación y resolución de dudas para el desarrollo, y para la ejecución y prueba de la aplicación.
- MongoDB Compass [8]: programa que ofrece una interfaz que facilita la gestión y consulta de los datos de la base de datos.
- Postman [9]: herramienta para verificar el correcto funcionamiento de las rutas del servidor.
- GitHub [10]: plataforma que permite el control de versiones del proyecto, mantener el historial de cambios y facilitar su almacenamiento en la nube y el acceso al código.
- Google Meet [11]: plataforma digital para mantener reuniones semanales en línea con el tutor del Trabajo de Fin de Grado.
- Trello [12]: herramienta de gestión de tareas utilizada para crear tableros de organización y seguimiento de la planificación del proyecto.

1.5. Estructura del documento

Esta memoria se estructura en seis capítulos además del índice de contenidos, figuras y tablas:

1. **Introducción:** en esta sección se introduce el contexto de este documento y del proyecto, se expone la motivación para su realización, los objetivos que esta persigue, el marco regulador que encuadra el contexto legislativo en que se desarrolla este software y los medios tanto físicos como digitales empleados, además del apartado actual.
2. **Estado del arte:** este capítulo cuenta con cuatro secciones. En la primera, se hace un recorrido por las tecnologías presentes y más utilizadas actualmente para el desarrollo de aplicaciones web, pasando por los campos de frontend, entornos de trabajo, backend, creación y consumo de APIs y bases de datos. En la segunda sección, se exponen algunas de las metodologías principales para el desarrollo de un proyecto de software. La tercera sección presenta varios ejemplos de negocio que podrían ser similares a la aplicación aquí desarrollada y por tanto actuar como competencia. Por último, encontramos una justificación de la solución aquí documentada.
3. **Análisis y planificación:** se presentan las historias de usuario y pruebas de aceptación que definen y validan la documentación y las funcionalidades de la aplicación, así como la matriz de trazabilidad que los contrapone. Tras ello, este capítulo incluye la planificación que se ha seguido para la realización del proyecto.
4. **Arquitectura del software:** arquitectura de la aplicación full-stack. Se exhibe la estructura, la lógica y los aspectos técnicos destacados de la base de datos, el frontend y el backend.
5. **Entorno socioeconómico:** incluye tanto los costes del proyecto como su impacto socioeconómico en el sector.
6. **Conclusiones:** se revisa el cumplimiento de los objetivos y se proponen diversas mejoras, líneas de trabajo futuro y una conclusión final valorando el proceso de aprendizaje adquirido durante el desarrollo del trabajo.

2. ESTADO DEL ARTE

En este capítulo se define el contexto en que se desarrolla el proyecto, estudiando tres factores principales: el sector tecnológico, teniendo en cuenta las tecnologías actuales para el desarrollo de aplicaciones web; las metodologías vigentes empleadas en este sector; y el modelo de negocio en que se ubica la aplicación, estudiando los principales ejemplos de soluciones similares presentes en el mercado.

2.1. Tecnologías para el Desarrollo de Aplicaciones Web

En el desarrollo de una aplicación web completa, entran en juego tecnologías de varios tipos. De entre ellos, se han seleccionado los más relevantes dado el tipo de solución que se implementa en este proyecto: tecnologías de frontend, frameworks de JavaScript, tecnologías de backend, herramientas para el consumo de APIs y bases de datos.

2.1.1. Frontend

El frontend es la parte visible de una aplicación web, con la que el usuario interactúa de forma directa. Se compone de tres tecnologías principales: lenguaje de marcado de hipertexto (HTML), lenguaje de diseño gráfico y lenguaje de programación; que trabajan juntas para definir la estructura, el diseño y la interactividad de la interfaz. Estas tecnologías son esenciales para cualquier desarrollo web, desde sitios estáticos hasta aplicaciones dinámicas y complejas como la que se plantea en este proyecto.

HTML: la estructura de la web

HTML (HyperText Markup Language) es el lenguaje de marcado utilizado para definir la estructura de una página web. Funciona mediante etiquetas que organizan el contenido en elementos como encabezados, párrafos, listas, imágenes y enlaces. Como ejemplo, las etiquetas correspondientes a estos elementos son `<h>`, `<p>`, `<ol>`, `<img>` y `<link>`. Existe una gran cantidad de etiquetas para construir los diferentes elementos que se pueden encontrar en una página web. Otros ejemplos de estos elementos son los botones, áreas de texto, formularios, menús, secciones, títulos, y muchos otros. La estructura de una etiqueta o *tag* HTML se compone de una etiqueta de apertura, el contenido, y una etiqueta de cierre:

- La etiqueta de apertura puede contener atributos que especifican información sobre ese elemento, como puede ser su clase, un link a otra página, funciones que se ejecutan cuando el usuario interactúa con ese elemento, etc. Existen atributos

opcionales y obligatorios para según qué etiqueta; cada atributo tendrá un valor, y es posible tener tags sin atributos.

- Un tag puede tener contenido, que se especifica entre la etiqueta de apertura y la de cierre. El contenido es el texto, imagen u otro elemento dentro de la etiqueta, lo que se mostrará en la pantalla. Así como existen tags sin atributos, también podemos crear tags sin contenido.
- La etiqueta de cierre marca el fin de la etiqueta. Si la etiqueta carece de contenido, se puede prescindir de la etiqueta de cierre marcando la etiqueta de apertura con un *slash* ("/"), es decir, se puede utilizar `<tag/>` en vez de `<tag></tag>`.

A diferencia de otros lenguajes, HTML no tiene lógica de control (como bucles, variables o condicionales), sino que se enfoca exclusivamente en estructurar la información. Por tanto, HTML es el pilar sobre el cual se construyen las páginas.

La evolución de HTML ha llevado a versiones más avanzadas, en particular HTML5, que introduce nuevas etiquetas semánticas como `<article>`, `<section>` y `<nav>`, mejorando la organización del contenido y optimizando su accesibilidad. También incorpora soporte para audio, vídeo y gráficos vectoriales sin necesidad de complementos adicionales, lo que amplía las posibilidades de una aplicación web moderna.[13]



Fig. 2.1. Logo de HTML5

CSS: El diseño y la presentación

Para construir el diseño y el aspecto visual de una página web, se utiliza CSS, que quiere decir *Cascade Style Sheets* u hojas de estilo en cascada. Se basa en reglas que seleccionan elementos HTML y les aplican propiedades de estilo (color, forma, márgenes, posición, animaciones, transiciones, y demás). Existen varias formas de incluir CSS en una página web:

- **CSS en línea:** los estilos se aplican directamente dentro de una etiqueta HTML mediante el atributo `style`. Esta opción es útil para aplicar estilos rápidos a elementos individuales, pero dificulta la mantenibilidad y reutilización del código.
- **CSS interno:** los estilos se definen dentro de una etiqueta `<style>` en el documento HTML. Esto permite centralizar los estilos dentro del mismo archivo, pero puede volverse poco eficiente en proyectos grandes.

- **CSS externo:** los estilos se escriben en un archivo separado, permitiendo una mejor organización y reutilización. Esta es la forma más recomendada en proyectos escalables, dado que facilita la separación entre la estructura (HTML) y el diseño (CSS).
- **CSS basado en utilidades:** se utilizan clases predefinidas aplicadas directamente en el HTML. Esta metodología permite construir interfaces rápidamente sin necesidad de escribir reglas CSS personalizadas. Aunque este enfoque facilita la productividad y la coherencia visual, puede generar un código HTML más cargado de clases, lo que a algunos desarrolladores les resulta menos legible. Tailwind CSS es el framework de CSS que introdujo la funcionalidad para aplicar estilos de esta manera.



Fig. 2.2. Logo de Tailwind

Con la introducción de CSS3, se han añadido características avanzadas como gradientes, sombras, transiciones y animaciones, así como técnicas de diseño responsivo como flexbox y grid, que permiten adaptar la interfaz a distintos tamaños de pantalla. Esto es especialmente importante para que el diseño una aplicación web sea correcto y agradable en cualquier tipo de dispositivo, tanto portátiles como tabletas, teléfonos móviles y otros. [14], [15]



Fig. 2.3. Logo de CSS

JavaScript: La interactividad y la lógica

JavaScript es el lenguaje de programación estándar para implementar la interactividad de un sitio web. Se trata de un lenguaje de propósito general que se ejecuta en el navegador del usuario y permite añadir interactividad y dinamismo a una aplicación web. Mediante JavaScript, una aplicación web puede responder a eventos como clics, introducción de texto o movimientos del ratón sin necesidad de recargar la página. Con esta herramienta se pueden implementar todo tipo de funcionalidades, como validar formularios, actualizar datos en tiempo real o integrar servicios externos.

Un *framework* o entorno de trabajo es una herramienta que ofrece varias funcionalidades y esquemas para desarrollar un proyecto de forma más organizada, ágil y escalable. Con la

introducción de frameworks de JavaScript, como React.js, Vue.js y Angular, el desarrollo frontend ha evolucionado hacia la creación de aplicaciones de una sola página (*Single Page Application* o SPA), donde JavaScript gestiona la navegación y actualización de contenido sin necesidad de recargar la página; y hacia la construcción de páginas mediante componentes reutilizables, lo que aumenta la escalabilidad y acelera el desarrollo de una aplicación compleja.

Otros aspectos fundamentales que JavaScript permite implementar incluyen técnicas como AJAX o Fetch API, para comunicarse con un servidor en segundo plano, enviando y recibiendo datos sin interrumpir la experiencia del usuario.[16], [17]



Fig. 2.4. Logo de JavaScript

En 2012, vista la falta de herramientas que facilitaran un desarrollo óptimo, apareció TypeScript, un *superset* de JavaScript que incorporaba clases, módulos y una estructura más organizada y conveniente para proyectos grandes. Un superset de JavaScript es una tecnología capaz de compilar y ejecutar código JavaScript pero que tiene su propia sintaxis y diferenciaciones, de forma que podemos incluir código TypeScript en un proyecto JavaScript y viceversa.

La principal diferencia que marca TypeScript es que se trata de un lenguaje de tipado estático, es decir, al declarar una variable se debe especificar el tipo de dato de esta, y el tipo no es modificable. Esto no solo facilita la detección de *bugs* o errores en el código, sino que también permite ofrecer sugerencias a la hora de operar con esas variables o tomar argumentos en funciones. [18], [19]



Fig. 2.5. Logo de TypeScript

2.1.2. Frameworks de JavaScript

La utilización de un framework de JavaScript para el desarrollo de una aplicación web es muy recomendable dadas las funcionalidades preprogramadas para agilizar las tareas de programación. Entre ellas: la modularización del código, la optimización del rendimiento, la integración de servicios, la manipulación de eventos, la creación de interfaces dinámicas, y el manejo del DOM (*Document Object Model*), la interfaz de programación que permite crear, eliminar o modificar elementos de la página.

Se estudiarán algunas de las alternativas de frameworks que hay actualmente a disposición del programador.[20]

React

React.js es una de las tecnologías más utilizadas en el desarrollo de interfaces dinámicas. Creada por ingenieros de Meta, esta biblioteca de JavaScript permite construir aplicaciones modulares y eficientes gracias a su sistema basado en componentes reutilizables y la utilización de un DOM virtual, que realiza una evaluación de los cambios que se aplicarán al DOM, y los aplica solamente al elemento o los elementos que cambian o se ven afectados por la modificación. Esto mejora el rendimiento en la actualización de la interfaz.

Aplicaciones como Facebook, Instagram y Airbnb han adoptado React debido a las facilidades que ofrece y su flexibilidad y ecosistema robusto, además de su facilidad de instalación y relativamente accesible curva de aprendizaje.

Actualmente es uno de los frameworks web más populares y utilizados. Esto implica que existe una gran cantidad de documentación y guías para aprender y resolver dudas sobre React, además de numerosas librerías compatibles con esta tecnologías y componentes gratuitos que uno puede incorporar en sus propios proyectos y que se encuentran en plataformas como 21st.dev, Aceternity ui o MUI.[21], [22]



Fig. 2.6. *Logo de React*

Next

Sobre React se ha construido Next.js, un framework que optimiza el rendimiento al incorporar renderizado del lado del servidor (la generación del contenido de una página web ocurre en el servidor, en lugar de hacerlo en el navegador del cliente, buscando una mejora del rendimiento) y generación automática de páginas estáticas, lo que lo convierte en una muy buena opción para aplicaciones con necesidades de optimización para motores de búsqueda.

Empresas como TikTok, Twitch y Hulu han apostado por Next.js debido a su capacidad para mejorar la velocidad de carga y la experiencia del usuario. Otra de las principales ventajas de utilizar Next.js es que cuenta con Vercel, una plataforma que permite desplegar y alojar proyectos de forma altamente optimizada, rápida y sencilla. Sin embargo, su configuración inicial puede ser más compleja en comparación con React puro, y algunas de sus funcionalidades avanzadas requieren un aprendizaje adicional.[23], [24]



Fig. 2.7. *Logo de Next*

Vue

Otra opción destacada en frontend es Vue.js, un framework que ha ganado popularidad por su curva de aprendizaje baja y su sintaxis intuitiva. Facilita la creación de sitios web de una sola página ligeros y con alta velocidad de carga. Empresas como Xiaomi, Alibaba y Nintendo lo utilizan por su facilidad de integración y su capacidad para construir interfaces interactivas con menor esfuerzo en comparación con React o Angular. Además, existe una librería llamada Vuetify que pone a disposición del desarrollador numerosos componentes para utilizar en sus aplicaciones. Una desventaja es que Vue cuenta con una comunidad más pequeña y menos recursos disponibles, lo que puede ser un inconveniente en proyectos grandes.[25], [26]



Fig. 2.8. *Logo de Vue*

Angular

Angular es un framework completo desarrollado por Google basado en TypeScript que ofrece una arquitectura robusta para el desarrollo de aplicaciones web a gran escala. Gmail y Microsoft Teams lo utilizan dada su capacidad para estructurar aplicaciones complejas con un fuerte soporte empresarial. Su ecosistema es muy completo, con herramientas y recursos de interfaces de usuario integrados, y tiene fuerte soporte y comunidad dada su popularidad. No obstante, su curva de aprendizaje es considerablemente más alta que la de React o Vue, y su enfoque basado en clases y módulos puede resultar más rígido.[27], [28]



Fig. 2.9. *Logo de Angular*

2.1.3. Backend

El backend es la parte de una aplicación que gestiona la lógica de negocio, el procesamiento de datos, la seguridad, y la comunicación con la base de datos y otros servicios. Funciona en el servidor y se encarga de recibir peticiones del cliente, procesarlas y devolver una respuesta adecuada.

Hay diversas opciones de tecnologías que permiten llevar a cabo estos objetivos. Algunas de las más relevantes a día de hoy son Node.js, Django, Spring Boot, Laravel, Ruby, y Gin.

Node.js

Node.js es un entorno de ejecución basado en JavaScript que permite ejecutar código en el servidor. Su principal ventaja es que permite desarrollar tanto el frontend como el backend con el mismo lenguaje, lo que facilita la comunicación entre ambas partes.

Su arquitectura está basada en eventos y es no bloqueante, es decir, el sistema no espera a que se termine de ejecutar cada tarea o consulta para empezar con la siguiente, sino que libera el flujo de ejecución para llevar a cabo otras tareas en paralelo. Esto lo hace altamente eficiente en aplicaciones que requieren muchas conexiones simultáneas.

Es posible cargar y utilizar una gran variedad de paquetes con npm (Node Package Manager), que ofrece miles de bibliotecas para ampliar sus funcionalidades y agilizar el desarrollo; y permite dar soporte y servicio en tiempo real a una aplicación.

Empresas como Netflix, PayPal y Uber utilizan Node.js dada su escalabilidad y su capacidad para manejar grandes volúmenes de tráfico. Sin embargo, no es la mejor opción para aplicaciones que requieren cálculos intensivos, como procesamiento de imágenes o simulaciones matemáticas complejas.

Dentro del ecosistema de Node.js, destaca el Express.js, un framework minimalista que facilita la creación de APIs y servidores web. Es rápido, flexible y ampliamente utilizado en el desarrollo de aplicaciones modernas.[29], [30], [31]



Fig. 2.10. Logo de NodeJS

Python con Django

Django es un framework de Python que se encuentra entre uno de los más utilizados en el ámbito del backend por su enfoque en la seguridad y el desarrollo rápido.

Python es un lenguaje de programación conocido por su simplicidad y legibilidad, con

una sintaxis clara y cercana al lenguaje humano. Su flexibilidad y estructura intuitiva, basada en indentación en lugar de símbolos complejos, lo hace fácil de entender y aprender. Por ello, es una buena opción tanto para principiantes como para desarrolladores en áreas diversas, desde inteligencia artificial hasta desarrollo web.



Fig. 2.11. Logo de Python

Django tiene un enfoque *batteries included*, es decir, ofrece muchas herramientas listas para usar: cuenta con ORM (Mapeo Objeto-Relacional) integrado, una herramienta que convierte los objetos de una aplicación en un formato compatible para guardarlos en una base de datos y así permitir la interacción con esta sin necesidad de escribir código SQL; y herramientas de seguridad avanzada contra ataques comunes como inyección SQL. Django es ampliamente utilizado en aplicaciones como Instagram, Spotify y Pinterest.

Por otra parte, es un framework monolítico, es decir, todo su código está integrado en una sola estructura. Por ello, es menos flexible cuando se necesita dividir la aplicación en partes independientes que trabajen de forma autónoma, como ocurre en arquitecturas basadas en microservicios, donde cada parte de la aplicación se maneja de manera independiente y puede ser escalada o actualizada por separado.[32], [33]



Fig. 2.12. Logo de Django

Java con SpringBoot

Java es una de las opciones más utilizadas en el desarrollo backend, especialmente en entornos empresariales, dadas la robustez de sus funciones de seguridad y su sólido rendimiento en tareas que demandan un uso intensivo del procesador, como cálculos matemáticos avanzados o intérpretes de lenguaje.

El framework más popular dentro de este ecosistema es Spring Boot, que agiliza la creación de aplicaciones escalables y distribuidas. Entre sus ventajas se encuentran su arquitectura modular, que permite crear servicios independientes y fácilmente escalables; su alta seguridad y estabilidad, que lo hace ideal para aplicaciones bancarias y corporativas; y su compatibilidad con múltiples bases de datos y herramientas empresariales.

Sin embargo, Java tiene una curva de aprendizaje más alta y puede ser más pesado en comparación con otras opciones como Node.js o Python.[34], [35]



Fig. 2.13. *Logo de Java*



Fig. 2.14. *Logo de SpringBoot*

PHP y Laravel

PHP ha sido históricamente uno de los lenguajes más utilizados para el desarrollo web, especialmente en sistemas de gestión de contenido como WordPress, que permite crear páginas como blogs o tiendas en línea sin apenas conocimientos técnicos. Permite agregar dinamismo a nuestros sitios web conectando el servidor y base de datos con la interfaz de usuario.

En la actualidad, Laravel es el framework más moderno dentro del ecosistema PHP, ofreciendo una sintaxis elegante y herramientas avanzadas para el desarrollo de aplicaciones web. Cuenta con un sistema de enrutamiento y autenticación integrado, buena documentación y comunidad activa. Además, ORM Eloquent, un Mapeador de Objetos Relacionales especialmente diseñado para Laravel, ofrece una experiencia de conexión con la base de datos más ágil y simple.

Se trata de una opción sólida para proyectos que necesitan desarrollo rápido, pero PHP no es tan popular en nuevas aplicaciones escalables como Node.js o Django. Entre sus desventajas se encuentran herramientas de debugging no tan potentes y los comunes problemas de seguridad.[36], [37]



Fig. 2.15. *Logo de PHP*



Fig. 2.16. *Logo de Laravel*

Ruby on Rails

Ruby es un lenguaje de programación con una sintaxis cercana al lenguaje humano y orientado a objetos. Ruby on Rails es un framework basado en el lenguaje Ruby que se centra en la productividad del desarrollador y la facilidad de uso dado que toma decisiones por el desarrollador basándose en convenciones preestablecidas e incluye una gran cantidad de herramientas integradas para tareas comunes. La generación automática de código que ofrece Ruby On Rails acelera el desarrollo; y además la seguridad está integrada, protegiendo contra ataques comunes. Aplicaciones como GitHub y Airbnb fueron construidas con esta tecnología debido a su facilidad para construir aplicaciones rápidamente. El principal inconveniente de Rails es que su rendimiento puede ser inferior al de otras tecnologías más modernas, como Node.js o Go.[38], [39]



Fig. 2.17. *Logo de Ruby*



Fig. 2.18. *Logo de Ruby On Rails*

Golang y Gin

Golang, también conocido como Go, es un lenguaje desarrollado por Google enfocado en el rendimiento, la seguridad y la concurrencia a la hora de atender solicitudes, lo cual es ideal para sistemas que requieren gran escalabilidad. Plataformas como Dropbox, Netflix o Uber están programadas con Go. Se trata de una alternativa de alto rendimiento, comparable al de C/C++, ya que utiliza código compilado, es decir, el código fuente es compilado para traducirse a código máquina. Su framework más popular para el backend es Gin, que permite construir APIs rápidas y eficientes.

Sin embargo, Go tiene menos bibliotecas y comunidad que otras tecnologías más maduras como Node.js o Python. Además, su funcionamiento puede resultar complejo en ciertos aspectos, por ejemplo, en ciertas situaciones se requiere manejar manualmente la memoria para optimizar la aplicación.[40], [41], [42]



Fig. 2.19. Logo de Go



Fig. 2.20. Logo de Gin

2.1.4. Consumo de APIs

Entre otros fines, estos frameworks y tecnologías backend son útiles para la creación de APIs. Una API o *Interfaz de Programación de Aplicaciones* es un mecanismo que permite conectar dos piezas de software para que una pueda interactuar con la otra, intercambiar información, invocar funciones o utilizar servicios.

Existen diferentes tecnologías que permiten consumir o utilizar una API desde una aplicación web, destacando Axios, Fetch API y cURL; además de herramientas que ayudan a probar APIs, como Postman e Insomnia.

Axios

Axios es una librería basada en JavaScript que permite la realización de solicitudes HTTP tanto en el navegador como en el servidor utilizando Node. HTTP o *Hypertext Transfer Protocol* es un protocolo de comunicación para transferir datos entre un cliente (generalmente una aplicación o navegador) y un servidor web a través de Internet. La comunicación se lleva a cabo mediante solicitudes y respuestas: el cliente realiza una solicitud HTTP al servidor para obtener un recurso (como una página, un archivo, una imagen o cualquier dato) y el servidor responde con la información solicitada en caso de éxito.

Axios soporta métodos estándar de HTTP y es especialmente popular debido a su capacidad para manejar promesas de manera eficiente, lo que permite gestionar de forma más sencilla la asincronía en las aplicaciones web. Entre sus ventajas se destacan la posibilidad de

interceptar solicitudes o respuestas, manejar errores de manera más eficaz y la capacidad de enviar datos en formato JSON de forma predeterminada. Dado su amplio uso en aplicaciones basadas en frameworks como React o Vue.js, Axios se ha consolidado como una de las opciones más elegidas por los desarrolladores.[43], [44]



Fig. 2.21. *Logo de Axios*

FetchAPI

Fetch API es una interfaz nativa en JavaScript que proporciona una forma moderna y flexible de hacer solicitudes HTTP desde el navegador. A diferencia de otras soluciones, FetchAPI es parte del estándar de JavaScript, por lo que no requiere la instalación de bibliotecas adicionales. Permite trabajar con promesas para manejar solicitudes asíncronas y es compatible con los métodos HTTP más utilizados. Aunque Fetch no ofrece algunas características avanzadas como la transformación automática de respuestas JSON o manejo de errores, su simplicidad y flexibilidad lo convierten en una excelente opción para aplicaciones web modernas que requieren realizar solicitudes HTTP sin depender de bibliotecas externas.[45], [46]



Fig. 2.22. *Logo de FetchAPI*

Postman

Postman es una herramienta gráfica ampliamente utilizada para probar, desarrollar y gestionar APIs. Permite enviar solicitudes HTTP, analizar las respuestas y organizar las pruebas en colecciones, lo cual resulta muy útil durante el desarrollo de servicios web y microservicios. Todo esto lo convierte en una herramienta muy potente en el proceso de desarrollo y pruebas de APIs.[47], [48]

Postman es particularmente valioso cuando se trabaja con APIs REST. Una API REST es una API que sigue los principios REST (*Representational State Transfer*): opera mediante métodos del protocolo HTTP, que son GET para recuperar información del servidor, POST para enviar datos al servidor, PUT para actualizar esos datos, DELETE para borrarlos, y otros; cada recurso está asociado a una URL única; y cada solicitud contiene toda la

información necesaria para ser completada, no solo no necesita ni depende de datos de otras solicitudes, sino que además no se transmiten ni comparten datos de una petición a otra. [49]

También, Postman permite visualizar respuestas en diversos formatos, como JSON, XML y texto plano; y ofrece funciones avanzadas como la automatización de pruebas, la creación de entornos y la automatización del proceso de integración y despliegue del código. Todo esto lo convierte en una herramienta muy potente en el proceso de desarrollo y pruebas de APIs.[47], [48]



Fig. 2.23. Logo de Postman

Insomnia

Insomnia es una herramienta similar a Postman, diseñada para facilitar la prueba y el consumo de APIs. Se distingue por su interfaz limpia y sencilla, que permite a los desarrolladores trabajar de manera eficiente sin distracciones. Al igual que Postman, Insomnia soporta solicitudes HTTP y puede gestionar entornos, variables y autenticación. Cuenta con soporte nativo para GraphQL, un lenguaje de consulta y entorno de ejecución para las APIs que permite obtener varios recursos con una sola consulta y obtener solamente los datos deseados, sin datos extra o campos que no queremos en ese momento. Una de sus principales ventajas es la posibilidad de crear pruebas automatizadas, lo que optimiza el proceso de desarrollo y mejora la calidad del código. Insomnia también es altamente personalizable y permite a los usuarios gestionar sus solicitudes de forma organizada, lo que lo hace adecuado tanto para desarrolladores principiantes como para profesionales experimentados.[50], [51]



Fig. 2.24. Logo de Insomnia

cURL

cURL es una herramienta de línea de comandos ampliamente utilizada para realizar solicitudes HTTP a través de un terminal o script. Aunque no proporciona una interfaz gráfica, cURL es extremadamente versátil y potente, permitiendo a los desarrolladores interactuar con APIs de forma directa.

Su principal ventaja es su capacidad para ser integrado en scripts y automatizar tareas de manera eficiente, lo que lo convierte en una herramienta útil para aquellos que prefieren trabajar desde la línea de comandos o necesitan realizar pruebas en entornos sin interfaz gráfica. cURL es compatible con una amplia variedad de protocolos y es especialmente útil para realizar pruebas rápidas y depurar servicios web.[52], [53]



Fig. 2.25. Logo de cURL

2.1.5. Base de datos

Una base de datos es una recopilación de información que se almacena en un sistema informático y a la que se puede acceder mediante consultas. Existen dos tipos principales de bases de datos: relacionales y no relacionales.

En una base de datos relacional, los datos se almacenan en tablas, donde cada fila representa un registro y cada columna contiene un atributo o campo de ese registro. Cada registro suele tener un identificador único, conocido como clave primaria, que permite distinguirlo de los demás. Existen relaciones entre las tablas que permiten organizar los datos de forma lógica y estructurada siguiendo un modelo relacional. Para escribir y leer datos de este tipo de bases de datos, se utiliza un lenguaje de consulta estructurado, como MySQL, PostgreSQL o MariaDB.

Por su parte, las bases de datos no relacionales o NoSQL ofrecen un enfoque más flexible ya que, en lugar de seguir una estructura rígida de tablas, almacenan la información en formatos como documentos JSON, pares clave-valor, grafos o familias de columnas. Esta estructura es especialmente útil cuando los datos cambian con frecuencia o no requieren un esquema fijo. Esto permite insertar, modificar y obtener los datos de forma más dinámica, lo que se hace mediante lenguajes de consulta más modernos. Este tipo de bases de datos ha ganado popularidad en el campo de las aplicaciones web modernas, microservicios y sistemas que manejan grandes volúmenes de datos no estructurados. MongoDB, Cassandra, Firebase y Elasticsearch se encuentran entre las bases de datos NoSQL más utilizadas a día de hoy.[54]

Bases de datos relacionales

MySQL: MySQL es uno de los sistemas de gestión de bases de datos más populares y utilizados en aplicaciones web. Originalmente desarrollado por MySQL AB y posteriormente adquirido por Oracle, MySQL es conocido por su fiabilidad, integridad, velocidad y facilidad de uso. Su arquitectura de replicación maestro-esclavo permite distribuir la

carga de trabajo y mejorar la disponibilidad. Aunque es de código abierto, también ofrece versiones comerciales con soporte adicional. MySQL ha sido fundamental en la creación de aplicaciones dentro de la pila LAMP (Linux, Apache, MySQL, PHP/Python), y grandes plataformas como Facebook y Twitter lo emplean para manejar grandes volúmenes de datos.[55], [56]



Fig. 2.26. Logo de MySQL

PostgreSQL: PostgreSQL destaca por su capacidad para gestionar transacciones complejas y grandes volúmenes de datos, manteniendo siempre la integridad relacional.[57], [58] Cuando varios usuarios acceden simultáneamente a una base de datos, los sistemas tradicionales suelen bloquear los registros para evitar conflictos entre lecturas y escrituras. Una de sus principales ventajas reside en que gestiona esta concurrencia de manera más eficiente gracias a su implementación de MVCC (Control de Concurrencia Multiversión), que permite que las operaciones de lectura no interfieran con las de escritura y viceversa. Esto se traduce en una mejora significativa del rendimiento y la experiencia en entornos con múltiples usuarios accediendo a los mismos datos.

Además, PostgreSQL es compatible con múltiples lenguajes de programación, como JavaScript, C/C++, Python, Ruby, etc. Aplicaciones de alto perfil como Instagram, Reddit y Discord emplean PostgreSQL debido a su estabilidad, capacidad para manejar grandes cantidades de datos y su sistema avanzado de consultas.[57], [58].



Fig. 2.27. Logo de Postgresql

Maria DB: MariaDB es un sistema de gestión de bases de datos relacional de código abierto que nació como una bifurcación de MySQL, creado por su fundador original, Michael Widenius, después de la compra de MySQL por Oracle. Este sistema mantiene una alta compatibilidad con MySQL, lo que facilita la migración entre ambos. MariaDB ofrece mejoras en términos de rendimiento y seguridad, incorporando nuevos motores de almacenamiento. Además, su enfoque en la escalabilidad y la alta disponibilidad lo hace adecuado para aplicaciones que requieren un rendimiento constante. Es ampliamente utilizado por empresas de gran tamaño y cuenta con una comunidad activa que garantiza una evolución constante.[59], [60]



Fig. 2.28. Logo de MariaDB

No relacionales

MongoDB: MongoDB es una base de datos NoSQL que ha ganado una gran popularidad en aplicaciones web modernas. Su principal característica es que almacena los datos en formato de documentos BSON, una versión de JSON en formato binario; lo que permite un enfoque flexible y escalable. JSON es el formato en el que se intercambian los datos en JavaScript, por lo que combinar estas dos tecnologías es una configuración ágil para un proyecto. Estos documentos se pueden agrupar en colecciones para facilitar la organización de los datos.

MongoDB es ideal para aplicaciones que necesitan escalabilidad horizontal, ya que permite distribuir los datos entre varios servidores de manera eficiente. Además, permite modificar fácilmente los datos almacenados sin tener que realizar grandes cambios en la base de datos. Empresas como Uber, eBay y Trello utilizan MongoDB debido a su capacidad para manejar grandes cantidades de datos sin la rigidez de las bases de datos tradicionales, además de su gran fiabilidad y alto rendimiento.[61], [62]



Fig. 2.29. Logo de MongoDB

Firebase: Firebase es un servicio de base de datos de Google que almacena los datos en la nube y está diseñado principalmente para aplicaciones web y móviles que requieren sincronización en tiempo real. Facilita la actualización instantánea de los datos en todas las plataformas de manera eficiente, lo que la convierte en una opción atractiva para proyectos como aplicaciones de mensajería o colaboración. Su integración con el ecosistema de Google permite aprovechar otros servicios como Firebase Authentication y Google Cloud Storage. Además, su facilidad de uso lo hace adecuado para desarrolladores que trabajan en proyectos pequeños o medianos, dado que permite desarrollar aplicaciones de manera rápida sin la necesidad de gestionar una infraestructura compleja.

No obstante, Firebase puede generar dependencia de la plataforma Google, lo que podría convertirse en un inconveniente si se desea cambiar de proveedor a largo plazo. También, los costos asociados con su uso pueden incrementarse a medida que la aplicación escala, lo que puede ser una preocupación para proyectos de gran tamaño.[63], [64]



Fig. 2.30. Logo de Firebase

Cassandra: Cassandra es una base de datos no relacional distribuida diseñada para manejar grandes volúmenes de datos en entornos de alta velocidad y disponibilidad y sin un único punto de fallo gracias a que opera mediante varios nodos o servidores cada uno de los cuales almacena una parte de los datos y responde a una parte de las consultas. Su arquitectura es distribuida, es decir, cada nodo tiene réplicas de los datos de los demás, y es por eso que, si uno falla, los demás pueden atender sus peticiones. Cassandra utiliza un modelo basado en columnas en lugar de filas, de forma que cada fila o registro puede tener columnas diferentes, y puede acceder a una columna específica en lugar de obtener toda la fila, lo que aumenta la velocidad de acceso a la información. Empresas que requieren una infraestructura que pueda escalar horizontalmente y manejar grandes volúmenes de datos, como eBay y Netflix, utilizan Cassandra para gestionar su información.[65], [66]



Fig. 2.31. Logo de Cassandra

2.2. Metodología

A lo largo de los años, la forma en que se abordan los proyectos de desarrollo web ha evolucionado significativamente, desde enfoques rígidos y estructurados hasta metodologías más flexibles y colaborativas. Cada método ha surgido en respuesta a distintas necesidades dentro del desarrollo de software y encaja con un tipo de proyecto u otro dependiendo de sus requisitos y equipo de trabajo. A continuación, se presentan algunas de las metodologías más utilizadas en proyectos de desarrollo web.

Metodología en Cascada

Este modelo es uno de los más antiguos en el desarrollo de software y sigue un enfoque secuencial, en el que cada fase debe completarse antes de pasar a la siguiente. Se divide en etapas bien definidas como análisis, diseño, implementación, pruebas y mantenimiento. Su principal ventaja es la claridad en la planificación, pero su rigidez puede dificultar la adaptación a cambios durante el proyecto.[67]

Metodología Incremental

La metodología incremental es un enfoque de desarrollo de software que consiste en construir el sistema por partes, entregando versiones más desarrolladas en cada etapa. Cada incremento añade nuevas funcionalidades al producto, se avanza de forma progresiva hasta completar el sistema final. Este método facilita la detección temprana de errores y la incorporación de cambios, ya que se obtiene retroalimentación continua del cliente. De esta manera, permite entregar valor desde las primeras fases del proyecto.

Sin embargo, el sistema requiere de una planificación inicial bastante completa, ya que los incrementos dependen de una buena definición de requisitos desde el principio.[68]

Metodología RAD

La metodología RAD (Desarrollo Rápido de Aplicaciones) se utiliza para desarrollar software partiendo del diseño de un prototipo funcional que se prueba con los usuarios desde etapas tempranas, permitiendo identificar fallos y nuevos requerimientos. Basándose en esa retroalimentación, se establecen prioridades para realizar mejoras rápidas y continuas, enfocándose en la velocidad de ejecución y en la entrega de resultados en plazos cortos.[69]

Metodología Agile

Agile es un enfoque iterativo e incremental que busca la entrega rápida de software funcional mediante ciclos cortos de trabajo llamados *sprints*. Se basa en la colaboración entre equipos multidisciplinarios, la adaptación continua a los cambios y la entrega de valor al cliente de manera constante, además de priorizar el software frente a la documentación.

Existen varias metodologías ágiles o entornos de trabajos que aplican diferentes enfoques para conseguir objetivos más concretos. Uno de los más populares es **Scrum**, donde el proceso comienza con la planificación del sprint, seleccionando tareas de la lista priorizada de funcionalidades que el producto necesita (*Product Backlog*) para formar una lista reducida que debe estar hecha al terminar ese sprint (*Sprint Backlog*). Durante el sprint, el equipo trabaja colaborativamente y realiza reuniones diarias para revisar el progreso. Al finalizar, se entrega un incremento del producto y se hace un trabajo de retrospectiva y análisis del proceso, con el objetivo de mejorar continuamente en el siguiente ciclo. [70], [71]

Un marco de trabajo similar a Scrum es **Extreme Programming** o XP, aunque este tiene un enfoque más técnico con métodos para los desarrolladores como la programación en pareja, la integración continua, el desarrollo basado en pruebas, y la refactorización iterativa y continua mediante ciclos de retroalimentación. XP tiene un enfoque muy técnico para asegurar la calidad y flexibilidad del software.[72]

Metodología Kanban

Kanban es una metodología con origen en Japón que ayuda a gestionar el flujo de trabajo de manera eficiente mediante un tablero con representaciones visuales del flujo de trabajo y las tareas a realizar, en proceso y terminadas. Estos tableros con columnas también pueden representar las diferentes etapas del desarrollo, permitiendo un control más flexible de las tareas y facilitando la identificación de cuellos de botella, que son limitaciones que restringen el flujo del trabajo o lo bloquean, y del número de tareas que se están llevando a cabo al mismo tiempo. Esta metodología es ideal para equipos que buscan mejorar la productividad de forma simple, rápida, visual y flexible.[73]

Metodología Lean

Derivada de los principios de manufactura de Toyota, Lean busca minimizar el desperdicio de tiempo y recursos, a la vez que maximizar la eficiencia. Se enfoca en la entrega de valor al cliente de la manera más rápida y eficiente posible, eliminando tareas innecesarias y fomentando la mejora continua. Esta metodología prescinde de una retroalimentación frecuente para asegurar una evaluación y mejora continua, con un enfoque perfeccionista.[74]

2.3. Negocio

Hoy en día, existen varias plataformas y aplicaciones centradas en el aprendizaje de idiomas. Estas herramientas varían en su enfoque, desde la corrección automática con inteligencia artificial hasta el intercambio lingüístico entre usuarios. Se expondrán varias de estas alternativas para estudiar en qué se diferencia el proyecto aquí expuesto: qué ideas, características y funcionalidades nuevas aporta.

2.3.1. HiNative

HiNative es una de las plataformas más conocidas en este ámbito, diseñada para resolver dudas sobre el uso de un idioma a través de preguntas y respuestas. Los usuarios pueden consultar sobre gramática, pronunciación y expresiones cotidianas, obteniendo respuestas de hablantes nativos. Aún así, HiNative no está enfocada en la corrección de textos largos, sino en consultas puntuales o discusiones de foro, lo que limita su utilidad para quienes buscan mejorar su escritura de manera estructurada. [75]



Fig. 2.32. Logo de HiNative

2.3.2. Tandem

Otra alternativa es Tandem, una aplicación centrada en el intercambio lingüístico en tiempo real. A través de mensajes de texto, llamadas de voz y videollamadas, los usuarios pueden practicar con hablantes nativos, incluyendo la opción de corregir mensajes dentro del chat. Sin embargo, Tandem prioriza la interacción oral y escrita breve, sin ofrecer herramientas específicas para la corrección detallada de textos largos ni la posibilidad de almacenarlos para un seguimiento a largo plazo.[76]



Fig. 2.33. *Logo de Tandem*

2.3.3. Grammarly

En el ámbito de la corrección automática, herramientas como Grammarly proporcionan análisis gramatical y sugerencias estilísticas mediante inteligencia artificial. Si bien son útiles para detectar errores básicos y mejorar la fluidez del texto, su funcionamiento se basa en reglas predefinidas y modelos de aprendizaje automático, por lo que no pueden ofrecer la misma riqueza y contexto que un hablante nativo. Además, carecen de un enfoque social o de intercambio colaborativo entre usuarios.[77]



Fig. 2.34. *Logo de Grammarly*

2.3.4. Duolingo

Duolingo es una de las aplicaciones más populares para el aprendizaje de idiomas, conocida por su enfoque gamificado y accesible. A través de ejercicios interactivos, los usuarios pueden mejorar su vocabulario, gramática y comprensión escrita y oral de un idioma. Cuenta con un sistema de recompensas que fomenta la constancia en el estudio. Aunque Duolingo ofrece ejercicios de traducción y escritura, su metodología se centra en frases cortas y respuestas predefinidas, sin permitir la redacción libre ni la corrección detallada de textos algo más extensos.[78]



Fig. 2.35. *Logo de Duolingo*

2.4. Justificación de la solución

Como se ha visto en esta última sección del Estado del Arte, existen varias herramientas con el mismo objetivo que la aquí presentada. Con todo, ninguna de ellas ofrece una solución que combine la escritura libre con la interacción social y el intercambio de correcciones. La aplicación propuesta en este documento se diferencia en salirse de las rutas de aprendizaje fijas y predeterminadas ofertadas en soluciones como Duolingo; basar el aprendizaje en la interacción con nativos, a diferencia de opciones como Grammarly; y en la creación de un espacio libre para el aprendizaje por un esfuerzo profundo, personal y colaborativo mediante la redacción de textos más largos, un enfoque alejado de las interacciones rápidas y concisas que ocurren en plataformas como HiNative o Tandem.

Por tanto, la principal innovación reside en que Letterex permite intercambiar textos personales con amigos o nuevos contactos, eligiendo la temática de sus escritos, como si fueran entradas de un diario. Esto permite el intercambio lingüístico, con un sistema de almacenamiento de textos y correcciones para llevar un registro de la evolución en la escritura del idioma que se está aprendiendo; pero también un intercambio personal, es decir, una vía para compartir experiencias e ideas entre usuarios favoreciendo así las conexiones internacionales.

Tras el estudio de las tecnologías presentes actualmente en la industria para el desarrollo de aplicaciones web, en la realización de este proyecto se ha optado por Next.js, Node.js, Axios y MongoDB, además de la herramienta Postman para pruebas de la API. La justificación de esta elección se detalla en profundidad en el cuarto capítulo del documento.

En cuanto a la metodología, se ha decidido utilizar la metodología Agile con Scrum como marco de trabajo, por su capacidad de adaptación continua a cambios en los requisitos y funcionalidades, su flexibilidad a la hora de detallar la planificación inicial, y su estructura sostenible que incluye un seguimiento del trabajo claro y funcional. Se trata de una metodología ampliamente utilizada en la industria en la actualidad, lo que hace que su aprendizaje resulte altamente atractivo para el estudiante.

3. ANÁLISIS Y PLANIFICACIÓN

En este capítulo, se lleva a cabo un análisis de las funcionalidades y requerimientos que componen el proyecto, no solo la construcción del software sino el Trabajo de Fin de Grado en su conjunto. Se presentarán las Historias de Usuario, Pruebas de Aceptación y Matriz de Trazabilidad que registran este análisis, seguidas de la planificación que se ha seguido para el desarrollo del proyecto.

3.1. Historias de Usuario

Las Historias de Usuario (HU) son una herramienta básica en la gestión de proyectos mediante metodologías ágiles como Scrum. Se utilizan para definir las funciones de un sistema desde la perspectiva del usuario final, de forma que las necesidades y funcionalidades que se plantean para el proyecto estén orientadas a aportar valor real al usuario. No se debe confundir con los requisitos del sistema, un método más tradicional que suele resultar rígido y técnico en comparación con las Historias de Usuario, las cuales consisten en una explicación breve y directa de cada funcionalidad del sistema.[79]

Este enfoque orientado al usuario final se consigue mediante esta fórmula:

Como *[rol del usuario]*,
quiero *[realizar una acción]*,
para *[obtener un valor o beneficio]*.

Así, se expresa el requerimiento con una descripción concisa, por ejemplo: "Como usuario registrado, quiero recuperar mi contraseña, para poder acceder a mi cuenta en caso de olvidarla". No se trata de un requisito cerrado sino de plantear situaciones para que el cliente y los desarrolladores tengan una conversación sobre las funcionalidades y sus detalles.

Cada historia cuenta con un identificador (HU-XX), que facilitará reconocerlas, cuantificarlas y asociarlas con las Pruebas de Aceptación; una descripción, siguiendo la fórmula presentada anteriormente; su prioridad ('Alta', 'Media' o 'Baja'), que representa la relevancia para el cliente y el orden que ocupará en el desarrollo; la estimación, que cuantifica las horas de trabajo que requiere la compleción de la historia mediante un sistema de puntos en el que 1 punto equivale a 4 horas; las tareas a realizar para satisfacerla; y las Pruebas de Aceptación para validarla.

A continuación, el listado de Historias de Usuario para el proyecto separadas en varias secciones: Investigación y Documentación, Gestión de Usuarios, Gestión de Perfiles, Módulo de Correspondencia, Módulo de Correcciones y Funcionalidad Social.

3.1.1. Investigación y Documentación

HU-01			
TÍTULO	Redacción de apartados introductorios del documento		
PRIORIDAD	Baja	ESTIMACIÓN	1
DESCRIPCIÓN	Como desarrolladora, quiero estructurar e introducir el documento de forma clara, y presentar el marco regulador pertinente.		
TAREAS	• Investigar plantillas y normativa de la universidad • Consultar ejemplos de TFG de otros compañeros • Investigar sobre las leyes que componen el marco regulador que incumbe a un proyecto web de este tipo • Redactar los apartados de resumen, dedicación, motivación, objetivos, marco regulador, medios empleados y estructura del documento		
PRUEBAS DE ACEPTACIÓN	PA-02		

Tabla 3.1. HU-01

HU-02			
TÍTULO	Investigación de tecnologías para el desarrollo web		
PRIORIDAD	Alta	ESTIMACIÓN	2
DESCRIPCIÓN	Como desarrolladora, quiero investigar las principales tecnologías para el desarrollo de aplicaciones web para poder hacer una elección sólida y fundada de stack del proyecto		
TAREAS	• Investigar tecnologías web para proyectos full-stack • Redactar la primera sección del Estado del Arte • Añadir las referencias bibliográficas necesarias		
PRUEBAS DE ACEPTACIÓN	PA-02		

Tabla 3.2. HU-02

HU-03			
TÍTULO	Investigación de metodologías de desarrollo de software		
PRIORIDAD	Alta	ESTIMACIÓN	1
DESCRIPCIÓN	Como desarrolladora, quiero investigar las principales metodologías para el desarrollo de proyectos de software para trabajar en el proyecto de manera ordenada y planificada		
TAREAS	• Investigar metodologías de desarrollo de software • Redactar las segunda sección del Estado del Arte • Añadir las referencias bibliográficas necesarias		
PRUEBAS DE ACEPTACIÓN	PA-02		

Tabla 3.3. *HU-03*

HU-04			
TÍTULO	Investigación de productos similares en el mercado		
PRIORIDAD	Alta	ESTIMACIÓN	1
DESCRIPCIÓN	Como desarrolladora, quiero investigar sobre otros productos o propuestas similares ya existentes, para un mejor planteamiento del proyecto y de su aporte.		
TAREAS	• Investigar propuestas similares a la aquí trabajada • Redactar las tercera sección del Estado del Arte • Añadir las referencias bibliográficas necesarias • Plantear las mejoras y/o puntos novedosos que mi propuesta aporta		
PRUEBAS DE ACEPTACIÓN	PA-02		

Tabla 3.4. *HU-04*

HU-05			
TÍTULO	Análisis de requerimientos y planificación		
PRIORIDAD	Media	ESTIMACIÓN	5
DESCRIPCIÓN	Como desarrolladora, quiero documentar el análisis de los requerimientos del proyecto y dejarlo constatado en forma de las presentes Historias de Usuario, Pruebas de Aceptación y Matriz de trazabilidad; para poder planificar el trabajo mediante la metodología escogida.		
TAREAS	• Plantear las necesidades de la aplicación y del documento • Redactar este apartado de Análisis y Planificación		
PRUEBAS DE ACEPTACIÓN	PA-01, PA-02		

Tabla 3.5. *HU-05*

HU-06			
TÍTULO	Documentación de la arquitectura final y modelo económico de la aplicación		
PRIORIDAD	Media	ESTIMACIÓN	5
DESCRIPCIÓN	Como desarrolladora, quiero documentar el proceso de la construcción del producto de software final y el modelo económico, incluyendo presupuesto y marco socioeconómico.		
TAREAS	• Desarrollar y probar la aplicación: frontend, backend y base de datos. • Documentar su estructura, arquitectura y puntos de interés en la sección Arquitectura del Software • Redacción del apartado Entorno Socioeconómico		
PRUEBAS DE ACEPTACIÓN	PA-02		

Tabla 3.6. *HU-06*

HU-07			
TÍTULO	Plantear conclusiones y líneas de trabajo futuro		
PRIORIDAD	Baja	ESTIMACIÓN	1
DESCRIPCIÓN	Como desarrolladora, al finalizar el grueso del trabajo quiero revisar el cumplimiento de los objetivos y plantear aspectos del producto final a mejorar en el futuro y conclusiones para cerrar el documento.		
TAREAS	• Analizar el producto final para hacer una reflexión sobre los objetivos alcanzados, las posibles mejoras a futuro y el recorrido de aprendizaje en la realización de este trabajo • Redactar la sección de Conclusiones		
PRUEBAS DE ACEPTACIÓN	PA-02		

Tabla 3.7. *HU-07*

3.1.2. Gestión de Usuarios

HU-08			
TÍTULO	Registro e inicio de sesión		
PRIORIDAD	Alta	ESTIMACIÓN	5
DESCRIPCIÓN	Como usuario visitante, quiero registrarme con mis datos y correo electrónico para acceder a las funcionalidades de mi cuenta de forma segura.		
TAREAS	• Crear una interfaz con formulario de registro e inicio de sesión • Incluir verificación via email de la identidad del usuario para el registro • Facilitar la entrega de un token de autenticación de usuario para obtener los datos que le correspondan en cada página, poder mantener la sesión abierta, y proteger las rutas privadas de la aplicación • Crear los servicios de registro e inicio de sesión en la API		
PRUEBAS DE ACEPTACIÓN	PA-03, PA-04, PA-06, PA-09		

Tabla 3.8. HU-08

HU-09			
TÍTULO	Navegación entre página y cierre de sesión		
PRIORIDAD	Media	ESTIMACIÓN	1
DESCRIPCIÓN	Como usuario registrado, quiero poder navegar fácilmente entre las páginas de la aplicación y cerrar la sesión iniciada		
TAREAS	• Crear un menú desplegable con links a las diferentes páginas y la opción de cerrar la sesión • Asegurarse de que se limpia el token de autenticación al cerrar la sesión		
PRUEBAS DE ACEPTACIÓN	PA-07, PA-08		

Tabla 3.9. HU-09

HU-10			
TÍTULO	Recuperación de contraseña		
PRIORIDAD	Baja	ESTIMACIÓN	1
DESCRIPCIÓN	Como usuario, quiero poder recuperar mi acceso a la plataforma si olvido la contraseña.		
TAREAS	• Crear una interfaz para la recuperación de contraseña y selección de una nueva • Crear un sistema de envío automático de un código de verificación por correo electrónico para la identificación del usuario		
PRUEBAS DE ACEPTACIÓN	PA-05		

Tabla 3.10. *HU-10*

3.1.3. Gestión de Perfiles

HU-11			
TÍTULO	Visualización y Edición de perfil de usuario		
PRIORIDAD	Alta	ESTIMACIÓN	5
DESCRIPCIÓN	Como usuario, quiero editar mis idiomas, biografía, imagen de perfil y país en mi página de perfil, así como mi contraseña.		
TAREAS	• Crear una interfaz que muestre los datos de perfil del usuario y que permita editarlos • Crear un componente que permita cargar imágenes a la plataforma • Implementar un mapa que muestre el país del usuario y que le permita editarlo • Añadir la opción de cambiar la contraseña del usuario • Crear un servicio para editar datos de perfil • Crear un servicio para editar la contraseña, reforzando la seguridad pidiendo la contraseña actual, además de la autenticación		
PRUEBAS DE ACEPTACIÓN	PA-10, PA-11, PA-12		

Tabla 3.11. *HU-11*

HU-12			
TÍTULO	Eliminación de cuenta		
PRIORIDAD	Baja	ESTIMACIÓN	1
DESCRIPCIÓN	Como usuario registrado, quiero poder eliminar mi cuenta de forma permanente para que mis datos personales sean borrados del sistema.		
TAREAS	• Crear un apartado de configuración en la interfaz de mi perfil de usuario • Implementar un modal en que se pida una doble confirmación de la acción para evitar borrados accidentales • Crear el servicio encargado de la eliminación de los datos del usuario en la base de datos • Asegurar el cierre de sesión automático y la redirección a la página de inicio tras la eliminación		
PRUEBAS DE ACEPTACIÓN	PA-13		

Tabla 3.12. *HU-12*

HU-13			
TÍTULO	Visualización de perfiles de otros usuario		
PRIORIDAD	Media	ESTIMACIÓN	1
DESCRIPCIÓN	Como usuario, quiero ver el perfil de otros para conocer sobre ellos y decidir si quiero agregarlos.		
TAREAS	• Modificar la interfaz anterior de perfil de usuario para que sirva como visualizador tanto de perfil propio como de perfil ajeno, restringiendo las funciones de edición si es ajeno		
PRUEBAS DE ACEPTACIÓN	PA-14		

Tabla 3.13. *HU-13*

3.1.4. Módulo de Correspondencia

HU-14			
TÍTULO	Creación de cartas		
PRIORIDAD	Alta	ESTIMACIÓN	5
DESCRIPCIÓN	Como usuario, quiero escribir cartas en diferentes idiomas otorgándoles un título, una fecha, una especificación del idioma en que están escritas, su contenido; además de tener la opción de clasificarlas por diario.		
TAREAS	• Crear una interfaz que permita la creación de una carta nueva, que cuente con un editor que permitan especificar todos los datos de una carta (título, contenido, fecha, diario, idioma) • Crear la lógica que guarde los datos de una carta en la base de datos		
PRUEBAS DE ACEPTACIÓN	PA-15		

Tabla 3.14. HU-14

HU-15			
TÍTULO	Gestión de borradores		
PRIORIDAD	Alta	ESTIMACIÓN	2
DESCRIPCIÓN	Como autor, quiero guardar mi progreso para terminar la carta en otro momento.		
TAREAS	• Incluir en la interfaz de creación de carta la opción de guardar la carta • Crear una interfaz que permita acceder a una carta, editarla y guardar los cambios		
PRUEBAS DE ACEPTACIÓN	PA-15		

Tabla 3.15. HU-15

HU-16			
TÍTULO	Envío de cartas		
PRIORIDAD	Alta	ESTIMACIÓN	2
DESCRIPCIÓN	Como autor, quiero compartir mis cartas con amigos.		
TAREAS	• Incluir en la interfaz de edición de carta la opción de enviarla a amigos • Crear el servicio para compartir una carta con otro usuario		
PRUEBAS DE ACEPTACIÓN	PA-16, PA-17		

Tabla 3.16. HU-16

HU-17			
TÍTULO	Listado y filtrado de cartas enviadas		
PRIORIDAD	Alta	ESTIMACIÓN	5
DESCRIPCIÓN	Como usuario, quiero ver la lista de mis cartas, con la opción de aplicar filtros para encontrar cartas determinadas más fácilmente.		
TAREAS	• Crear una interfaz que muestre las cartas escritas por el usuario • Añadir barras con herramientas de filtrado por título, idioma o diario • Añadir opción de visualizar la lista ordenada por diarios, y la lista de diarios • Crear el servicio para acceder al listado de cartas • Implementar paginación y filtrado en este servicio		
PRUEBAS DE ACEPTACIÓN	PA-18		

Tabla 3.17. *HU-17*

HU-18			
TÍTULO	Eliminación lógica de cartas		
PRIORIDAD	Baja	ESTIMACIÓN	2
DESCRIPCIÓN	Como usuario, quiero poder borrar mis cartas sin que desaparezcan del buzón de otros.		
TAREAS	• Añadir a la interfaz con la lista de cartas creadas por el usuario la opción de borrar una selección de cartas • Pedir confirmación del usuario antes de realizar el borrado • Crear los servicios encargados del borrado lógico de cartas		
PRUEBAS DE ACEPTACIÓN	PA-19, PA-20		

Tabla 3.18. *HU-18*

3.1.5. Módulo de Correcciones

HU-19			
TÍTULO	Listado y filtrado de cartas para corregir		
PRIORIDAD	Alta	ESTIMACIÓN	3
DESCRIPCIÓN	Como revisor, quiero ver qué cartas han compartido conmigo para empezar a corregir, con la opción de aplicar filtros para encontrar cartas determinadas fácilmente, y pudiendo ver claramente cuáles de esas cartas ya han sido corregidas y cuáles no.		
TAREAS	• Añadir a la interfaz de cartas creadas, un apartado con la lista de cartas recibidas • Incluir en esta lista una distinción por colores de carta corregida o pendiente • Añadir barras con herramientas de filtrado por título, remitente o pendiente de corregir • Crear el servicio para acceder al listado de cartas recibidas • Implementar paginación y filtrado en este servicio		
PRUEBAS DE ACEPTACIÓN	PA-21		

Tabla 3.19. HU-19

HU-20			
TÍTULO	Eliminación de cartas recibidas		
PRIORIDAD	Baja	ESTIMACIÓN	1
DESCRIPCIÓN	Como revisor, quiero poder borrar de mi buzón de recibidas aquellas cartas que han sido eliminadas por el usuario que me las envió, en caso de no querer conservar la carta y las correcciones que he aplicado.		
TAREAS	• Añadir a la interfaz con la lista de cartas recibidas la opción de eliminar una carta que ha sido eliminada por su autor • Crear el servicio encargado del borrado físico de cartas (si la carta es eliminada tanto por creador como por remitente, no hay necesidad de mantener el registro en la base de datos)		
PRUEBAS DE ACEPTACIÓN	PA-22		

Tabla 3.20. HU-20

HU-21			
TÍTULO	Realización de corrección de texto		
PRIORIDAD	Alta	ESTIMACIÓN	4
DESCRIPCIÓN	Como revisor, quiero añadir correcciones sobre el texto de mi amigo para señalar sus errores u ofrecer sugerencias.		
TAREAS	• Crear una interfaz de corrección que permita visualizar una carta recibida y añadir correcciones sobre su contenido • Incluir en dicha interfaz la opción de guardar la corrección para editarla más adelante • Crear el servicio para guardas las correcciones y acceder a ellas		
PRUEBAS DE ACEPTACIÓN	PA-23		

Tabla 3.21. HU-21

HU-22			
TÍTULO	Añadir comentarios finales a la corrección		
PRIORIDAD	Media	ESTIMACIÓN	1
DESCRIPCIÓN	Como revisor, quiero escribir una nota explicativa al final de mi corrección.		
TAREAS	• Incluir en la interfaz de corrección un campo para añadir retroalimentación o comentarios finales • Incluir ese campo en el servicio de guardado y acceso a cartas corregidas		
PRUEBAS DE ACEPTACIÓN	PA-23		

Tabla 3.22. HU-22

HU-23			
TÍTULO	Devolución de la corrección al autor		
PRIORIDAD	Alta	ESTIMACIÓN	1
DESCRIPCIÓN	Como revisor, quiero marcar la corrección como enviada para que mi amigo la vea.		
TAREAS	• Añadir en la interfaz de corrección la opción de enviar la corrección de vuelta: un botón "Send Back" • Crear el servicio para enviar la carta de vuelta a su autor con las correcciones y comentarios añadidos		
PRUEBAS DE ACEPTACIÓN	PA-23		

Tabla 3.23. HU-23

HU-24			
TÍTULO	Visualización de correcciones recibidas		
PRIORIDAD	Alta	ESTIMACIÓN	2
DESCRIPCIÓN	Como remitente, quiero recibir la corrección de la carta que envié a mi amigo una vez este la haya corregido y enviado de vuelta.		
TAREAS	• Añadir a las tarjetas de la interfaz de listas de cartas un botón por cada corrección recibida para esa carta • Crear una página de visualización de corrección, similar a la de corrección pero no editable • Añadir un link hacia esa página al click del botón mencionado anteriormente • Crear el servicio para obtener la información de la corrección de una carta		
PRUEBAS DE ACEPTACIÓN	PA-24		

Tabla 3.24. *HU-24*

3.1.6. Funcionalidad Social

HU-25			
TÍTULO	Listado de usuarios sugeridos		
PRIORIDAD	Alta	ESTIMACIÓN	3
DESCRIPCIÓN	Como usuario, quiero que al plataforma me sugiera usuarios que puedo añadir como amigos basándose en intereses lingüísticos comunes.		
TAREAS	• Crear una interfaz que muestre una lista de usuarios sugeridos • Crear el servicio para acceder y crear dicha lista mediante un algoritmo de recomendación personalizado		
PRUEBAS DE ACEPTACIÓN	PA-25		

Tabla 3.25. *HU-25*

HU-26			
TÍTULO	Búsqueda de usuarios por nombre de usuario		
PRIORIDAD	Alta	ESTIMACIÓN	1
DESCRIPCIÓN	Como usuario, quiero buscar personas para agregarlas a mi lista de amigos.		
TAREAS	• Incluir en la interfaz de amistades una barra de búsqueda para realizar la tarea • Crear la lógica para obtener una lista de usuarios a partir de una búsqueda		
PRUEBAS DE ACEPTACIÓN	PA-26		

Tabla 3.26. HU-26

HU-27			
TÍTULO	Gestión de solicitudes de amistad		
PRIORIDAD	Alta	ESTIMACIÓN	2
DESCRIPCIÓN	Como usuario, quiero solicitar amistad a otros usuarios para intercambiar cartas y correcciones; y rechazar o aceptar las solicitudes de amistad que me lleguen.		
TAREAS	• Añadir a las tarjetas de usuarios sugeridos o buscados de la interfaz de amistades un botón para enviar una solicitud de amistad • Crear el servicio para enviar una solicitud de amistad • Añadir un componente de interfaz que muestre la lista de solicitudes de amistad recibidas • Incluir en cada tarjeta de solicitud información básica del usuario que la envió y botones con las opciones de rechazarla o aceptarla • Crear los servicios para rechazar y aceptar solicitudes de amistad		
PRUEBAS DE ACEPTACIÓN	PA-27, PA-28		

Tabla 3.27. HU-27

HU-28			
TÍTULO	Visualización y gestión del listado de amigos		
PRIORIDAD	Baja	ESTIMACIÓN	1
DESCRIPCIÓN	Como usuario, quiero ver quiénes son mis amigos actuales para saber con quién puedo interactuar, así como eliminar un contacto si ya no es de mi interés intercambiar cartas con él.		
TAREAS	• Añadir un componente de interfaz que muestre la lista de amigos del usuario • Crear el servicio para acceder a la lista de amigos del usuario • Añadir un componente de interfaz que permita eliminar un amigo de la lista de contactos • Crear el servicio para eliminar una relación entre usuarios		
PRUEBAS DE ACEPTACIÓN	PA-29, PA-30		

Tabla 3.28. *HU-28*

3.2. Pruebas de Aceptación

En esta sección se detallan los procedimientos de prueba para validar las Historias de Usuario. Cada prueba cuenta con un identificador (PA-XX), un título, las Historias de Usuario que cubre, una descripción, y el procedimiento a seguir para realizarla. No se especifica la compleción de estas ya que todas ellas han sido completadas con éxito. Se encuentran divididas en las mismas secciones que las Historias de Usuario.

3.2.1. Investigación y Documentación

PA-01	
TÍTULO	Aceptación de la propuesta
HU IMPLICADAS	HU-05
DESCRIPCIÓN	Obtención de una validación positiva por parte del tutor de las historias de usuario.
PROCEDIMIENTO	1. Redacción de historias de usuario 2. Validación de estas en una revisión con el tutor

Tabla 3.29. PA-01

PA-02	
TÍTULO	Aceptación de la estructura y contenido del documento
HU IMPLICADAS	HU-01, HU-02, HU-03, HU-04, HU-05, HU-06, HU-07
DESCRIPCIÓN	Obtención de una validación positiva por parte del tutor de la estructura planteada en el documento, el esqueleto de las diferentes secciones y su contenido.
PROCEDIMIENTO	El tutor valida que la documentación cumple con los requisitos establecidos.

Tabla 3.30. PA-02

3.2.2. Gestión de Usuarios

PA-03	
TÍTULO	Registro en la plataforma como nuevo usuario
HU IMPLICADAS	HU-08
DESCRIPCIÓN	Asegurar que un nuevo usuario puede registrarse y que el sistema impide el registro hasta que el correo es verificado.
PROCEDIMIENTO	1. El usuario accede a la URL de la aplicación a través de su navegador. 2. El usuario selecciona el botón "Register" para iniciar el proceso de registro. 3. El usuario introduce un nombre de usuario y una contraseña. 4. El sistema valida los datos introducidos. En caso de que el nombre tenga menos de 5 caracteres o la contraseña menos de 8, se muestra un mensaje de error. 5. El usuario introduce un correo electrónico para vincular la cuenta. 6. El sistema envía un código de verificación al correo electrónico proporcionado. 7. El usuario introduce el código de verificación en la interfaz. 8. El sistema valida el código. En caso de ser incorrecto, se muestra un mensaje de error. 9. El usuario selecciona los idiomas que domina. En caso de no seleccionar ninguno, se muestra un mensaje de error. 10. El usuario selecciona los idiomas que desea aprender. En caso de no seleccionar ninguno, se muestra un mensaje de error. 11. El sistema muestra un resumen de los datos introducidos, incluyendo la opción de añadir una imagen de perfil. 12. El usuario, de forma opcional, añade una imagen de perfil y selecciona el botón "All set" para finalizar el registro. 13. El sistema crea el nuevo usuario con los datos proporcionados y almacena la imagen de perfil en caso de haber sido añadida.

Tabla 3.31. PA-03

PA-04	
TÍTULO	Inicio de sesión en la plataforma
HU IMPLICADAS	HU-08
DESCRIPCIÓN	Asegurar que el usuario puede acceder a la plataforma introduciendo sus credenciales.
PROCEDIMIENTO	1. El usuario selecciona el botón "Log in" para acceder al formulario de inicio de sesión. 2. El usuario introduce su nombre de usuario o correo electrónico, junto con su contraseña. 3. El sistema valida las credenciales introducidas. 4. En caso de ser correctas, el usuario es redirigido a la página principal. En caso contrario, el sistema muestra un mensaje de error en el formulario.

Tabla 3.32. PA-04

PA-05	
TÍTULO	Flujo de Recuperación de Credenciales
HU IMPLICADAS	HU-10
DESCRIPCIÓN	Validar que un usuario que ha olvidado su contraseña puede restablecerla mediante código por email.
PROCEDIMIENTO	1. El usuario accede a la URL de la aplicación a través de su navegador. 2. El usuario selecciona el botón "Log in" para acceder al formulario de inicio de sesión. 3. El usuario localiza la opción "Forgot password" y la selecciona. 4. El sistema muestra un formulario en el que el usuario introduce el correo electrónico vinculado a su cuenta. 5. El sistema envía un correo electrónico con un código de verificación. 6. El usuario introduce dicho código en el formulario correspondiente. 7. El sistema valida el código. En caso de no ser válido, se muestra un mensaje de error. 8. El sistema muestra un formulario para introducir una nueva contraseña y su confirmación. 9. El usuario introduce la nueva contraseña y su confirmación. 10. El sistema valida que la contraseña tiene al menos 8 caracteres y que ambas coinciden. En caso contrario, se muestra un mensaje de error. 11. El usuario es redirigido al formulario de inicio de sesión. 12. El usuario introduce sus nuevas credenciales y accede correctamente a la aplicación.

Tabla 3.33. PA-05

PA-06	
TÍTULO	Persistencia de sesión
HU IMPLICADAS	HU-08
DESCRIPCIÓN	Comprobar que el token de autenticación persiste en el navegador aunque este se cierre.
PROCEDIMIENTO	1. El usuario accede a la aplicación e inicia sesión correctamente. 2. El usuario cierra el navegador. 3. El usuario vuelve a abrir el navegador e introduce la URL de la aplicación. 4. El sistema reconoce la sesión del usuario. 5. El usuario es dirigido a la página principal con sus datos cargados correctamente.

Tabla 3.34. PA-06

PA-07	
TÍTULO	Menú de Navegación
HU IMPLICADAS	HU-09
DESCRIPCIÓN	Validar la existencia y funcionamiento del menú de navegación desplegable.
PROCEDIMIENTO	1. El usuario accede a la aplicación e inicia sesión correctamente. 2. El usuario desplaza el cursor hacia el menú de navegación situado en la parte izquierda de la ventana. 3. El sistema muestra el menú de navegación, que incluye las páginas Página Principal, Mi Perfil y Amigos, así como el logo de la aplicación y el botón de cierre de sesión. 4. El usuario navega por las distintas páginas seleccionando sus enlaces y se comprueba que conducen a la página correspondiente.

Tabla 3.35. PA-07

PA-08	
TÍTULO	Cierre de sesión
HU IMPLICADAS	HU-09
DESCRIPCIÓN	Validar el cierre de sesión.
PROCEDIMIENTO	1. El usuario accede a la aplicación con una sesión iniciada y desplaza el cursor hacia el menú de navegación situado en la parte izquierda de la ventana. 2. El usuario selecciona la opción de cierre de sesión "Log out". 3. El sistema cierra la sesión y redirige al usuario a la página de inicio de la aplicación. 4. El usuario abre una nueva ventana del navegador y se muestra la página de inicio de sesión de la aplicación.

Tabla 3.36. PA-08

PA-09	
TÍTULO	Protección de rutas
HU IMPLICADAS	HU-08
DESCRIPCIÓN	Validar el correcto funcionamiento de las rutas protegidas de la aplicación, asegurando que un usuario no autenticado no puede acceder a ellas y es redirigido a la página de inicio de sesión.
PROCEDIMIENTO	1. El usuario accede a una URL correspondiente a una página protegida sin haber iniciado sesión. 2. El sistema redirige automáticamente al usuario a la página de inicio de sesión.

3.2.3. Gestión de Perfiles

PA-10	
TÍTULO	Visualización del Perfil Propio
HU IMPLICADAS	HU-11
DESCRIPCIÓN	Comprobar la existencia y propiedad de la página de perfil del usuario.
PROCEDIMIENTO	1. El usuario inicia sesión en la plataforma. 2. El usuario abre el menú desplegable y hace clic en Profile. 3. El usuario es dirigido a la página de perfil donde puede ver su información de perfil (nombre, email, biografía, lenguajes, estadísticas y mapa con su localidad) 4. El usuario puede ver una barra de herramientas con botones para editar el perfil (edit) y acceder a ajustes de configuración de cuenta (Settings).

Tabla 3.37. PA-10

PA-11	
TÍTULO	Edición del Perfil Propio
HU IMPLICADAS	HU-11
DESCRIPCIÓN	Comprobar que los campos de la página de perfil de usuario pueden ser editados correctamente.
PROCEDIMIENTO	1. El usuario inicia sesión en la plataforma. 2. El usuario accede al menú de navegación y selecciona Profile. 3. El sistema muestra la página de perfil con la información del usuario. 4. El usuario hace clic en Edit. 5. El sistema activa el modo edición de los campos del perfil. 6. El usuario modifica los campos deseados (biografía, lenguajes, localidad e imagen) y pulsa Save. 7. El sistema muestra una confirmación de guardado y actualiza los datos. 8. El usuario recarga la página. 9. El sistema muestra la información actualizada. 10. Se verifica en la base de datos que los cambios han sido persistidos correctamente.

Tabla 3.38. PA-11

PA-12	
TÍTULO	Cambio de contraseña
HU IMPLICADAS	HU-11
DESCRIPCIÓN	Comprobar la existencia y correcto funcionamiento del panel de cambio de contraseña.
PROCEDIMIENTO	1. El usuario inicia sesión en la plataforma. 2. El usuario accede a Profile y abre el panel Settings. 3. El sistema muestra las opciones de configuración, incluyendo el cambio de contraseña. 4. El usuario accede al formulario de cambio de contraseña. 5. El usuario realiza intentos de envío con datos inválidos (contraseñas incorrectas, no coincidentes o campos incompletos). 6. El sistema rechaza los intentos y muestra mensajes de error adecuados. 7. El usuario introduce la contraseña actual correcta, la nueva contraseña y su confirmación, y pulsa Save. 8. El sistema confirma el cambio de contraseña. 9. El usuario cierra sesión e intenta acceder con credenciales antiguas, siendo rechazado. 10. El usuario accede exitosamente con las nuevas credenciales.

Tabla 3.39. PA-12

PA-13	
TÍTULO	Eliminación de cuenta
HU IMPLICADAS	HU-12
DESCRIPCIÓN	Comprobar la existencia y correcto funcionamiento del panel de eliminación de cuenta.
PROCEDIMIENTO	1. El usuario inicia sesión en la plataforma. 2. El usuario accede a la página Profile y abre el panel Settings. 3. El sistema muestra las opciones de configuración, incluyendo la eliminación de cuenta. 4. El usuario selecciona la opción Delete account. 5. El sistema muestra una advertencia de acción irreversible y solicita la contraseña del usuario. 6. El usuario introduce una contraseña incorrecta y el sistema muestra un mensaje de error. 7. El usuario introduce la contraseña correcta y confirma la eliminación. 8. El sistema elimina la cuenta y redirige al usuario a la página de inicio. 9. El usuario intenta iniciar sesión nuevamente y el sistema rechaza el acceso indicando que el usuario no existe. 10. Se verifica en la base de datos que la cuenta ha sido eliminada y que sus cartas han sido marcadas como eliminadas (borrado lógico).

Tabla 3.40. PA-13

PA-14	
TÍTULO	Visualización de Perfil Ajeno
HU IMPLICADAS	HU-13
DESCRIPCIÓN	Comprobar la visualización del perfil de otro usuario y la restricción de edición sobre el mismo.
PROCEDIMIENTO	1. El usuario inicia sesión en la plataforma. 2. El usuario accede a Friends. 3. El usuario selecciona el perfil de otro usuario. 4. El sistema muestra el perfil del usuario seleccionado con su información (nombre, fecha de registro, biografía, lenguajes, estadísticas y localización). 5. El sistema no muestra opciones de edición ni configuración en este perfil.

Tabla 3.41. PA-14

3.2.4. Módulo de Correspondencia

PA-15	
TÍTULO	Gestión de Borradores y Persistencia de Cartas Creadas
HU IMPLICADAS	HU-14, HU-15
DESCRIPCIÓN	Validar que una carta que se ha creado puede ser guardada parcialmente y retomada posteriormente sin perder información.
PROCEDIMIENTO	1. El usuario inicia sesión en la plataforma. 2. El usuario accede a la creación de una nueva carta mediante el botón New de la página principal. 3. El sistema redirige a la página de creación de carta, con los campos título, fecha, diario, idioma y contenido. 4. El usuario intenta guardar la carta sin completar todos los campos obligatorios. 5. El sistema impide el guardado y muestra un mensaje de validación. 6. El usuario completa todos los campos obligatorios (título, fecha, idioma y contenido; el diario es opcional) y pulsa Save. 7. El sistema guarda la carta, muestra un mensaje de confirmación y redirige a la página de edición de carta, donde muestra los datos tal y como que se han guardado. 8. El usuario modifica alguno de los campos y el sistema vuelve a habilitar la opción de guardado. 9. El usuario navega a la página principal y posteriormente vuelve a la carta desde el listado. 10. El sistema muestra la carta con los datos tal y como fueron guardados.

Tabla 3.42. PA-15

PA-16	
TÍTULO	Envío de Cartas a Contactos
HU IMPLICADAS	HU-16
DESCRIPCIÓN	Asegurar que una carta puede ser compartida con otros usuarios desde la página de edición.
PROCEDIMIENTO	1. El usuario inicia sesión en la plataforma. 2. El usuario accede a una carta desde el listado o crea una nueva y la guarda. 3. El sistema muestra la carta junto con la opción Share letter. 4. El usuario selecciona la opción de compartir. 5. El sistema muestra un diálogo con la lista de contactos del usuario para seleccionar destinatarios. 6. El usuario selecciona contactos y confirma el envío mediante Send. 7. El sistema registra el envío, actualiza el estado de la carta a "Enviada", cierra el diálogo y deshabilita la opción de editar. 8. El usuario vuelve a abrir la opción de compartir y el sistema muestra los contactos ya seleccionados como no disponibles para selección.

Tabla 3.43. PA-16

PA-17	
TÍTULO	Recepción de Cartas Compartidas
HU IMPLICADAS	HU-16
DESCRIPCIÓN	Asegurar que una carta enviada por otro usuario aparece correctamente en el buzón del usuario destinatario.
PROCEDIMIENTO	1. El usuario destinatario inicia sesión en la plataforma. 2. El sistema muestra en el buzón de cartas recibidas una carta enviada por otro usuario, con su título, idioma, fecha de envío y remitente. 3. El usuario clica la carta recibida. 4. El sistema redirige a la página de corrección, que muestra los detalles de la carta enviada por el remitente.

Tabla 3.44. PA-17

PA-18	
TÍTULO	Filtrado y Paginación de Cartas Enviadas
HU IMPLICADAS	HU-17
DESCRIPCIÓN	Verificar el correcto funcionamiento de la paginación, filtrado y ordenación del listado de cartas propias.
PROCEDIMIENTO	1. El usuario inicia sesión en la plataforma con al menos 6 cartas creadas. 2. El sistema muestra el listado de cartas propias paginado (5 elementos por página). 3. El usuario navega entre páginas y el sistema actualiza correctamente los elementos mostrados. 4. El usuario utiliza la barra de búsqueda. 5. El sistema filtra las cartas cuyo título o idioma coincide total o parcialmente con la búsqueda. 6. El sistema muestra las cartas ordenadas de más recientes a más antiguas según su fecha.

Tabla 3.45. PA-18

PA-19	
TÍTULO	Borrado de Cartas No Compartidas
HU IMPLICADAS	HU-18
DESCRIPCIÓN	Comprobar que al eliminar una carta no compartida, esta desaparece del listado y su registro se elimina de la base de datos (borrado físico).
PROCEDIMIENTO	1. El usuario inicia sesión en la plataforma con al menos una carta no compartida. 2. El usuario activa el modo de eliminación desde el listado de cartas. 3. El sistema permite seleccionar la carta a eliminar. 4. El usuario selecciona una o varias cartas no compartidas y confirma la acción de borrado. 5. El sistema solicita confirmación de la eliminación. 6. El usuario confirma la acción. 7. El sistema elimina las cartas del listado y del sistema, manteniendo el cambio tras recargar la página o reiniciar sesión. 8. Se verifica en la base de datos que el registro de las cartas ha sido eliminado.

Tabla 3.46. PA-19

PA-20	
TÍTULO	Borrado de Cartas Compartidas
HU IMPLICADAS	HU-18
DESCRIPCIÓN	Comprobar que al borrar una carta compartida, esta desaparece del listado del remitente pero permanece accesible para el destinatario como carta eliminada (borrado lógico).
PROCEDIMIENTO	1. El usuario remitente con al menos una carta compartida inicia sesión. 2. El usuario selecciona una carta compartida y confirma su eliminación. 3. El sistema elimina la carta del listado del remitente y la marca como eliminada sin borrar su registro. 4. Se verifica en la base de datos que la carta existe pero está marcada como eliminada. 5. El usuario destinatario inicia sesión. 6. El sistema muestra la carta en su buzón de recibidas marcada como eliminada por el autor, pero accesible. 7. El usuario abre la carta y el sistema permite su visualización y la adición de correcciones, pero sin opción de enviarla de vuelta.

Tabla 3.47. PA-20

3.2.5. Módulo de Correcciones

PA-21	
TÍTULO	Gestión y Filtrado del Buzón de Cartas Recibidas
HU IMPLICADAS	HU-19
DESCRIPCIÓN	Validar que el buzón de cartas recibidas permite distinguir visualmente el estado de las cartas y aplicar filtros por estado (corregida, pendiente o eliminada por el autor), remitente y título, junto con paginación del listado.
PROCEDIMIENTO	1. El usuario con al menos 6 cartas recibidas inicia sesión en la plataforma. 2. El sistema muestra el buzón de cartas recibidas junto con un panel de búsqueda y filtros por estado y remitente. 3. El sistema diferencia visualmente las cartas según su estado. 4. El usuario aplica uno o varios filtros (estado, remitente o búsqueda por título), individual o conjuntamente, y el sistema actualiza el listado mostrando únicamente las cartas que cumplen los criterios aplicados. 5. El usuario navega entre páginas y el sistema mantiene correctamente el filtrado sobre los resultados paginados.

Tabla 3.48. PA-21

PA-22	
TÍTULO	Borrado de Cartas Recibidas
HU IMPLICADAS	HU-20
DESCRIPCIÓN	Comprobar que al eliminar una carta recibida que ya ha sido eliminada por su remitente, esta desaparece del buzón del destinatario y de la base de datos (borrado físico).
PROCEDIMIENTO	1. El usuario destinatario de una carta borrada por su autor inicia sesión en la plataforma. 2. El sistema muestra en su buzón dicha carta, con la distinción visual de las eliminadas por el autor y un botón de borrado. 3. El usuario elimina la carta desde su lista de cartas recibidas. 4. El sistema elimina la carta del listado del destinatario y de la base de datos. 5. Se verifica que la eliminación persiste tras recargar la página o reiniciar sesión.

Tabla 3.49. PA-22

PA-23	
TÍTULO	Página de Corrección y Envío de Carta Corregida
HU IMPLICADAS	HU-21, HU-22, HU-23
DESCRIPCIÓN	Validar el proceso de corrección de una carta recibida, incluyendo la adición de anotaciones sobre el texto, comentarios finales y el envío de la corrección al autor.
PROCEDIMIENTO	1. El usuario destinatario inicia sesión en la plataforma. 2. El usuario accede a una carta pendiente de corrección desde su buzón. 3. El sistema muestra la vista de corrección con el contenido de la carta. 4. El usuario activa el modo de corrección y añade correcciones sobre el texto recibido. 5. El usuario añade un comentario final a la carta. 6. El usuario envía la corrección mediante la opción Send back y confirma la acción. 7. El sistema marca la carta como corregida y enviada de vuelta al autor. 8. El usuario remitente inicia sesión. 9. El sistema muestra la carta con una corrección disponible en su panel de cartas.

Tabla 3.50. PA-23

PA-24	
TÍTULO	Recepción de Carta Corregida y Visualización de Correcciones
HU IMPLICADAS	HU-24
DESCRIPCIÓN	Validar la recepción de una corrección, de forma que esta quede accesible desde el listado de cartas del usuario remitente.
PROCEDIMIENTO	• El usuario, remitente de una carta previamente corregida y enviada de vuelta por otro usuario, inicia sesión en la plataforma. • El sistema muestra en el listado de cartas una indicación de que la carta tiene una corrección disponible. • El usuario clicla dicho indicador y el sistema redirige a la página de visualización de corrección, que muestra las anotaciones realizadas por el corrector y sus comentarios finales.

Tabla 3.51. PA-24

3.2.6. Funcionalidad Social

PA-25	
TÍTULO	Interfaz de Usuarios Sugeridos
HU IMPLICADAS	HU-25
DESCRIPCIÓN	Comprobar que el sistema recomienda usuarios con intereses afines.
PROCEDIMIENTO	1. El usuario inicia sesión en la plataforma. 2. El usuario accede a la página Friends del menú desplegable. 3. El sistema muestra un panel de usuarios sugeridos en función de afinidad de idiomas. 4. El sistema prioriza usuarios con coincidencias de idiomas con el usuario y posteriormente muestra otros usuarios. 5. El sistema no incluye en las sugerencias usuarios que ya formen parte de la lista de amigos del usuario.

Tabla 3.52. PA-25

PA-26	
TÍTULO	Búsqueda de Usuarios
HU IMPLICADAS	HU-26
DESCRIPCIÓN	Comprobar que el buscador por nombre de usuario funciona correctamente.
PROCEDIMIENTO	1. El usuario inicia sesión en la plataforma. 2. El usuario accede a la página Friends del menú desplegable. 3. El sistema muestra el panel de usuarios sugeridos, con una barra de búsqueda en la parte superior. 4. El usuario introduce un texto en la barra de búsqueda. 5. El sistema muestra los usuarios cuyo nombre de usuario coincide total o parcialmente con el texto introducido, independientemente de si forman parte de las sugerencias iniciales.

Tabla 3.53. PA-26

PA-27	
TÍTULO	Flujo de Solicitud de Amistad
HU IMPLICADAS	HU-27
DESCRIPCIÓN	Verificar que el envío de la solicitud de amistad queda disponible para su aceptación o rechazo por parte del usuario receptor de dicha solicitud.
PROCEDIMIENTO	1. El usuario inicia sesión en la plataforma y accede a la página Friends del menú desplegable. 2. El usuario envía una solicitud de amistad a otro usuario desde el panel de sugerencias. 3. El sistema registra la solicitud enviada. 4. El usuario receptor inicia sesión y accede a la página Friends. 5. El sistema muestra la solicitud recibida en el panel de solicitudes de amistad, con opciones de aceptación y de rechazo.

Tabla 3.54. PA-27

PA-28	
TÍTULO	Aceptación y Rechazo de Solicitudes de Amistad
HU IMPLICADAS	HU-27
DESCRIPCIÓN	Comprobar que el usuario puede aceptar o rechazar solicitudes de amistad, actualizando correctamente su lista de amigos y eliminando la solicitud tras su gestión.
PROCEDIMIENTO	1. El usuario inicia sesión en la plataforma y accede a la página Friends. 2. El sistema muestra las solicitudes de amistad recibidas. 3. El usuario rechaza una solicitud y el sistema la elimina sin modificar la lista de amigos. 4. El sistema elimina el registro de la solicitud en la base de datos. 5. El usuario acepta otra solicitud y el sistema la elimina de la lista de solicitudes y añade al usuario correspondiente a su lista de amigos. 6. El sistema registra la nueva relación de amistad en la base de datos y elimina el registro de la solicitud.

Tabla 3.55. PA-28

PA-29	
TÍTULO	Visualización de la Lista de Amigos
HU IMPLICADAS	HU-28
DESCRIPCIÓN	Validar que el usuario puede visualizar su lista de contactos con paginación.
PROCEDIMIENTO	1. El usuario con al menos 7 amigos añadidos inicia sesión en la plataforma. 2. El usuario accede a la página Friends del menú desplegable. 3. El sistema muestra la lista de amigos en formato paginado (6 amigos por página). 4. El usuario navega entre páginas y el sistema actualiza correctamente los contactos mostrados.

Tabla 3.56. PA-29

PA-30	
TÍTULO	Eliminación de Contactos
HU IMPLICADAS	HU-28
DESCRIPCIÓN	Validar que el usuario puede eliminar un contacto de su lista de amigos.
PROCEDIMIENTO	1. El usuario con al menos un contacto añadido inicia sesión en la plataforma. 2. El usuario accede a la página <i>Friends</i> del menú desplegable. 3. El sistema muestra la lista de amigos del usuario. 4. El usuario elimina un contacto desde su lista de amigos. 5. El sistema actualiza la lista de amigos de forma que el contacto eliminado ya no aparece en la lista. 6. El sistema elimina la relación de amistad de la base de datos.

Tabla 3.57. PA-30

3.3. Matriz de Trazabilidad

En esta tabla se presenta la correspondencia entre las Historias de Usuario y las Pruebas de Aceptación. Se trata de la matriz de trazabilidad que ayuda a verificar que cada Historia de Usuario está cubierta por sus correspondientes Pruebas de Aceptación (una o varias), de forma que no quede ninguna sin probar y validar. Como se puede observar, todas las Historias de Usuario quedan cubiertas.

	Historias de Usuario (HU)																											
	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
01					✓																							
02	✓	✓	✓	✓	✓	✓	✓																					
03								✓																				
04								✓																				
05										✓																		
06								✓																				
07									✓																			
08								✓	✓																			
09								✓																				
10										✓																		
11										✓																		
12										✓																		
13											✓																	
14												✓																
15													✓	✓														
16														✓														
17														✓														
18															✓													
19																✓												
20																✓												
21																	✓											
22																		✓										
23																			✓	✓	✓							
24																					✓							
25																						✓						
26																							✓					
27																									✓			
28																										✓		
29																											✓	
30																												✓

Tabla 3.58. Matriz de trazabilidad

3.4. Planificación

La planificación del proyecto se ha realizado a partir del conjunto de historias de usuario definidas previamente, cubriendo todas las fases del desarrollo: análisis, diseño, implementación, validación y documentación.

Cada historia de usuario ha sido estimada en puntos de historia, considerando que:

- 1 punto de historia equivale aproximadamente a 4 horas de trabajo.
- La dedicación media al proyecto ha sido de 2 horas diarias.
- Esto supone unas 14 horas semanales, es decir, 56 horas mensuales.

El desarrollo del proyecto se ha extendido a lo largo de **5 meses**, organizados en 4 sprints según la metodología Scrum. Aunque inicialmente se planteó una duración homogénea para los sprints, la planificación se ajustó en función de la naturaleza de las tareas. En particular, el tercer sprint presenta una mayor duración al concentrar tareas de mayor complejidad y dependencia entre sí, lo que requirió más tiempo de desarrollo y pruebas. Esta decisión permite mantener una coherencia funcional dentro del sprint y evitar la fragmentación de funcionalidades relacionadas.

La asignación de historias de usuario a cada sprint se ha realizado teniendo en cuenta su prioridad, dependencias y complejidad, permitiendo una evolución progresiva del sistema desde las funcionalidades más básicas hasta las más avanzadas, y finalizando con la documentación del proceso y el resultado. Por tanto, la planificación se estructura en los siguientes 4 sprints:

- **Sprint 1 (01/01/2026 – 31/01/2026).** En este primer sprint se han abordado principalmente tareas de análisis, investigación y definición del proyecto. Se incluyen las historias de usuario HU-02, HU-03, HU-04 y HU-05.

Esfuerzo estimado: 14 puntos.

- **Sprint 2 (01/02/2026 – 28/02/2026).** En este sprint se ha trabajado en la gestión de usuarios: registro, inicio de sesión, recuperación de credenciales, y perfiles. Se incluyen HU-08, HU-09, HU-10, HU-11, HU-12 y HU-13. En esta fase se consolida la base funcional del proyecto dado que con un sistema sólido de usuarios con acceso a la plataforma se habilita la construcción del resto de funcionalidades y su validación.

Esfuerzo estimado: 14 puntos.

- **Sprint 3 (01/03/2026 – 30/04/2026).** Este sprint se ha centrado en el núcleo del proyecto: el desarrollo de la lógica principal de correspondencia y corrección de cartas. Se incluyen once historias, de la HU-14 a la HU-24. Se trata del grueso del

código, por lo que su compleción requiere una cantidad más elevada de horas de trabajo.

Esfuerzo estimado: 28 puntos.

- **Sprint 4 (01/05/2026 – 31/05/2026).** En el último sprint se han desarrollado las funcionalidades sociales de la aplicación, como la adición y eliminación de contactos y la página de Amigos; además de completar las tareas de documentación. Las historias cubiertas son la HU-25, HU-26, HU-27 y HU-28 para ese primer conjunto de tareas, y la HU-01, HU-06 y HU-07 para la parte de redacción del documento.

Esfuerzo estimado: 14 puntos.

A continuación, se muestra un resumen de la planificación del proyecto del que se puede concluir que se emplearon un total de 280 horas (70 puntos).

Sprint	Fechas	Historias de Usuario	Puntos
Sprint 1	Enero 2026	HU-02 a HU-05	14
Sprint 2	Febrero 2026	HU-8 a HU-13	14
Sprint 3	Marzo - Abril 2026	HU-14 a HU-24	28
Sprint 4	Mayo 2026	HU-25 a HU-28, HU-01, HU-06, HU-07	14

Tabla 3.59. *Resumen de la planificación por sprints*

Para la gestión de la planificación se ha utilizado un tablero en Trello, donde las historias de usuario se han organizado en distintas columnas que reflejan su estado dentro del flujo de trabajo del proyecto.

Estas son las columnas utilizadas: *Product Backlog*, que contiene todas las historias de usuario pendientes de priorización o inicio; *Sprint Backlog*, que incluye las historias seleccionadas para el sprint actual; *En proceso*, para aquellas que se encuentran en fase de implementación; *En revisión*, donde se sitúan las funcionalidades finalizadas a la espera de validación en la Sprint Review con el tutor; y finalmente *Finalizado*, donde se encuentran las funcionalidades validadas y aceptadas.

Al final de cada sprint se realiza una revisión del incremento generado junto con el tutor, en la que se valida el funcionamiento de las funcionalidades desarrolladas y se decide su aceptación o posibles mejoras.

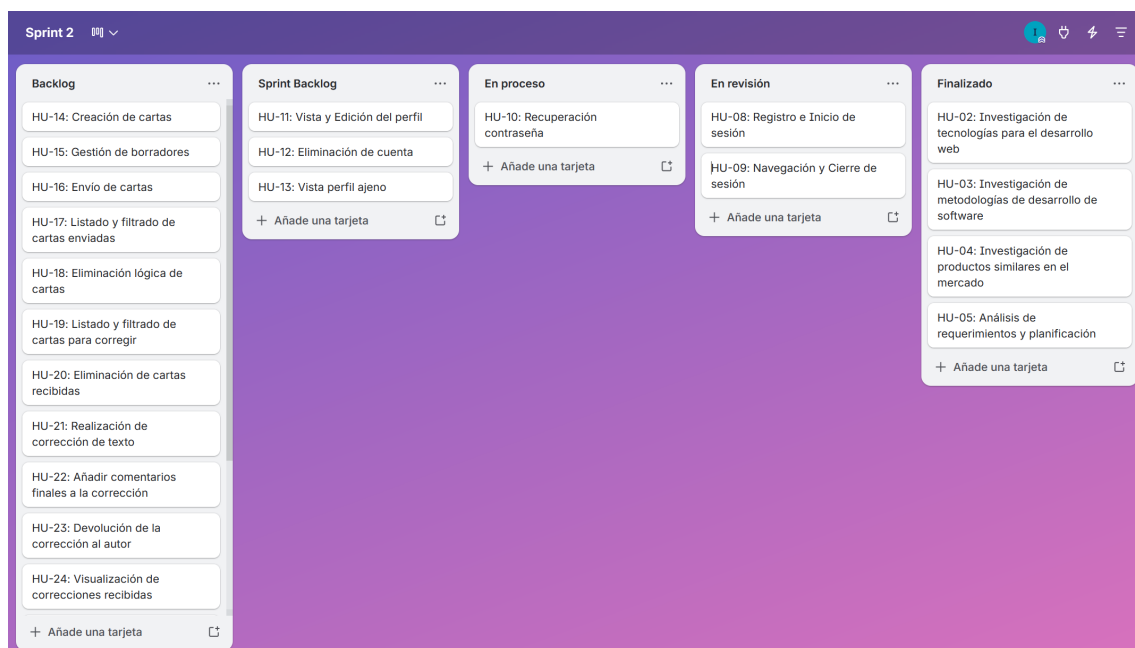


Fig. 3.1. *Ejemplo de sprint en Trello*

4. ARQUITECTURA DEL SOFTWARE

En el panorama actual del desarrollo de software, la construcción de una aplicación como la que se presenta en este Trabajo de Fin de Grado demanda una plataforma que sea no solo funcional, sino también interactiva, escalable y capaz de gestionar datos de forma dinámica. Para atender esta necesidad, este proyecto se ha desarrollado bajo una arquitectura full-stack, un enfoque integral que permite desacoplar la interfaz de usuario (frontend) de la lógica de negocio (backend) que accede y manipula la base de datos. Esta arquitectura se ilustra de forma esquemática en la siguiente figura.

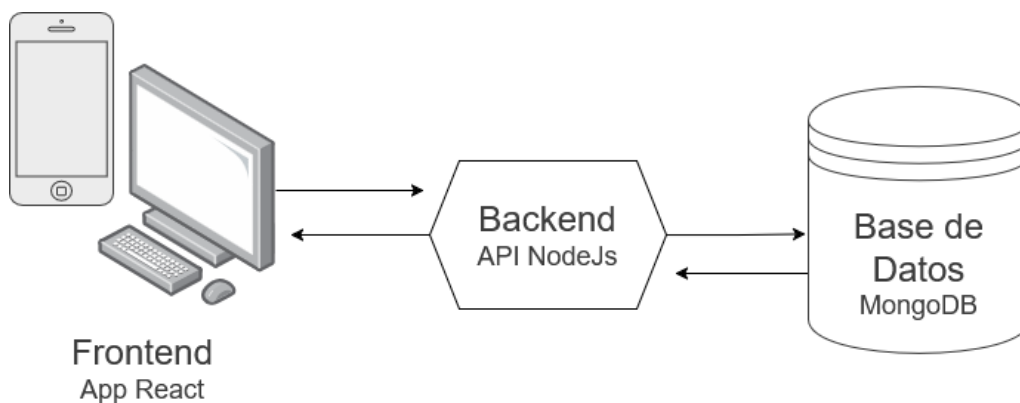


Fig. 4.1. *Arquitectura full-stack*

Al estructurar el backend exclusivamente como una API, se garantiza que el frontend sea una entidad independiente que consume recursos de forma asíncrona, optimizando la carga y mejorando la experiencia de usuario. Al decir que es una entidad independiente, se estrapola que esta API podría ser utilizada por otras aplicaciones que se construyan a futuro y quieran acceder a los servicios que la API ofrece. Esta es razón de más para testear los métodos de la API mediante la herramienta de pruebas Postman en lugar de solamente desde nuestro frontend, dado que nos ayudará a comprobar y asegurar el correcto funcionamiento de éstos de forma individual, precisa y documentada.

Además, para llevar a cabo esa tarea de optimización del frontend, se ha relegado el máximo de tareas lógicas y computacionales posibles al backend. Por ejemplo, si la aplicación de React precisa de un conjunto de datos paginados, es la API la que se encargará de la paginación de estos datos, es decir, de servir estos datos por páginas mediante métodos más precisos en lugar de entregar todos los datos de golpe y dejar que el frontend los pagine, puesto que esto lo ralentizaría.

Esta sección analiza el diseño de este ecosistema, detallándolo en estas cuatro secciones:

- **Stack seleccionado:** Justificación del conjunto de tecnologías escogido para el desarrollo de este proyecto de software.

- **Modelado de Datos NoSQL:** El uso de MongoDB y Mongoose para estructurar la información de forma flexible mediante documentos JSON modelados, facilitando el almacenamiento de estructuras de datos.
- **Arquitectura del backend (API REST):** Se explica la estructura del backend, así como puntos técnicos destacados como la automatización de envío de emails, el mecanismo de autenticación, el almacenamiento de imágenes, y varias técnicas de optimización que se han implementado; y el diseño de los endpoints (cada una de las ubicaciones donde la API recibe llamadas) y controladores que implementan las operaciones para gestionar de los datos.
- **Arquitectura del frontend (SPA):** Recoge el listado de tecnologías específicas empleadas para el frontend de la aplicación, la arquitectura del proyecto, las páginas de la interfaz y, de nuevo, las técnicas de optimización aplicadas.

4.1. Stack tecnológico seleccionado

El stack tecnológico es el conjunto de herramientas, programas, aplicaciones u otras piezas de software que son utilizadas para la creación de un producto tecnológico. Una vez estudiadas las principales opciones para el desempeño de cada sector del proyecto, se cuenta con una visión global lo suficientemente rica como para hacer una elección que se ajuste a las necesidades y preferencias del proyecto y su desarrollador.

En esta sección se expone cada punto de esta selección:

- Para el desarrollo del frontend de esta aplicación se ha optado por **Next.js** debido a sus ventajas en rendimiento, optimización para SEO y compatibilidad con React, que es una de las tecnologías más populares e innovadoras hoy en día, por lo que mejorar mis habilidades con React me es de gran interés. Entre los aspectos más atractivos de esta opción se encuentra también que cuenta con una plataforma de lanzamiento y hosting de aplicaciones gratuito, rápido y simple: Vercel; además de que la configuración inicial de un proyecto web se hace muy sencilla y existe una documentación rica, una comunidad activa, y numerosas librerías para facilitar el desarrollo web y numerosos recursos y componentes disponibles.
- En el backend, se ha elegido **Node.js con Express** por su flexibilidad y el beneficio de utilizar el mismo lenguaje de programación, JavaScript, en todo el stack de desarrollo. Además, se trata de una tecnología ampliamente utilizada en el desarrollo de microservicios y cuenta con librerías que facilitan el acceso al tipo de base de datos que se utilizará.
- En cuanto a la base de datos, se ha decidido emplear **MongoDB** por su modelo flexible y su capacidad para manejar estructuras de datos dinámicas y en formato JSON, el mismo en que se manejan las estructuras de información en JavaScript.

También, al ser las bases de datos relacionales aquellas que se han trabajado más durante el grado, me es de especial interés cultivar mis habilidades con modelos no relacionales como el de MongoDB.

- Para el consumo de APIs se usará **Axios** frente a FetchAPI por simplicidad y aceleración del desarrollo, dado que Axios ofrece más funcionalidades listas para usar como la conversión automática a formato JSON, el manejo de tokens o el manejo de errores integrado; mientras que Fetch, aunque es más ligero, requiere más código manual para tareas frecuentes. Dado que en esta aplicación se implementará autenticación de usuarios y una cantidad considerable de funciones, Axios se presenta como la opción más cómoda y compatible con el resto del stack.
- Como herramienta de pruebas de la API, se utiliza **Postman** frente a Insomnia por una razón principal: la comodidad de tenerlo ya instalado y estar familiarizada, al estar trabajando con ello en otros proyectos personales.

Calidad del Código

Adicionalmente, tanto en el frontend como en el backend, se han empleado varias herramientas destinadas a asegurar la calidad del código. En concreto, se emplean las siguientes:

- ESLint: herramienta de análisis estático configurada con reglas para mantener consistencia de código, es decir, un estilo limpio y homogéneo entre los distintos componentes, páginas y otros archivos auxiliares. También ayuda a detectar posibles errores en el código, como variables o importaciones no utilizadas.[80]
- TypeScript: proporciona verificación de tipos de datos en tiempo de compilación, lo cual permite detectar incompatibilidades en las propiedades de los componentes, llamadas a funciones mal tipadas, o en ocasiones el uso incorrecto de estados.
- Prettier: se trata de un formateador automático de código que garantiza una apariencia consistente de este. Se han configurado reglas para limitar la longitud de línea (`printWidth: 80`), definir una indentación consistente (`tabWidth: 2`), forzar el punto y coma al final de las sentencias (`semi: true`), y estandarizar el uso de comillas dobles (`singleQuote: false`), entre otras.[81]

4.2. Modelado de datos

Para la persistencia de la información se ha optado por MongoDB, una base de datos NoSQL orientada a documentos que aporta gran flexibilidad para evolucionar el esquema ágilmente. Aunque no es estrictamente necesario al usar MongoDB, se ha optado por

utilizar la librería Mongoose como ODM para definir esquemas, tipos de datos y reglas de validación para facilitar el manejo de los datos y garantizar la integridad de estos.

Así, se han estructurado los datos en diferentes colecciones, cada una de las cuales guarda registros con varios campos de diferentes tipos de datos. El modelado de datos sigue el siguiente esquema.

User	
id	ObjectId
nickname	String
email	String
password	String
masterLanguage (3)	String
learningLanguage (3)	String
role	String
image	String
bio	String
location	String
createdAt	Date

Letter	
id	ObjectId
author	String
title	String
content	String
diary	String
language	String
sharedWith	User []
deleted	Boolean
createdAt	Date

CorrectedLetter	
id	ObjectId
originalLetter	Letter
reviewer	User
sender	User
sentBack	Boolean
seen	Boolean
sharedWith	User []
deleted	Boolean
receivedAt	Date
correctedAt	Date

Follow	
id	ObjectId
user1	User
user2	User
createdAt	Date

FriendRequest	
id	ObjectId
sender	User
receiver	User
createdAt	Date

VerificationCode	
id	ObjectId
email	String
code	String
purpose	String
verified	Boolean
expiresAt	Date

Fig. 4.2. Estructura de Datos

En el diseño destaca el uso del ObjectId, un identificador hexadecimal de 12 bytes que garantiza unicidad y permite la indexación eficiente basada en *timestamps*. Además, facilita la vinculación mediante referencias entre colecciones. Cabe mencionar que, aunque internamente se usa `_id`, el backend lo transforma en `id` antes de enviarlo al frontend para ofrecer una interfaz más limpia y desacoplada.

A continuación, se describen las colecciones del sistema:

- **Usuario (User):** gestiona la identidad y el perfil del usuario. Incluye su nombre de usuario, email, rol, imagen de perfil, biografía, localidad, contraseña, los idiomas que aprende y conoce (limitados a tres entradas por optimización técnica) y la fecha de creación. Por seguridad, no se almacena el texto plano de la contraseña, sino que se utiliza la librería **bcrypt** con un factor de coste (*salting*) de 10 para cifrar la clave antes de su persistencia. El salting es un parámetro que determina la complejidad computacional del hash para dificultar ataques de fuerza bruta al aumentar el tiempo de procesamiento necesario. Se ha escogido 10 por ser el estándar actual en la industria, aquel que se considera el punto de equilibrio óptimo entre seguridad y rendimiento. Cada vez que se quiera comprobar la contraseña, se hasha la entrada con el mismo salting y se compara el resultado con el valor guardado en la base

de datos, de forma que ni el administrador ni un atacante pueda tener acceso a la contraseña.

- **Carta (Letter):** almacena los textos originales. Contiene el author (referencia al usuario creador), título, contenido, diario, idioma y fecha de creación. También incluye un array de referencias a los usuarios con que se comparte la carta (*shared-With*) y un campo *deleted* para el borrado lógico, permitiendo que los receptores conserven las correcciones que hicieron.
- **Carta Corregida (CorrectedLetter):** gestiona el flujo de revisión. Vincula la corrección a la carta original mediante la referencia *originalLetter*, además de referenciar al revisor o receptor de la carta (*reviewer*) y a su autor (*sender*). Controla los estados de envío de vuelta mediante el booleano *sentBack*, y de si el receptor vio ya la nueva carta que le llegó con el booleano *seen*, de forma que en el frontend se puedan crear etiquetas de "Nuevo" a modo de notificación. También, guarda la fecha de envío y de corrección (envío de vuelta) de la carta.
- **Amistad (Follow):** gestiona el grafo social mediante referencias a *user1* (seguidor) y *user2* (seguido), facilitando el intercambio de correcciones y el listado de amigos y usuarios sugeridos.
- **Solicitud de Amistad (FriendRequest):** capa intermedia para peticiones de amistad entre un usuario *sender* y un usuario *receiver*. Es una colección efímera: al aceptar o rechazar, el documento se elimina, creando el vínculo en la colección *Follow* en caso de aceptación.
- **Código de Verificación (VerificationCode):** entidad de apoyo para la validación de registros y recuperación de claves. Almacena el código hashado, el email del usuario implicado, un campo *verified* que indica si ya ha sido utilizado, y un campo *purpose* que indica si sirve para un registro o para una recuperación de contraseña. Además, se utiliza un índice TTL (Time To Live) en el campo *expiresAt*, que sirve para que MongoDB elimine automáticamente los códigos al cabo de 7 minutos de su creación.

4.3. Arquitectura del backend

En esta sección se expone cómo está estructurado el servidor del proyecto, los flujos lógicos que sigue, varios puntos interesantes sobre su funcionamiento, y las diferentes rutas con las que cuenta la API.

4.3.1. Estructura del backend

El servidor se organiza en módulos interconectados para ofrecer servicios robustos al frontend y facilitar un desarrollo escalable. Una petición HTTP sigue esta secuencia: el

cliente consume una ruta (endpoint), se ejecutan los middlewares de autenticación y/o carga de imágenes si corresponde, y un controlador procesa los datos interactuando con uno o varios modelos para luego devolver la respuesta que corresponda. Por ello, se distinguen los siguientes módulos principales:

- **Entrada y Configuración Inicial:** en la raíz del proyecto se encuentra el archivo `index.js`, encargado de configurar y levantar el servidor. Para ello, se conecta a la base de datos y carga los endpoints y los middlewares pertinentes.

Para que el frontend tenga acceso a la API, es esencial configurar el mecanismo de seguridad **CORS (Cross-Origin Resource Sharing)**. En una arquitectura desacoplada como la de este proyecto, el cliente y el servidor suelen operar en dominios o puertos distintos. Por seguridad, los navegadores modernos aplican la "Política de Mismo Origen" (*Same-Origin Policy*), que prohíbe que un script cargado en un origen acceda a recursos de otro distinto. Con la configuración de CORS se define una lista de dominios autorizados a acceder a la API, así como habilitar el intercambio de credenciales.

```
app.use(  
  cors({  
    origin: process.env.FRONTEND_URL || "http://localhost:3000",  
    credentials: true,  
  }),  
);
```

Fig. 4.3. Configuración del CORS en la entrada del backend

- **Lógica de negocio:** la lógica de negocio se organiza en una estructura con tres partes, basándose en el principio de separación por responsabilidades ("SoC" o Separation of Concerns), que busca dividir sistemas complejos en partes más pequeñas y manejables cada una de las cuales cubre una funcionalidad. Así, tenemos la definición de los puntos de acceso (`routes/`), la implementación de las reglas de negocio (`controllers/`) y la definición de los modelos de datos (`models/`).
- **Middlewares:** la carpeta (`middlewares/`) incluye las configuraciones de autenticación de usuario y de subida de archivos, que se explicarán en detalle más adelante.

Además, el proyecto cuenta con la carpeta `templates/`, que contiene la plantilla para enviar correos automatizados; y la carpeta `uploads/profile_pictures/`, que actúa como almacenamiento local para las imágenes de perfil de los usuarios.

Esta separación garantiza que el sistema sea escalable y fácil de mantener.

4.3.2. Emails automatizados

La función del backend encargada la generación de códigos de verificación y su envío via email al usuario, que se emplea tanto para el registro de una nueva cuenta como para la recuperación de sus credenciales en caso de olvido; es capaz de realizar sus tareas de forma automatizada gracias a un transportador que utiliza el servicio SMTP (Simple Mail Transfer Protocol, un protocolo de red para intercambiar mensajes de correo electrónico) de Gmail. Un transportador es un objeto de configuración donde se define este servicio SMTP y se inyectan las credenciales para utilizarlo, es decir, la cuenta de correo electrónico y su contraseña.

En realidad, ya no se utiliza la contraseña de la cuenta de correo directamente, Google bloqueó eso por ser una práctica poco segura y la sustituyó por Google App Password, códigos de 16 caracteres que se pueden obtener en la configuración de la cuenta de Google y que actúan como las credenciales para usar nuestra cuenta de correo de Gmail mediante código. Estas credenciales se pueden obtener solamente al tener la 2FA (2 Factor Authentication) de Google activada, y se pueden revocar de una aplicación u otra cuando se necesite; manteniendo así las credenciales primarias seguras.

Lo que se envía en estos correos automatizados es una plantilla HTML con el código de verificación generado. De esta forma, el email que reciben los usuarios es visualmente agradable y fácil de entender en un vistazo, como se muestra a continuación.

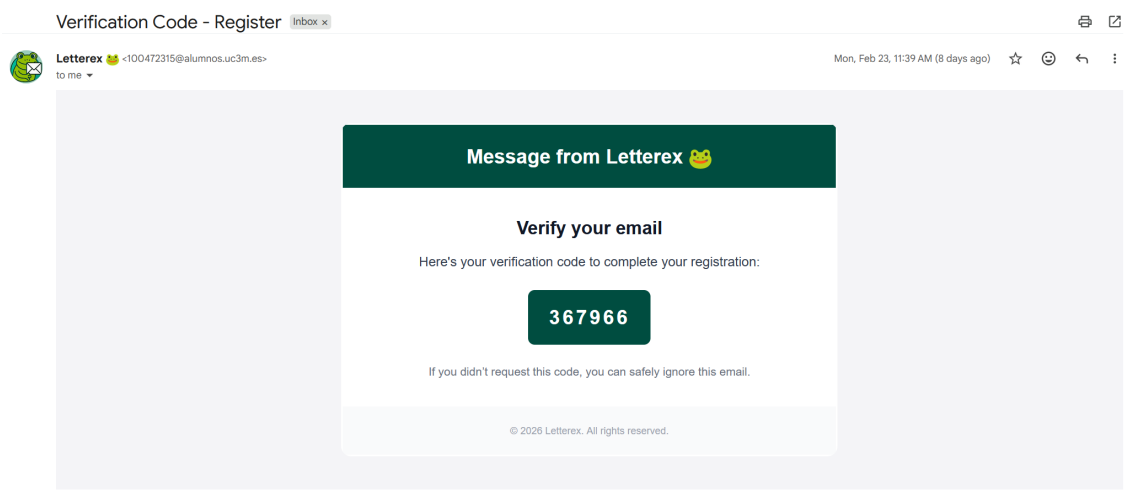


Fig. 4.4. Email automatizado

4.3.3. Mecanismo de Autenticación y Autorización

Al acceder a la plataforma introduciendo su nombre de usuario o email y contraseña, el usuario recibe un token que contiene información de la sesión como el momento en que inició, el momento de caducidad, y los datos no sensibles del usuario. Se trata de un token JWT (JSON Web Token) [82], un contenedor de información seguro y compacto

que permite verificar la identidad del cliente en cada interacción con la API.



Fig. 4.5. *JSON Web Tokens*

Este token se envía desde el backend y se almacena en el navegador mediante *cookies*, con el objetivo de que el navegador pueda recordar la sesión del usuario. Al cerrar la sesión, esta cookie se elimina. El navegador adjunta automáticamente el token a las solicitudes HTTP que envía al servidor. De esta forma, el servidor es capaz de identificar al usuario y aplicar los permisos correspondientes sin necesidad de consultar una tabla de sesiones activa en la base de datos, lo que optimiza los recursos del sistema.

Sobre el refuerzo de la seguridad de la cookie, hay dos puntos interesantes que mencionar. En primer lugar, la cookie es *HttpOnly*, es decir, no se puede leer mediante JavaScript sino que solo puede leerla el navegador. De esta forma, el token está protegido contra ataques maliciosos como **Cross-Site Scripting (XSS)**, una vulnerabilidad en la que un atacante inyecta y ejecuta código malicioso (generalmente de JavaScript) en el navegador de otros usuarios que usan la aplicación.

En segundo lugar, se ha implementado el atributo **SameSite** con el valor "Strict". Esta configuración es una medida de seguridad crítica para prevenir ataques de tipo **CSRF (Cross-Site Request Forgery)**, que aprovechan la naturaleza automática de los navegadores, dado que estos adjuntan las cookies de sesión a cualquier petición dirigida al dominio de origen. El atacante podría engañar a un usuario autenticado para que haga clic en un enlace malicioso en una pestaña diferente, y dicho enlace dispararía una petición a nuestra API (por ejemplo, para borrar una carta o cambiar la contraseña). Con la configuración de SameSite, este tipo de ataques son mitigados.

Al llegar una solicitud HTTP al sistema, si se trata de una ruta protegida se ejecuta el **Middleware de Autenticación**. Este componente intercepta la solicitud, extrae el token JWT de la cookie y verifica su validez. Si es válido, el middleware decodifica la información del usuario y la inyecta en el objeto de la petición, permitiendo que el resto del flujo identifique al autor de la acción, es decir, el usuario autenticado en la aplicación.

Estas decisiones técnicas refuerzan la robustez del sistema y aseguran que cada acción realizada en este sea legítima y provenga exclusivamente de la interacción directa del usuario con la interfaz oficial, de forma que usuario pueda acceder de forma segura a las acciones que requieren su autenticación.

4.3.4. Almacenamiento de imágenes con Multer y Cloudinary

Para la gestión de imágenes de perfil, se ha implementado una solución que combina la librería **Multer** para el procesamiento eficiente de archivos con **Cloudinary** como servicio de almacenamiento en la nube. Multer es un middleware de Express que facilita la subida de archivos mediante formularios web [83]. Por su parte, Cloudinary es un servicio de gestión de recursos digitales en la nube, en especial imágenes y vídeos, que proporciona almacenamiento persistente, optimización automática y una distribución segura y eficiente a nivel global, gracias a su infraestructura de red de entrega de contenido (CDN o *Content Delivery Network*) [84], [85], [86].

El módulo de subida de archivos se exporta como un middleware independiente, lo que permite inyectarlo exclusivamente en las rutas de la API que lo requieren, manteniendo así el principio de responsabilidad única en el código.

Su configuración se divide en tres pilares fundamentales:

- **Filtrado de seguridad:** Para prevenir la subida de archivos maliciosos (como *scripts* ejecutables disfrazados), se ha implementado un filtro por tipo MIME y extensión. El sistema solo acepta formatos de imagen estándar (JPEG, JPG, PNG), verificando tanto la extensión del archivo como su tipo MIME real para asegurar la integridad del recurso. También, se establece un máximo de tamaño del archivo de 5 MB para evitar **ataques de denegación de servicio (DoS)** por agotamiento de almacenamiento o ancho de banda.
- **Procesamiento en memoria:** Se utiliza Multer para interceptar y procesar la subida de archivos directamente en memoria con un búfer, evitando la necesidad de almacenamiento temporal en disco. Una decisión de diseño clave ha sido el renombrado del archivo por el identificador del usuario y la normalización de formato a JPG, de modo que cada usuario posea una única imagen de perfil en Cloudinary. Al combinar esta transformación con el parámetro `overwrite: true`, se reemplaza el recurso anterior automáticamente en caso de actualización, eliminando la necesidad de gestionar manualmente archivos obsoletos. Adicionalmente, el uso de `invalidate: true` garantiza que la actualización se propague correctamente al invalidar las versiones previas almacenadas en la caché de la CDN.
- **Almacenamiento en la nube:** Una vez procesado el búfer, la imagen se sube directamente a Cloudinary. El sistema almacena la URL segura retornada por Cloudinary en la base de datos, facilitando un acceso rápido sin depender del almacenamiento local del servidor. Además, Cloudinary permite aplicar transformaciones automáticas, como la limitación de dimensiones a 800x800 píxeles, para garantizar consistencia visual y reducir consumo de ancho de banda. La CDN de Cloudinary se encarga de que, cuando alguien quiera ver la imagen, esta cargue en cuestión de milisegundos.

```
const uploadedImage = await uploadBuffer(req.file.buffer, {
  public_id: profilePicturePublicId,
  use_filename: false,
  unique_filename: false,
  overwrite: true,
  invalidate: true,
  resource_type: "image",
  transformation: [
    {
      width: 800,
      height: 800,
      crop: "limit",
      format: "jpg",
    },
  ],
});
```

Fig. 4.6. Configuración del proceso de subida en Cloudinary

4.3.5. Interfaz de Programación de Aplicaciones (API): Rutas y Controladores

El backend de la plataforma actúa como una capa de servicios diseñada para que el frontend interactúe con la base de datos de forma segura y eficiente. Esta comunicación se articula mediante una **API RESTful**, que expone una serie de puntos de entrada o endpoints organizados por recursos (usuarios, cartas, correcciones, etc.).

La lógica de interacción se fundamenta en las operaciones **CRUD** (*Create, Read, Update, Delete*), que representan las acciones esenciales para la gestión de persistencia de datos. En este proyecto, dichas acciones se mapean directamente con los métodos del protocolo **HTTP**, garantizando una arquitectura estandarizada:

- **POST (Create):** Utilizado para la creación de nuevos documentos, como el registro de un usuario o el envío de una nueva carta.
- **GET (Read):** Empleado para la recuperación de información, ya sea una lista de recursos o un documento específico identificado por su ID.
- **PUT / PATCH (Update):** Destinados a la actualización de registros existentes. Se utiliza PUT para sustituciones integrales y PATCH para modificaciones parciales, como marcar una carta como "leída".
- **DELETE (Delete):** Encargado de la eliminación de recursos o, en el caso de las cartas, de la activación del borrado lógico mencionado anteriormente.

Cada ruta definida en el servidor está vinculada a un **controlador** específico, el cual contiene la lógica final para procesar la solicitud. Una vez superada la fase de autenticación, la ejecución pasa al controlador específico correspondiente, cuya función es extraer los

parámetros de la consulta, utilizar la identidad del usuario ya validada para aplicar reglas de negocio, ejecutar las operaciones correspondientes en MongoDB mediante los modelos de Mongoose, y devolver una respuesta HTTP con los datos en formato JSON y un código de éxito o error.

A continuación, se muestra una tabla con todos los métodos que se han desarrollado para esta API y de los que el frontend más adelante se servirá. Estás separados en categorías según el controlador y enrutador que se encarga de ellos: User, Letter, CorrectedLetter y Follow. Podría crearse uno nuevo para los modelos de VerificationCode y FriendRequest, pero por simplicidad se han incluido las pocas funciones de estos en el grupo de User y Follow, respectivamente. En la tabla, se marca con un asterisco los valores de entrada obligatorios.

Función	Ruta	Descripción
User Routes		
POST	/api/users/register	Registrar un nuevo usuario. Valores de entrada: nickname*, email*, password*, masterLanguage*, learningLanguage* Valores de salida: status, message, user
POST	/api/users/login	Iniciar sesión. Si las credenciales recibidas son correctas, se crea y envía la cookie de autenticación. Valores de entrada: email* (string, puede ser email o nickname), password* Valores de salida: status, userData, token, cookie authToken
POST	/api/users/logout	Cerrar la sesión. Se limpia la cookie de autenticación. Valores de entrada: - Valores de salida: status, message
GET	/api/users/profile/:id?	Obtener los datos del perfil de usuario cuyo id es especificado en la ruta, o en su defecto (es un parámetro opcional, de ahí el signo de interrogación en la ruta) del usuario autenticado. Valores de salida: status, user
GET	/api/users/list-users/:page?	Listar usuarios con paginación. Si no se recibe el parámetro page, que indica la página que se pide, se entregan los usuarios de la primera página. Valores de salida: status, users, page, itemsPerPage, totalUsers, totalPages
PUT	/api/users/update	Actualizar datos del usuario autenticado. Valores de salida: status, userData

Función	Ruta	Descripción
PUT	/api/users/change-password	Cambiar contraseña actual. Valores de salida: status, message
PUT	/api/users/profile-picture	Subir foto de perfil. Se trata de un método PUT porque no solo sube una nueva imagen al sistema de archivos sino que modifica los datos del usuario. Valores de salida: status, userId, profilePicture
DELETE	/api/users/profile-picture	Eliminar foto de perfil. Valores de salida: status, message
GET	/api/users/profile-picture/:id	Obtener foto de perfil de un usuario. En caso de que el parámetro id no esté especificado, se entrega la del usuario autenticado. Valores de salida: Archivo (imagen)
GET	/api/users/check-nickname/:nickname	Verificar disponibilidad de nickname. Valores de salida: status, inUse: boolean
GET	/api/users/check-email/:email	Verificar si email está registrado. Valores de salida: result, inUse: boolean
POST	/api/users/verification-code	Enviar código de verificación. Valores de entrada: email*, purpose Valores de salida: message
POST	/api/users/check-code	Verificar código de verificación. Valores de entrada: email*, code*, purpose Valores de salida: status, message
PUT	/api/users/reset-password	Restablecer contraseña con código de verificación. Valores de salida: status, message
DELETE	/api/users/delete-account	Eliminar cuenta de usuario. Valores de salida: status, message
Letter Routes		
POST	/api/letters/new	Guardar una carta nueva. Valores de entrada: title*, content*, language*, created_at*, diary Valores de salida: status, message, letter
GET	/api/letters/view/:letterId	Obtener información y contenido de una carta. Valores de salida: status, letter, sharedWithDetails
PUT	/api/letters/edit/:id	Editar una carta existente. Valores de salida: status, message, letter
PUT	/api/letters/edit-diary	Editar el diario de una carta. Valores de salida: status, message, letter

Función	Ruta	Descripción
DELETE	/api/letters/delete/	Eliminar una o varias cartas. Valores de salida: status, message
GET	/api/letters/list	Listar todas las cartas del usuario. Valores de salida: status, letters, count
GET	/api/letters/list/search	Buscar cartas por título. Valores de salida: status, letters
POST	/api/letters/share/:id	Compartir una carta con otro usuario (uno o varios). Valores de entrada: sharedWith* Valores de salida: {message, letter, correctedLetters}
GET	/api/letters/diaries	Listar diarios del usuario. Valores de salida: status, diaries
GET	/api/letters/count/:id?	Contar cartas por idioma. Si no se especifica id en la ruta, se cuentan las del usuario autenticado. Valores de salida: Array de {language, count}
Follow Routes		
POST	/api/follow/add/:id	Mandar una solicitud de amistad a un usuario. Valores de salida: status, message
DELETE	/api/follow/unfollow/:id	Dejar de seguir a un usuario. Valores de salida: status, message
GET	/api/follow/friends	Obtener lista de amigos. Valores de salida: status, count, friends
GET	/api/follow/non-friends	Obtener usuarios no agregados. Valores de salida: status, users, totalUsers, totalPages
GET	/api/follow/non-friends/:filter	Obtener no usuarios no agregados con un filtro por su nickname. Valores de salida: status, users, totalUsers, totalPages
GET	/api/follow/suggested	Obtener usuarios sugeridos. Son usuarios a los que el usuario autenticado no sigue y que tienen idiomas en común con este. Valores de salida: status, suggestedUsers
POST	/api/follow/request/:id	Enviar solicitud de amistad. Valores de salida: status, message, friendRequest
GET	/api/follow/requests	Listar solicitudes de amistad recibidas. Valores de salida: status, requests
POST	/api/follow/accept/:id	Aceptar solicitud de amistad. Valores de salida: status, message

Función	Ruta	Descripción
POST	/api/follow/decline/:id	Rechazar solicitud de amistad. Valores de salida: status, message
Corrected Letter Routes		
PATCH	/api/corrected-letters/send-back/:correctedLetterId	Mandar de vuelta su autor la carta corregida. Valores de salida: status, message, correctedLetter
GET	/api/corrected-letters/corrections/:original-LetterId	Obtener correcciones de una carta. Valores de salida: status, corrections
GET	/api/corrected-letters/corrected-Letter/:correctedLetterId	Obtener corrección específica por ID. Valores de salida: status, correctedLetter
GET	/api/corrected-letters/received/	Obtener cartas recibidas para corregir. Valores de salida: status, correctedLetters, count
GET	/api/corrected-letters/received/search	Buscar cartas recibidas por un filtro "search". Valores de salida: status, correctedLetters
PUT	/api/corrected-letters/update/:corrected-LetterId	Actualizar correcciones en proceso. Si la carta ya ha sido enviada de vuelta no se puede realizar esta acción. Valores de salida: status, message, correctedLetter
GET	/api/corrected-letters/count/:id?	Contar cartas corregidas por idioma. Si no se especifica el id de usuario en la ruta, se cuentan las del usuario autenticado. Valores de salida: Array de {language, count}
DELETE	/api/corrected-letters/:correctedLetterId	Eliminar carta corregida. Valores de salida: status, message

Tabla 4.1. *Rutas de la API de Letterex*

4.3.6. Técnicas de optimización

Optimización de búsqueda datos

La lógica de filtrado se procesa íntegramente en el servidor para garantizar la escalabilidad y mejorar la velocidad de la interfaz liberándola de tareas programacionales. El controlador recibe los términos de búsqueda y ejecuta una consulta optimizada en MongoDB. Esto reduce drásticamente el tamaño de los datos transferidos a través de la red en comparación con un filtrado realizado en el lado del cliente sobre la totalidad de los registros.

Además, para mejorar la usabilidad en la búsqueda y filtrado de cartas, se ha implementado

un sistema de comunicación eficiente entre el cliente y el servidor. Donde tenemos una barra de búsqueda, en lugar de realizar peticiones por cada pulsación de tecla, lo que derivaría en un consumo excesivo de recursos y latencia en la red, se ha aplicado la técnica de **debouncing**. Esta técnica sigue la siguiente lógica:

- **Control de Eventos:** Cuando el usuario escribe en el campo de búsqueda, se inicia un temporizador interno.
- **Retraso Controlado:** Si el usuario sigue escribiendo antes de que transcurran un intervalo definido de tiempo (500ms), el temporizador se reinicia.
- **Petición Asíncrona:** Solo cuando el usuario deja de escribir durante el tiempo estipulado, se dispara la solicitud formal a la API.

Así, se logra un consumo y envío de recursos mucho más eficiente.

Procesamiento Avanzado de Datos: Aggregate de MongoDB

Otra forma de optimización clave utilizada en el desarrollo del backend es el uso del Aggregate o operaciones de agregación de MongoDB [87]. Esta herramienta permite acceder a diferentes colecciones y aplicar filtrados, ordenamientos, uniones, proyecciones y paginación en una única operación atómica. Esto evita el procesar grandes volúmenes de datos en la capa de JavaScript o caer en el problema de las **N+1 consultas** (realizar múltiples peticiones a la base de datos para completar un solo registro). Así, delegamos la carga de procesamiento al motor de la base de datos, manteniendo tiempos de respuesta bajos y escalables.

Un ejemplo representativo es el flujo de obtención de cartas junto con la información de sus correcciones y usuarios asociados. La tubería de agregación de MongoDB permite resolver estas múltiples relaciones en una única consulta. El proceso se estructura en las siguientes etapas:

1. **Filtrado y paginación:** Se seleccionan únicamente las cartas del usuario autenticado que no han sido eliminadas, aplicando además ordenación cronológica y paginación. Realizar esto al inicio asegura que las operaciones posteriores, más costosas, solo se apliquen a los resultados deseados.
2. **Enriquecimiento de datos:** Se incorporan los datos de los usuarios relacionados, aquellos con los que se ha compartido la carta, mediante operaciones equivalentes a uniones (*joins*), resolviendo los identificadores almacenados en el campo `sharedWith`.
3. **Obtención de correcciones:** Se recuperan las correcciones asociadas a cada carta, filtrando para obtener solamente aquellas que han sido enviadas de vuelta al autor.

4. **Transformación de la respuesta:** Finalmente, se construye la estructura de salida combinando la información obtenida, de forma que cada carta incluye los datos necesarios para su representación en la interfaz.

Aunque construir esta lógica requiere mayor tiempo de desarrollo, el uso de la Agregación de MongoDB permite una mejora drástica en el rendimiento del sistema reduciendo el tráfico de red entre el servidor y la base de datos.

4.4. Frontend de la aplicación

El frontend ha sido desarrollado como una aplicación web moderna y responsiva, utilizando las últimas tecnologías del ecosistema React. En esta sección, se presentan las diferentes tecnologías que han intervenido en el desarrollo de esta pieza de software, la arquitectura que sigue el proyecto de frontend, las diferentes páginas que lo componen, y varias técnicas de optimización que se han empleado.

4.4.1. Stack Tecnológico

La aplicación se ha construido utilizando el siguiente stack tecnológico:

- React 19: Biblioteca principal para la construcción de interfaces de usuario basadas en componentes.
- Next.js 15: Framework de React que proporciona renderizado del lado del servidor (SSR), enrutamiento basado en archivos y optimizaciones de rendimiento automáticas.
- TypeScript: Superset tipado de JavaScript que proporciona seguridad de tipos en tiempo de compilación.
- Tailwind CSS: Framework de CSS utility-first para un desarrollo de estilos rápido y consistente. El uso de esta librería agiliza la aplicación de interfaces responsivas, adaptándose a todo tipo de dispositivos con tamaños de pantalla diversos.
- Framer Motion: Biblioteca de animaciones para crear transiciones fluidas y experiencias interactivas.[88]
- React Quill: Editor de texto enriquecido basado en Quill.js para la escritura de cartas.[89]
- Axios: Cliente HTTP para comunicación con la API backend.
- React Leaflet: Biblioteca para integración de mapas interactivos.[90]

4.4.2. Arquitectura del Frontend y Gestión del Estado

El proyecto sigue la estructura de *App Router* de Next.js 13+, que organiza las rutas de la aplicación en función de la estructura de directorios. La estructura consta de las siguientes capas:

- **app/**: Directorio principal que contiene las páginas y rutas de la aplicación. Cada subcarpeta se convierte en una ruta de la web automáticamente si incluimos dentro un archivo `page.tsx`.
- **components/**: Componentes reutilizables de la interfaz de usuario.
- **services/**: Capa de servicios que encapsula las llamadas a la API.
- **context/**: Contextos de React para gestión de estado global.
- **lib/**: Utilidades, tipos de TypeScript y constantes.
- **stylesheets/**: Aunque los estilos se aplican mayoritariamente con Tailwind CSS, para ciertos casos se utilizan estilos personalizados complementarios compartidos por varios componentes.

Para facilitar la escalabilidad y la reutilización del código, se han desarrollado **componentes reutilizables** para encapsular elementos como botones, inputs, selectores, descripciones emergentes, etc. Estos se incluyen en diferentes páginas y componentes de la aplicación para garantizar patrones de diseño consistentes, evitar la duplicación de lógica, y facilitar la implementación de cambios globales.

En cuanto a la **comunicación con el backend**, la capa de servicios encapsula toda la lógica de acceso a la API, que se implementa con Axios. La capa incluye cuatro módulos diferenciados por funcionalidad: usuarios, cartas, correcciones y amistades. Cada módulo de servicio exporta funciones asíncronas que manejan configuración de headers de autenticación, errores HTTP y de Axios, transformación de datos entre el formato de la API y el formato esperado por los componentes; y validación básica de datos antes del envío.

Respecto a la gestión del estado de la aplicación, se deben almacenar ciertos datos comunes a todas o varias páginas, así como datos individuales de cada página o componente. Para ello, se utilizan múltiples estrategias:

- **Local Storage**: Utilizado para datos no sensibles de la experiencia de usuario, como la preferencia de tema (modo claro u oscuro), que se establece inicialmente como el que tenga por defecto el usuario en su navegador, pero puede ser configurarse desde el menú desplegable de Letterex.

- **React Hooks:** Se usan funciones especiales de React como `useState`, `useEffect` o `useRef`, para controlar el estado local de los componentes. Este último se usa para implementar una caché en memoria por página, por ejemplo, guardando resultados paginados. Cuando el usuario vuelve a una página ya consultada, los datos se leen desde caché y se muestran al instante. Así, se consigue una mejora del rendimiento por la reducción de llamadas redundantes al servidor.
- **Context API:** guarda el estado del componente para gestionar diálogos modales de forma centralizada, evitando *prop drilling*, fenómeno en que los componentes se pasan datos de padres a hijos sucesivamente quedando varios hijos intermedios que no los utilizan. Con un proveedor de contexto, la app es capaz de renderizar un solo componente de diálogo que se abre y se cierra con dos funciones a las que cualquier componente puede acceder fácilmente, en este caso mediante un hook personalizado llamado `useDialog` que devuelve el contexto del diálogo. Por ejemplo, la siguiente figura muestra la función que abrirá el diálogo de error a su derecha. Esta estrategia se utiliza también para guardar los datos no sensibles del usuario autenticado.

```

1  const { openDialog, closeDialog } =
    useDialog();
2
3  openDialog({
4    type: "askConfirmation",
5    title: "Are you sure you want to
      delete the selected letters",
6    description: "This action cannot be
      undone",
7    primaryActionText: "Confirm",
8    onPrimaryAction: () => {
9      deleteSelectedLetters();
10     closeDialog();
11   }
12 });

```

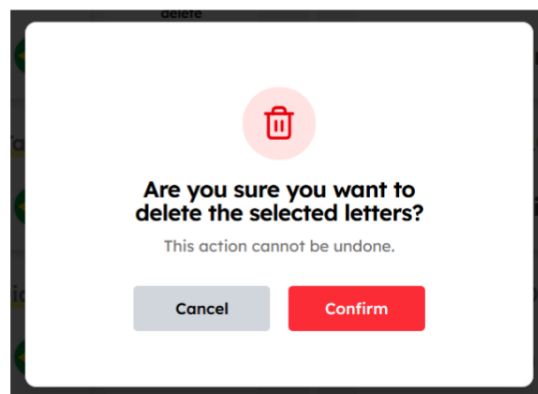


Fig. 4.7. Código de apertura del diálogo y resultado en la interfaz

4.4.3. Páginas de la aplicación

En este apartado se irán mostrando las diferentes páginas que componen la interfaz, que en total son siete: la página de inicio de sesión, la página principal, la página para la creación de una nueva carta, la página de corrección de cartas recibidas, la página de visualización de una corrección recibida, la página de amigos, y la página de perfil de usuario.

En las paginas protegidas (aquellas a las que se accede tras iniciar sesión), el componente `Sidebar` envuelve todo el contenido para proporcionar una barra lateral de navegación persistente con acceso rápido a todas las páginas de la aplicación.

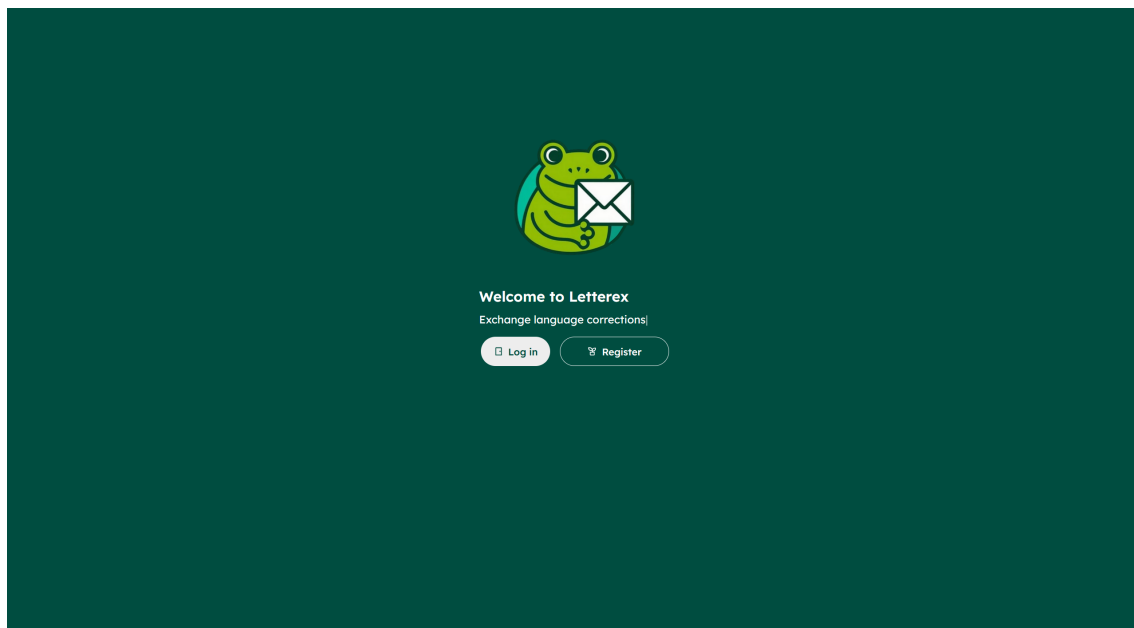


Fig. 4.8. *Página de inicio de sesión y registro*

Página de Inicio y Autenticación

La página de inicio presenta una interfaz atractiva y animada que da la bienvenida a los usuarios. Esta página utiliza el componente `AnimatedForm` que gestiona tres estados principales:

- **Bienvenida:** Presenta el propósito de la aplicación con un efecto de escritura animado (*typewriter*) que muestra frases como “Write about your passions”, “Exchange language corrections” y “Find people to share and learn”, para dar una idea general de aquello en que consiste la plataforma.
- **Inicio de sesión:** Formulario de inicio de sesión para usuarios existentes.
- **Registro:** Formulario de registro con verificación de email mediante el código enviado a su correo, selección de idioma nativo e idiomas de interés, selección de contraseña y nombre de usuario, y posibilidad de subir una foto de perfil.
- **Olvido de contraseña:** Formulario que permite resetear la contraseña del usuario mediante el código de verificación enviado a su email.

Una característica distintiva de esta página es la animación del logo implementada con `Framer Motion`, que se desliza suavemente entre diferentes posiciones según el formulario activo, proporcionando un elemento lúdico y memorable a la experiencia del usuario.

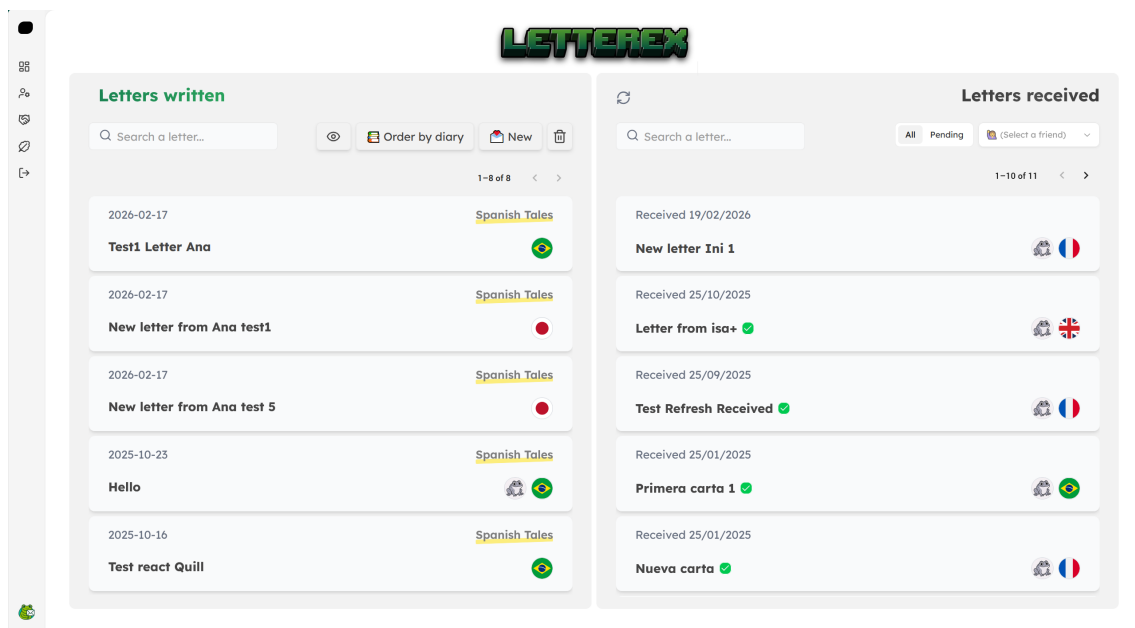


Fig. 4.9. *Página principal*

Página Principal

El dashboard principal (/homepage) es la página central de la aplicación después del inicio de sesión. Está estructurado en dos secciones principales:

1. **Letters Written:** Muestra las cartas que el usuario ha escrito, organizadas en tarjetas interactivas con un efecto de desliz que revela información adicional al interactuar, y paginadas en tandas de 5 elementos. Permite filtrar por una búsqueda, ordenar las cartas por diarios, y eliminar una o varias cartas de la lista. Clicando en la tarjeta de una carta se accede a la página de visualización o edición de esta, clicando en la tarjeta de usuario corrector que aparece al *hover* (posar el cursor sobre el elemento) se accede a la página de visualización de corrección en caso de haberla, y clicando en el botón *New* de la barra de herramientas se accede a la página para crear una nueva carta.
2. **Letters Received:** Presenta las cartas recibidas de usuarios que solicitan una corrección por parte del usuario actual, paginadas de la misma forma que las anteriores. Permite filtrar por una búsqueda, por usuario que envió la carta, y por si esa carta está pendiente de corregir (no fue corregida y enviada de vuelta todavía). Clicando en la tarjeta de una carta recibida, se accede a la página de corrección de dicha carta.

Páginas de Creación y Edición de Cartas

La página de creación de cartas (/new-letter) proporciona una interfaz completa para escribir nuevas entradas de diario. El componente implementa validación en tiempo real de

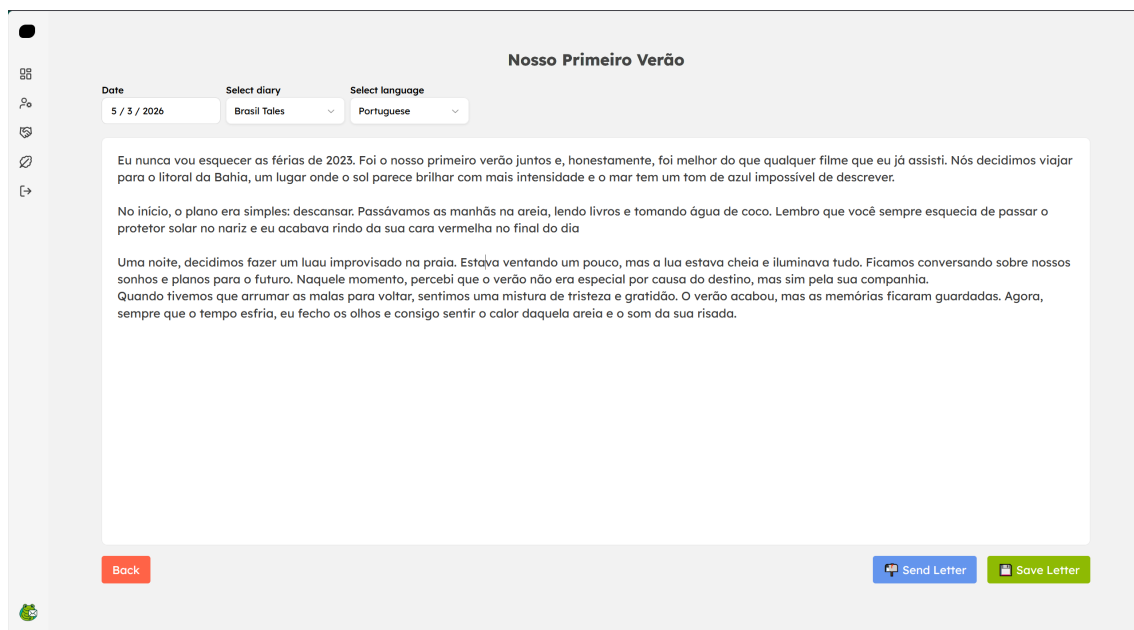


Fig. 4.10. *Página de creación de cartas*

los campos requeridos, mostrando mensajes de error específicos cuando el usuario intenta guardar sin completar todos los campos obligatorios. Los elementos principales incluyen campo de título, selectores de fecha, idioma y diario; y un editor de texto enriquecido implementado con React Quill, permite formatear texto con negrita, cursiva, listas, enlaces, etc.[89]

Al guardar la carta, se redirige al usuario a una página muy similar (`/edit-letter`), lo que permite el guardado de borradores de forma que se pueda seguir editando la carta en otro momento. Esta última página es muy similar a la de Creación de Cartas, salvo que en ella es posible enviar la carta a amigos para que la corrigan.

Página de Corrección de Cartas

Esta página (`/correct-letter`) permite a los usuarios corregir las cartas escritas por otros en idiomas que dominan.

Para añadir correcciones, el usuario debe seleccionar fragmentos específicos del texto original que considere que contienen errores o se puede matizar. El texto seleccionado se resalta visualmente, y se abre un panel de escritura para asociarlo con una corrección sugerida o un comentario explicativo. Además de las correcciones específicas, el corrector puede dejar un comentario general sobre la carta al final.

El componente `TextCorrections` renderiza de forma dinámica el texto con las correcciones aplicadas. El proceso incluye:

1. **Parseo del texto original:** Se divide el texto en fragmentos según las posiciones de corrección.

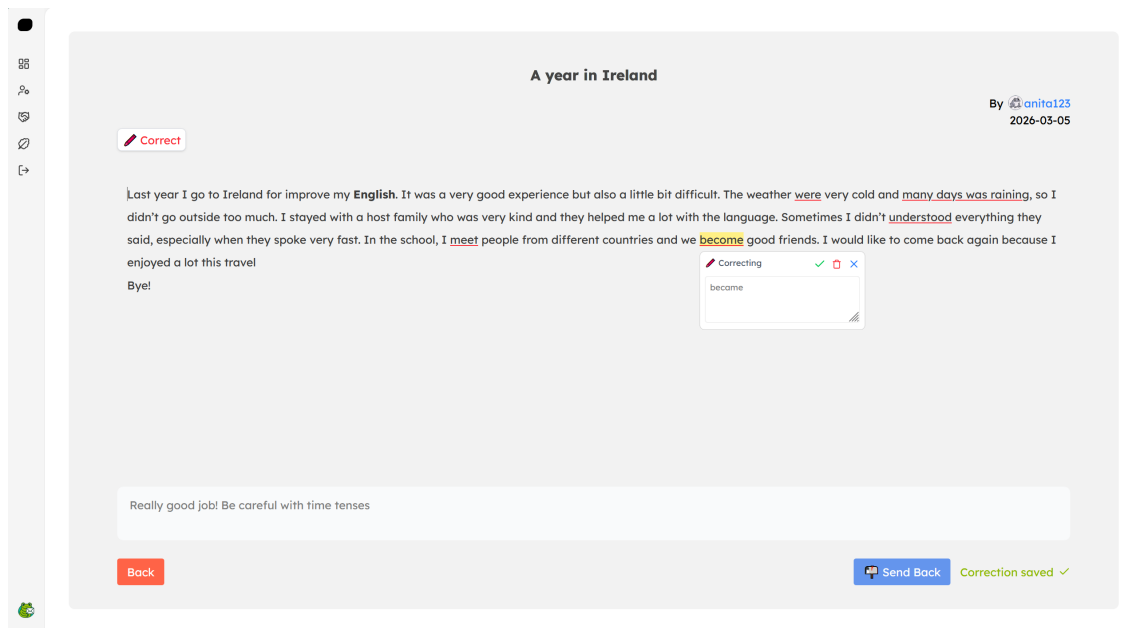


Fig. 4.11. *Página de corrección de cartas*

2. Detección de solapamientos: Antes de añadir una nueva corrección, se verifica que no se solape con correcciones existentes mediante comparación de índices.
3. Renderizado diferenciado: Cada fragmento se renderiza con estilos diferentes según sea texto sin corregir o texto corregido. Concretamente, el texto corregido va marcado en amarillo al hover y subrayado en rojo.
4. Interactividad: Los fragmentos corregidos son interactivos. Al clicar en ellos o pasar el cursor, se abre un panel donde se pueden añadir o editar las correcciones.

Para hacer esto posible, esta es la estructura de datos que se asocia a una corrección:

```
interface Correction {
    id: string;
    startIndex: number;
    endIndex: number;
    originalText: string;
    correctedText: string;
}
```

Página de Visualización de Correcciones Recibidas

La página de visualización (/view-correction) de correcciones presenta al usuario remitente las correcciones recibidas en sus cartas. Esta interfaz permite al usuario aprender de sus errores de forma efectiva, clara y visualmente agradable. La interfaz es similar a la de Corrección de Cartas, pero sin permitir editar o enviar la corrección.

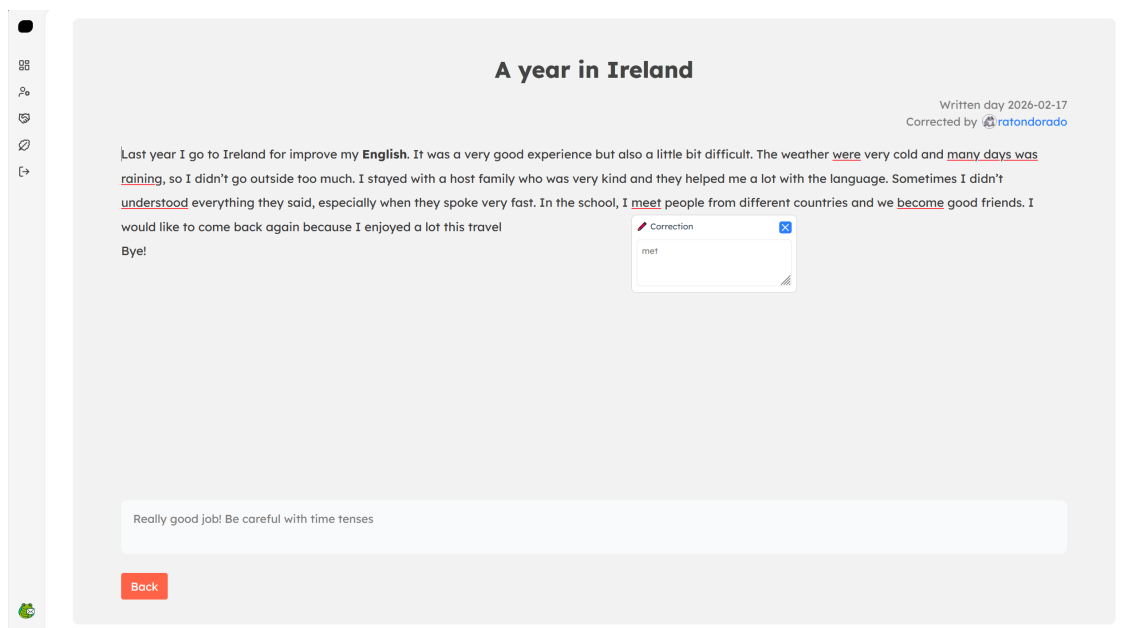


Fig. 4.12. *Página de visualización de corrección recibida*

Se muestra el texto original con las secciones corregidas resaltadas en color diferenciado. Al hacer click o hover sobre una sección corregida, se muestra un panel con la corrección sugerida. Se muestran también los comentarios generales del corrector sobre la carta, y la información del corrector (nombre e imagen de perfil), que actúa como enlace a su página de perfil.

Página de Amigos

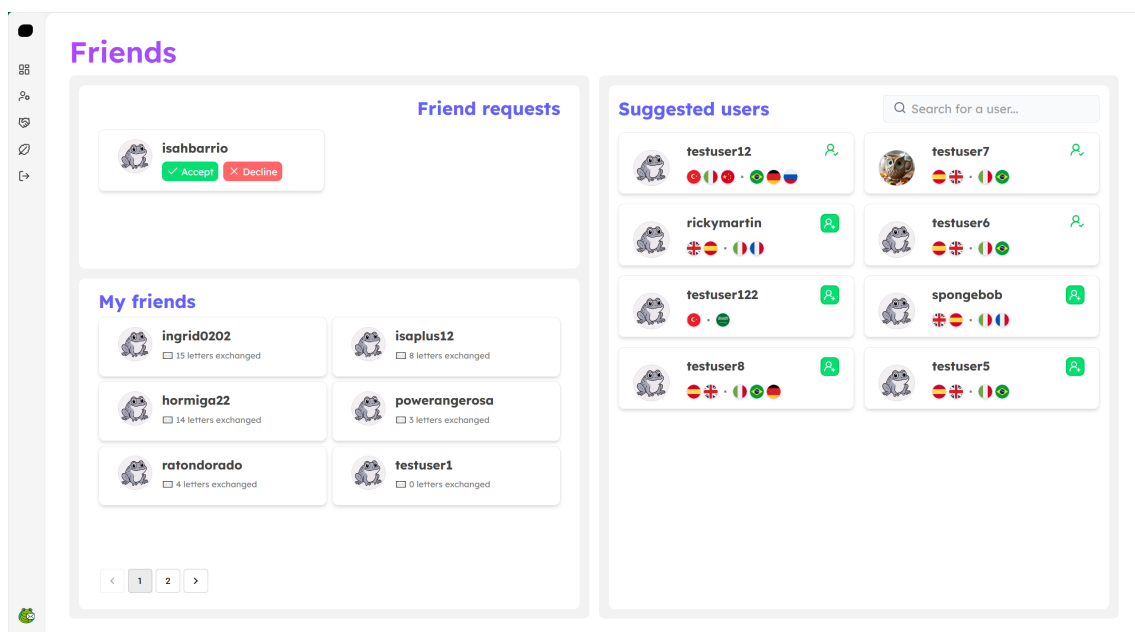


Fig. 4.13. *Página de amigos*

En la página de amigos (/friends), el usuario puede gestionar las relaciones sociales dentro de la aplicación. Se divide en tres secciones:

1. **Solicitudes de amistad:** Muestra las solicitudes pendientes recibidas, con opciones para aceptar o rechazar.
2. **Lista de amigos:** Presenta las amistades establecidas con su avatar, nombre y estadísticas como el número de cartas intercambiadas.
3. **Usuarios sugeridos:** Muestra usuarios recomendados basándose en idiomas en común, además de un buscador por nombre de usuario para poder encontrar un usuario en concreto que se desee agregar.

La página implementa paginación para todas estas listas de usuarios y solicitudes. También, al clicar cualquiera de las tarjetas de usuario, se accede a la página de perfil de este.

Página de Perfil de Usuario

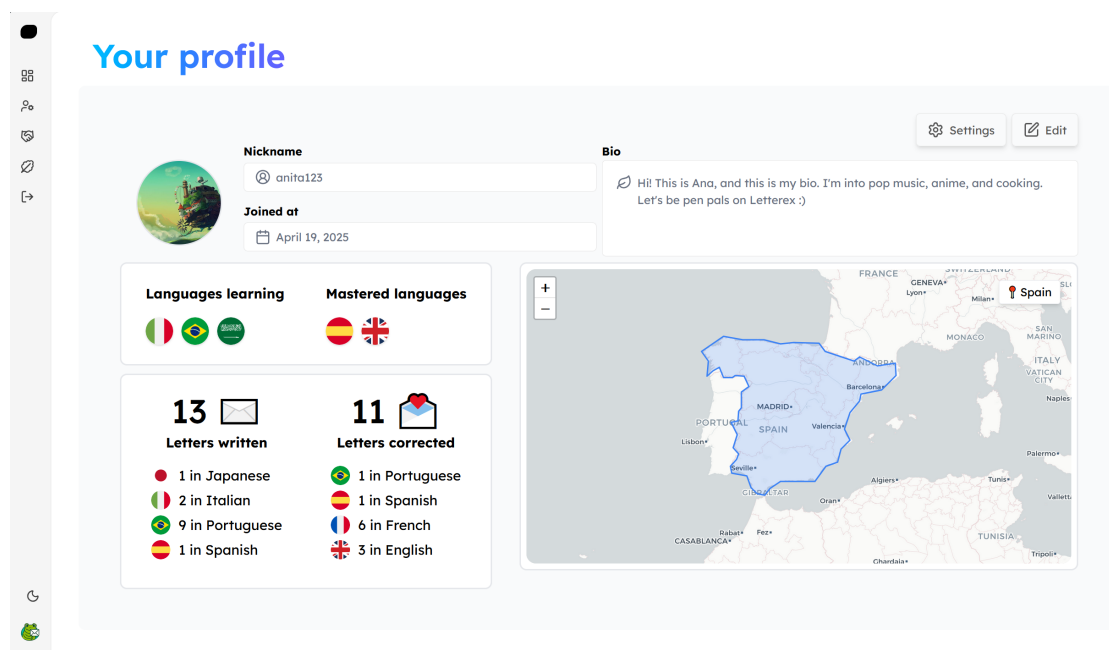


Fig. 4.14. *Página de perfil de usuario*

La página de perfil muestra información detallada sobre un usuario. Se utiliza la misma página para el perfil propio y para el de otros usuarios. La página incluye:

- Foto de perfil, nombre y biografía.
- Idiomas nativos y lenguas que está aprendiendo.
- Estadísticas de cartas escritas y corregidas en cada idioma.

- Ubicación geográfica visualizada en un mapa interactivo de React Leaflet.
- Para el perfil de usuario ajeno, un botón para dar la opción de eliminar de la lista de amigos en caso de tener a ese usuario agregado.
- Para el perfil de usuario autenticado (mi perfil), opciones de edición de la información del perfil y de configuración (cambiar contraseña o eliminar la cuenta).

4.4.4. Técnicas de Optimización

Finalmente, al igual que en el backend, en el frontend se han implementado varias técnicas de optimización. Estas técnicas se han aplicado de forma complementaria y selectiva, con el objetivo de mejorar el rendimiento de la aplicación y la experiencia de usuario en distintos niveles: carga de recursos, fluidez de la navegación, renderizado de componentes y comunicación con el backend.

1. **Importaciones Dinámicas:** Componentes pesados como el mapa de la página de perfil se cargan dinámicamente solo cuando son necesarios. Esto es posible gracias a la función `dynamic` de Next.js, diseñada para optimizar el rendimiento mediante la técnica de carga diferida (*Lazy Loading*) de los componentes, que asegura que el navegador descargue el código de estos componentes no al inicio sino en el momento en que los necesite. Mientras se carga, aparece un mensaje para informar al usuario del estado de carga.

```
const ImageUploader = dynamic(
  () => import("@/components/imageUploader"),
  { loading: () => <div>Loading...</div> }
);
```

2. **Memoización:** En la app se usan las funciones de React `useCallback` y `useMemo` para evitar recrear funciones y recalculando datos en cada renderizado.
3. **Paginación:** Las listas de registros (cartas, correcciones, amigos) incluyen paginación para reducir la cantidad de datos cargados y renderizados simultáneamente. Se cargarán más elementos del servidor solo cuando el usuario lo solicite pasando de página en la lista.
4. **Debouncing:** En campos de búsqueda y filtrado, se implementa debouncing para reducir el número de llamadas a la API, como se ha descrito en la sección anterior de arquitectura del backend.
5. **Lazy Loading de imágenes:** Las imágenes de perfil se cargan de forma diferida para mejorar el tiempo de carga inicial.

6. **Pantallas esqueleto:** Para mejorar la experiencia del usuario durante la carga de datos, se han utilizado pantallas esqueleto (*skeleton screens*). Se genera una estructura visual gris translúcida que replica la de la página real (títulos, bloques de contenido, botones...) mientras se cargan los datos del servidor. Esto reduce la sensación de espera porque el usuario ya ve dónde aparecerá cada elemento. Una vez que los datos llegan, el esqueleto se reemplaza con el contenido definitivo.



Fig. 4.15. *Ejemplo de Pantalla Esqueleto*

5. ENTORNO SOCIOECONÓMICO

En este capítulo, se presenta el entorno socioeconómico del proyecto, que comprende tanto el presupuesto para la elaboración de este Trabajo de Fin de Grado, como el impacto socio-económico del mismo en su sector.

5.1. Presupuesto

El objetivo de este apartado es detallar los aspectos económicos asociados al desarrollo del proyecto, dado que su realización conlleva una serie de costes. En este caso, se pueden dividir en dos categorías: costes de herramientas y costes humanos.

Costes de herramientas

Esta categoría incluye los recursos hardware y software que se han utilizado para el desarrollo de la aplicación y para la elaboración de la documentación. En cuanto al software, se ha hecho uso principalmente de herramientas gratuitas o con licencias proporcionadas por la universidad, por lo que este no ha supuesto un gasto real. No obstante, sí se considera el coste imputable del equipo utilizado, cuyo modelo y detalles se especificaron en el apartado de Medios empleados, situado al inicio de este documento.

Para calcular el coste imputable o amortizado del equipo, se utiliza esta fórmula:

$$Costeamortizado = \left(\frac{m}{d}\right) \cdot C \cdot u$$

Donde:

- m : número de meses en los que el equipo se utiliza en el proyecto.
- d : número de meses del periodo de depreciación total o vida útil del equipo.
- C : coste del equipo sin IVA.
- u : porcentaje de uso dedicado al proyecto.

Teniendo como valores 5 meses de uso del equipo, 60 meses como periodo de depreciación, un coste de 951€, y un porcentaje de uso dedicado de un 100%; nos queda un coste amortizado de 79,25€.

Herramienta	Descripción	Coste (€)
Ordenador portátil	Coste amortizado del equipo utilizado para desarrollo, pruebas y redacción de la memoria.	79,25
Lenguajes y frameworks de desarrollo	Tecnologías empleadas para frontend, backend y documentación (Latex) con licencia gratuita.	0,00
Visual Studio Code	Editor utilizado tanto para el código como para la documentación, con licencia gratuita.	0,00
Google Meet	Plataforma de comunicación para las reuniones de seguimiento con el tutor.	0,00
TOTAL		79,25 €

Tabla 5.1. *Costes de herramientas*

Costes humanos

El desarrollo del proyecto ha requerido una dedicación aproximada de 5 meses. Durante este periodo se distinguen dos roles principales: desarrollo y gestión del proyecto.

Por un lado, la autora del proyecto ha desempeñado el rol de desarrolladora (Ingeniera Junior), encargándose de la implementación completa del sistema. Por otro lado, también ha asumido el rol de Product Owner. Finalmente, el tutor ha actuado como Scrum Master.

A partir de la planificación realizada, se estima la siguiente dedicación:

Perfil	Horas/mes	Meses	Horas totales	€/hora	Coste (€)
Ingeniera Junior	54	5	270	30	8.100,00
Product Owner	2	5	10	40	400,00
Scrum Master	4	5	20	40	800,00
extbfTOTAL			316		9.300,00 €

Tabla 5.2. *Costes humanos*

Resumen de costes

A partir de los costes anteriores, el presupuesto total del proyecto es el siguiente:

Concepto	Coste (€)
Costes de herramientas	79,25
Costes humanos	9.300,00
Coste total (sin IVA)	9.379,25 €
IVA (21%)	1.969,64
TOTAL (IVA incluido)	11.348,89 €

Tabla 5.3. *Resumen del coste del proyecto*

En resumen, el desarrollo del proyecto presenta un coste concentrado en el esfuerzo humano que asciende a un total de 11.348,89 €.

5.2. Impacto socioeconómico

Hoy en día, el conocimiento de lenguas extranjeras se ha convertido en una competencia clave tanto a nivel académico como profesional. Es más, no son solo las instituciones académicas y empresas las entidades que optan por la internacionalización, sino que también la cultura se presenta como global en múltiples dominios como el cine, la música, el teatro, las redes sociales o el mundo de los videojuegos. Por ello, este proyecto busca poner el aprendizaje de idiomas al alcance de cualquiera con una conexión a Internet, ofreciendo una aplicación web con un enfoque personal, colaborativo y social. Así, el conjunto de sus beneficiarios incluye a cualquier persona interesada en mejorar sus habilidades lingüísticas por una amplia gama de motivos, desde académicos y profesionales hasta culturales, sociales y personales.

En primer lugar, la aplicación contribuye a crear un impacto en el **acceso a la educación en idiomas**, permitiendo a los usuarios aprender de forma libre y flexible ya que, a diferencia de los métodos tradicionales, una solución digital elimina barreras como la localización geográfica o los horarios, facilitando un aprendizaje continuo adaptado a los ritmos y disponibilidad de cada uno. La adopción de soluciones tecnológicas fomenta la innovación en el ámbito educativo, promoviendo métodos más interactivos, personalizadas y accesibles.

Además, la digitalización del aprendizaje contribuye a la **optimización de recursos**, lo que tiene un impacto positivo en el medio ambiente. Al tratarse de una aplicación software, se promueve un modelo más sostenible ya que se reducen los costes asociados a infraestructuras físicas, materiales educativos tradicionales, costes humanos adicionales y desplazamientos.

Otro impacto relevante es la mejora de la **empleabilidad** de los usuarios. En un mundo globalizado como el de hoy, el dominio de lenguas extranjeras es una habilidad cada vez más demandada en el mercado laboral, llegando a ser imprescindible en múltiples sectores como la consultoría, las ventas o la ingeniería. Esta aplicación puede contribuir al

desarrollo y mejora de estas aptitudes, lo que aumenta las oportunidades laborales de los usuarios, especialmente en sectores internacionales o tecnológicos, que abarcan cada vez más espacio en el mercado.

Asimismo, la aplicación puede favorecer la **inclusión social** y la **integración cultural**. Por un lado, las competencias en idiomas permiten a las personas comunicarse en entornos diversos, facilitando la movilidad internacional, la integración de personas extranjeras y el intercambio cultural. Por otro lado, esta plataforma no solo contribuye a mejorar dichas competencias para sus usuarios, sino que es capaz de crear en si misma una comunidad que conecta gente de diferentes localidades, nacionalidades y contextos, y que, al estar enfocada a la redacción de temas libres en lugar de seguir un método rígido y cerrado, permite un intercambio real y personal de experiencias e ideas.

Desde el punto de vista económico, el desarrollo de una aplicación educativa no solo genera valor para los usuarios, sino que también pueden derivar en una **oportunidad de emprendimiento** con el desarrollo de un modelo de negocio digital. Esto, a su vez, contribuye al avance del cada vez más en auge sector "EdTech" o "Educational Technology", en español Tecnología Educativa, que está generando múltiples oportunidades de mercado.

En cuanto al impacto tecnológico, la realización de este proyecto ha implicado la adquisición y aplicación de conocimientos en áreas como el desarrollo de software, diseño de interfaces, programación de APIs y gestión de sistemas, lo que ha contribuido enormemente a mi formación en competencias altamente demandadas en el mercado laboral actual.

En resumen, el desarrollo de una aplicación para el aprendizaje de idiomas tiene un impacto socioeconómico positivo al mejorar la accesibilidad de la educación, aumentar la empleabilidad, favorecer la inclusión social, impulsar el sector tecnológico y contribuir a la transformación digital del aprendizaje. Estos beneficios repercuten tanto en los usuarios individuales como en la sociedad en su conjunto.

6. CONCLUSIONES

Para culminar este Trabajo de Fin de Grado, se revisará el cumplimiento de los objetivos planteados en la introducción y se presentarán varias líneas de trabajo futuro y propuestas de mejora, y una conclusión final.

6.1. Cumplimiento de Objetivos

En la siguiente tabla se resume el cumplimiento de los objetivos establecidos y la forma en que se han alcanzado.

Objetivo	Cómo se ha conseguido	Estado
Sistema de autenticación	Registro con validación de email, inicio de sesión seguro con JWT, y recuperación de contraseña	✓
Bandeja de cartas enviadas y recibidas	Desarrollo de interfaces con filtrado por título, idioma, diario y visualización del estado de corrección	✓
Interfaz para crear, editar y enviar cartas	Diseño de editor intuitivo para especificar título, contenido, idioma, fecha y diario, guardar borradores y compartir con contactos	✓
Página de corrección de cartas	Página especializada con editor enriquecido, herramientas de adición y edición de correcciones y reenvío de cartas corregidas	✓
Sistema de solicitudes de amistad	Implementación de solicitudes de amistad, gestión de contactos, buscador de usuarios y algoritmo de recomendación de usuarios	✓
Página de perfil	Edición de datos personales, cambio de contraseña con verificación, eliminación de cuenta y carga segura de imagen	✓
Arquitectura limpia y escalable	Patrones de diseño reconocidos, separación de responsabilidades y optimización del frontend y del backend	✓
Metodología Scrum	Creación de historias de usuario, pruebas de validación, matriz de trazabilidad, y planificación por sprints, usando tableros de Trello	✓
Documentación completa	Redacción de todas las secciones de la presente memoria con contenido y forma adecuados	✓

Tabla 6.1. *Cumplimiento de objetivos*

Todos los objetivos técnicos, de gestión y de documentación establecidos han sido alcanzados satisfactoriamente.

6.2. Líneas de Trabajo Futuro y Propuestas de Mejora

Tras la culminación de la fase de desarrollo actual, se han identificado diversas áreas que permitirían escalar la plataforma y mejorar la experiencia de usuario final. Estas propuestas se detallan a continuación:

- **Autenticación de Terceros:** Con el fin de reducir la fricción en el registro de nuevos usuarios, se plantea la implementación de Google Sign-In. Esto delegaría la seguridad de la autenticación a un proveedor de identidad consolidado, mejorando la confianza del usuario y la tasa de conversión de la plataforma.
- **Enriquecimiento de la Mensajería:** Evolucionar el sistema de cartas permitiendo adjuntar archivos de audio o imágenes dentro del cuerpo del mensaje. Esto potenciaría el aprendizaje lingüístico al permitir que los usuarios practiquen no solo la expresión escrita, sino también la comprensión auditiva y la pronunciación.
- **Módulo de Comunidad:** Adición de una página de comunidad donde los usuarios puedan crear grupos para compartir cartas, temas sobre los que escribir, foros, canciones, etcétera. Esto podría mejorar la experiencia del usuario al tener acceso a más funcionalidades y a una conexión más grupal con otros usuarios; aunque antes de su implementación se debería realizar un estudio para determinar si existiría un interés real en esta nueva página por parte de los usuarios de la app.
- **Mejor comunicación de la propuesta de valor:** Mejora de la página de inicio, añadiendo una sección que de forma más detallada y visual el propósito de la aplicación y las funcionalidades que incluye. El objetivo es que el usuario no registrado obtenga una idea clara y precisa de experiencia que ofrece la plataforma.

6.3. Conclusión final

La realización de este Trabajo de Fin de Grado ha representado un hito fundamental en mi formación académica. El desarrollo de una plataforma full-stack para el intercambio lingüístico me ha permitido aplicar de manera transversal y práctica conocimientos teóricos adquiridos durante el grado, y profundizar en tecnologías de interés personal y profesional.

En el reto de construir un ecosistema de software completo, robusto y funcional desde sus cimientos, la consolidación de habilidades técnicas ha ido de la mano con el aprendizaje autónomo. La investigación del marco legislativo pertinente y la implementación de medidas de seguridad avanzadas para la protección contra ciberataques me ha llevado a un conocimiento más profundo y práctico los estándares actuales de la industria, comprendiendo que el desarrollo de software no solo consiste en crear funcionalidad, sino en garantizar la integridad y privacidad de los datos del usuario.

Asimismo, este trabajo ha sido un ejercicio de resiliencia y resolución de problemas. Encontrarme con desafíos técnicos, como la gestión de archivos estáticos con Multer, la autenticación mediante tokens JWT o la sincronización del estado global en Next.js, me ha permitido desarrollar una mentalidad analítica para buscar soluciones eficientes y documentadas. Esta capacidad de "aprender a aprender" es, sin duda, la competencia más valiosa que me llevo para mi futura etapa profesional.

Desde el punto de vista de la gestión, la aplicación de la metodología Scrum y el uso de herramientas como Trello han sido determinantes. La necesidad de reajustar los sprints y organizar las tareas de forma estructurada, funcional y centrada en el aporte de valor me ha proporcionado una visión realista de la ingeniería de software: un equilibrio constante entre requisitos, tiempo y calidad. Se optó por el estudio y uso de esta metodología sobretudo para el aprendizaje y acercamiento a formas de trabajo reales de la industria. Sin embargo, se ha observado que no es el mejor enfoque para proyectos desarrollados principalmente por una sola persona, dado que está más orientado a entornos colaborativos con múltiples roles y equipos.

En definitiva, este proyecto no solo ha validado mi capacidad técnica como futura profesional, sino que ha forjado mi identidad como ingeniera informática, dotándome de la confianza y las herramientas necesarias para abordar retos tecnológicos reales de la industria.

BIBLIOGRAPHY

- [1] Parlamento Europeo. “Reglamento (ue) 2016/679 del parlamento europeo y del consejo: Reglamento general de protección de datos (rgpd).” Accedido: 2026-05-02. [Online]. Available: <https://www.boe.es/buscar/doc.php?id=DOUE-L-2016-80807>
- [2] Parlamento Español. “Ley orgánica 3/2018, de 5 de diciembre, de protección de datos personales y garantía de los derechos digitales (lopdgdd).” Accedido: 2026-05-04. [Online]. Available: <https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673>
- [3] Parlamento Español. “Ley 34/1988, de 11 de noviembre, sobre la sociedad de la información y de comercio electrónico (Lssice).” Accedido: 2026-05-06. [Online]. Available: <https://www.boe.es/buscar/act.php?id=BOE-A-2002-13758>
- [4] Parlamento Europeo y Consejo de la Unión Europea. “Reglamento (ue) 2024/2847 del parlamento europeo y del consejo, de 23 de octubre de 2024, relativo a los requisitos horizontales de ciberseguridad para productos con elementos digitales (ley de ciberresiliencia).” Accedido: 2026-05-08. [Online]. Available: <https://www.boe.es/buscar/doc.php?id=DOUE-L-2024-81720>
- [5] Parlamento Español. “Texto refundido de la ley de propiedad intelectual.” Accedido: 2026-05-10. [Online]. Available: <https://boe.es/buscar/act.php?id=BOE-A-1996-8930>
- [6] Microsoft. “Visual studio code.” Accedido: 2026-05-12. [Online]. Available: <https://code.visualstudio.com/>
- [7] Mozilla Foundation. “Firefox developer edition.” Accedido: 2026-05-14. [Online]. Available: <https://www.mozilla.org/es/firefox/developer/>
- [8] MongoDB Inc. “Mongodb compass.” Accedido: 2026-05-16. [Online]. Available: <https://www.mongodb.com/products/compass>
- [9] Postman Inc. “Postman.” Accedido: 2026-05-18. [Online]. Available: <https://www.postman.com/>
- [10] GitHub Inc. “Github.” Accedido: 2026-05-20. [Online]. Available: <https://github.com/>
- [11] Google LLC. “Google meet.” Accedido: 2026-05-24. [Online]. Available: <https://meet.google.com/>
- [12] Atlassian. “Trello.” Accedido: 2026-05-22. [Online]. Available: <https://trello.com/>

- [13] Universidad Europea. “Qué es html y cuáles son sus características principales.” Accedido: 2026-01-03. [Online]. Available: <https://universidadeuropea.com/blog/que-es-html/>
- [14] Hostinger. “Tipos de estilos css: Cómo aplicar css en html.” Accedido: 2026-01-05. [Online]. Available: <https://www.hostinger.com/mx/tutoriales/tipos-de-estilos-css>
- [15] Tailwind Labs. “Tailwind css documentation.” Accedido: 2026-01-07. [Online]. Available: <https://tailwindcss.com/docs>
- [16] MDN Web Docs. “Javascript.” Accedido: 2026-01-09. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- [17] GeeksforGeeks. “Difference between ajax and fetch api.” Accedido: 2026-02-15. [Online]. Available: <https://www.geeksforgeeks.org/javascript/difference-between-ajax-and-fetch-api/>
- [18] 4Geeks Academy. “¿qué es typescript?” Accedido: 2026-01-11. [Online]. Available: <https://4geeks.com/es/lesson/que-es-typescript>
- [19] TypeScript Team. “Typescript.” Accedido: 2026-01-13. [Online]. Available: <https://www.typescriptlang.org/>
- [20] The Bridge. “Framework javascript: Qué es, ventajas y cuáles son los más usados.” Accedido: 2026-01-15. [Online]. Available: <https://thebridge.tech/blog/framework-javascript/>
- [21] Meta Open Source. “React documentation.” Accedido: 2026-01-17. [Online]. Available: <https://react.dev/>
- [22] EBAC. “Qué es react.” Accedido: 2026-01-15. [Online]. Available: <https://ebac.mx/blog/que-es-react>
- [23] Vercel. “Next.js documentation.” Accedido: 2026-01-15. [Online]. Available: <https://nextjs.org/docs>
- [24] freeCodeCamp. “Manual de next.js.” Accedido: 2026-01-15. [Online]. Available: <https://www.freecodecamp.org/espanol/news/manual-de-next-js/#principales>
- [25] Vue.js. “Vue.js guide: Introduction.” Accedido: 2026-01-15. [Online]. Available: <https://vuejs.org/guide/introduction.html>
- [26] OpenWebinars. “¿qué es vue.js y qué lo diferencia de otros frameworks?” Accedido: 2026-01-15. [Online]. Available: <https://openwebinars.net/blog/que-es-vue-js-y-que-lo-diferencia-de-otros-frameworks/>
- [27] Google. “Angular overview.” Accedido: 2026-01-15. [Online]. Available: <https://angular.dev/overview>

- [28] Hiberus. “¿qué es angular y para qué sirve?” Accedido: 2026-01-15. [Online]. Available: <https://www.hiberus.com/crecemos-contigo/que-es-angular-y-para-que-sirve/>
- [29] OpenJS Foundation. “Node.js documentation.” Accedido: 2026-01-15. [Online]. Available: <https://nodejs.org/en/docs>
- [30] Express. “Express - fast, unopinionated, minimalist web framework for node.js.” Accedido: 2026-01-15. [Online]. Available: <https://expressjs.com/>
- [31] Evolve. “Guía definitiva de node.js — introducción para desarrolladores.” Accedido: 2026-01-15. [Online]. Available: <https://evolve.es/blog/desarrollo-web/guia-definitiva-de-node-js-introduccion-para-desarrolladores/>
- [32] Python Software Foundation. “The python tutorial.” Accedido: 2026-01-15. [Online]. Available: <https://docs.python.org/3/>
- [33] Django Software Foundation. “Django documentation.” Accedido: 2026-01-15. [Online]. Available: <https://docs.djangoproject.com/en/stable/>
- [34] Oracle. “Learn java.” Accedido: 2026-01-15. [Online]. Available: <https://dev.java/learn/>
- [35] VMware. “Spring boot.” Accedido: 2026-01-15. [Online]. Available: <https://spring.io/projects/spring-boot>
- [36] The PHP Group. “Php manual.” Accedido: 2026-01-15. [Online]. Available: <https://www.php.net/docs.php>
- [37] Laravel. “Laravel documentation.” Accedido: 2026-01-15. [Online]. Available: <https://laravel.com/docs>
- [38] Ruby Language. “Ruby documentation.” Accedido: 2026-01-15. [Online]. Available: <https://www.ruby-lang.org/en/documentation/>
- [39] Ruby on Rails. “Ruby on rails guides.” Accedido: 2026-01-15. [Online]. Available: <https://guides.rubyonrails.org/>
- [40] Go. “Go documentation.” Accedido: 2026-01-15. [Online]. Available: <https://go.dev/doc/>
- [41] Gin-Gonic. “Gin web framework documentation.” Accedido: 2026-01-15. [Online]. Available: <https://gin-gonic.com/docs/>
- [42] Wikipedia. “Go (lenguaje de programación).” Accedido: 2026-01-15. [Online]. Available: https://es.wikipedia.org/wiki/Go_%28lenguaje_de_programaci%C3%B3n%29
- [43] Axios. “Axios introduction.” Accedido: 2026-01-15. [Online]. Available: <https://axios-http.com/docs/intro>
- [44] Arsys. “Axios: Introducción y usos.” Accedido: 2026-01-15. [Online]. Available: <https://www.arsys.es/blog/axios>

- [45] MDN Web Docs. “Fetch api.” Accedido: 2026-01-15. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- [46] LogRocket Blog. “Axios vs fetch (2025).” Accedido: 2026-01-15. [Online]. Available: <https://blog.logrocket.com/axios-vs-fetch-2025/>
- [47] Postman. “Postman documentation.” Accedido: 2026-01-15. [Online]. Available: <https://learning.postman.com/docs/>
- [48] Imagina Formacion. “¿qué es postman y para qué sirve?” Accedido: 2026-01-15. [Online]. Available: <https://imaginaformacion.com/tutoriales/que-es-postman-y-para-que-sirve>
- [49] IBM. “What are rest apis?” Accedido: 2026-06-09. [Online]. Available: <https://www.ibm.com/es-es/think/topics/rest-apis>
- [50] Insomnia. “Insomnia documentation.” Accedido: 2026-01-15. [Online]. Available: <https://docs.insomnia.rest/>
- [51] NutterTools. “Insomnia — best tools listing.” Accedido: 2026-01-15. [Online]. Available: <https://best-tools-for.nuttertools.dev/es/best-tools-for-software-engineers/insomnia>
- [52] cURL. “Curl documentation.” Accedido: 2026-01-15. [Online]. Available: <https://curl.se/docs/>
- [53] Hostinger. “Comando curl: Guía.” Accedido: 2026-01-15. [Online]. Available: <https://www.hostinger.com/es/tutoriales/comando-curl>
- [54] Amazon Web Services. “The difference between relational and non-relational databases.” Accedido: 2026-01-15. [Online]. Available: <https://aws.amazon.com/es/compare/the-difference-between-relational-and-non-relational-databases/>
- [55] Oracle. “Mysql documentation.” Accedido: 2026-01-15. [Online]. Available: <https://dev.mysql.com/doc/>
- [56] OpenWebinars. “¿qué es mysql?” Accedido: 2026-01-15. [Online]. Available: <https://openwebinars.net/blog/que-es-mysql/>
- [57] PostgreSQL Global Development Group. “Postgresql documentation.” Accedido: 2026-01-15. [Online]. Available: <https://www.postgresql.org/docs/>
- [58] Hostinger. “¿qué es postgresql y sus características?” Accedido: 2026-01-15. [Online]. Available: <https://www.hostinger.com/es/tutoriales/que-es-postgresql-y-sus-caracteristicas>
- [59] MariaDB Foundation. “Mariadb documentation.” Accedido: 2026-01-15. [Online]. Available: <https://mariadb.com/kb/en/documentation/>
- [60] Wikipedia. “Mariadb.” Accedido: 2026-01-15. [Online]. Available: <https://es.wikipedia.org/wiki/MariaDB>

- [61] MongoDB. “Mongodb documentation.” Accedido: 2026-01-15. [Online]. Available: <https://www.mongodb.com/docs/>
- [62] Wikipedia. “Mongodb.” Accedido: 2026-01-15. [Online]. Available: <https://es.wikipedia.org/wiki/MongoDB>
- [63] Google. “Firebase documentation.” Accedido: 2026-01-15. [Online]. Available: <https://firebase.google.com/docs>
- [64] Digital55. “¿qué es firebase? funcionalidades y ventajas.” Accedido: 2026-01-15. [Online]. Available: <https://digital55.com/blog/que-es-firebase-funcionalidades-ventajas-conclusiones/>
- [65] Apache Software Foundation. “Apache cassandra documentation.” Accedido: 2026-01-15. [Online]. Available: <https://cassandra.apache.org/doc/latest/>
- [66] KeepCoding. “¿qué es apache cassandra y cómo funciona?” Accedido: 2026-01-15. [Online]. Available: <https://keepcoding.io/blog/que-es-apache-cassandra-y-como-funciona/>
- [67] *Modelo en cascada*, Referencia de metodología.
- [68] *Metodología incremental*, Referencia de metodología.
- [69] *Rad: Desarrollo rápido de aplicaciones*, Referencia de metodología.
- [70] Agile Alliance. “Manifesto for agile software development.” Accedido: 2026-05-02. [Online]. Available: <https://agilemanifesto.org/>
- [71] Scrum.org. “The scrum guide.” Accedido: 2026-05-04. [Online]. Available: <https://scrumguides.org/>
- [72] *Extreme programming (xp)*, Referencia de metodología.
- [73] *Kanban guide*, Referencia de metodología.
- [74] *Lean methodology*, Referencia de metodología.
- [75] HiNative. “Hinative.” Accedido: 2026-05-08. [Online]. Available: <https://hinative.com/>
- [76] Tandem. “Tandem.” Accedido: 2026-05-12. [Online]. Available: <https://www.tandem.net/>
- [77] Grammarly. “Grammarly.” Accedido: 2026-05-14. [Online]. Available: <https://www.grammarly.com/>
- [78] Duolingo. “Duolingo.” Accedido: 2026-05-16. [Online]. Available: <https://www.duolingo.com/>
- [79] Atlassian. “User stories - project management guide.” Accedido: 2026-05-06. [Online]. Available: <https://www.atlassian.com/es/agile/project-management/user-stories>
- [80] ESLint. “Eslint documentation.” Accedido: 2026-01-19. [Online]. Available: <https://eslint.org/>

- [81] Prettier. “Prettier documentation.” Accedido: 2026-01-21. [Online]. Available: <https://prettier.io/>
- [82] Auth0. “Jwt.io - json web tokens introduction and debugger.” Accedido: 2026-05-26. [Online]. Available: <https://www.jwt.io/>
- [83] npm. “Multer.” Accedido: 2026-05-28. [Online]. Available: <https://www.npmjs.com/package/multer>
- [84] Cloudinary. “Cloudinary documentation.” Accedido: 2026-05-30. [Online]. Available: <https://cloudinary.com/documentation>
- [85] Cloudinary. “Image upload api reference.” Accedido: 2026-05-03. [Online]. Available: https://cloudinary.com/documentation/image_upload_api_reference
- [86] Cloudflare. “What is a cdn?” Accedido: 2026-05-05. [Online]. Available: <https://www.cloudflare.com/learning/cdn/what-is-a-cdn/>
- [87] MongoDB. “Aggregation — mongodb manual.” Accedido: 2026-05-07. [Online]. Available: <https://www.mongodb.com/es/docs/manual/aggregation/>
- [88] Motion. “Motion documentation.” Accedido: 2026-01-23. [Online]. Available: <https://motion.dev/>
- [89] Quill. “Quill react playground.” Accedido: 2026-01-27. [Online]. Available: <https://quilljs.com/playground/react>
- [90] React Leaflet. “React leaflet documentation.” Accedido: 2026-01-25. [Online]. Available: <https://react-leaflet.js.org/>